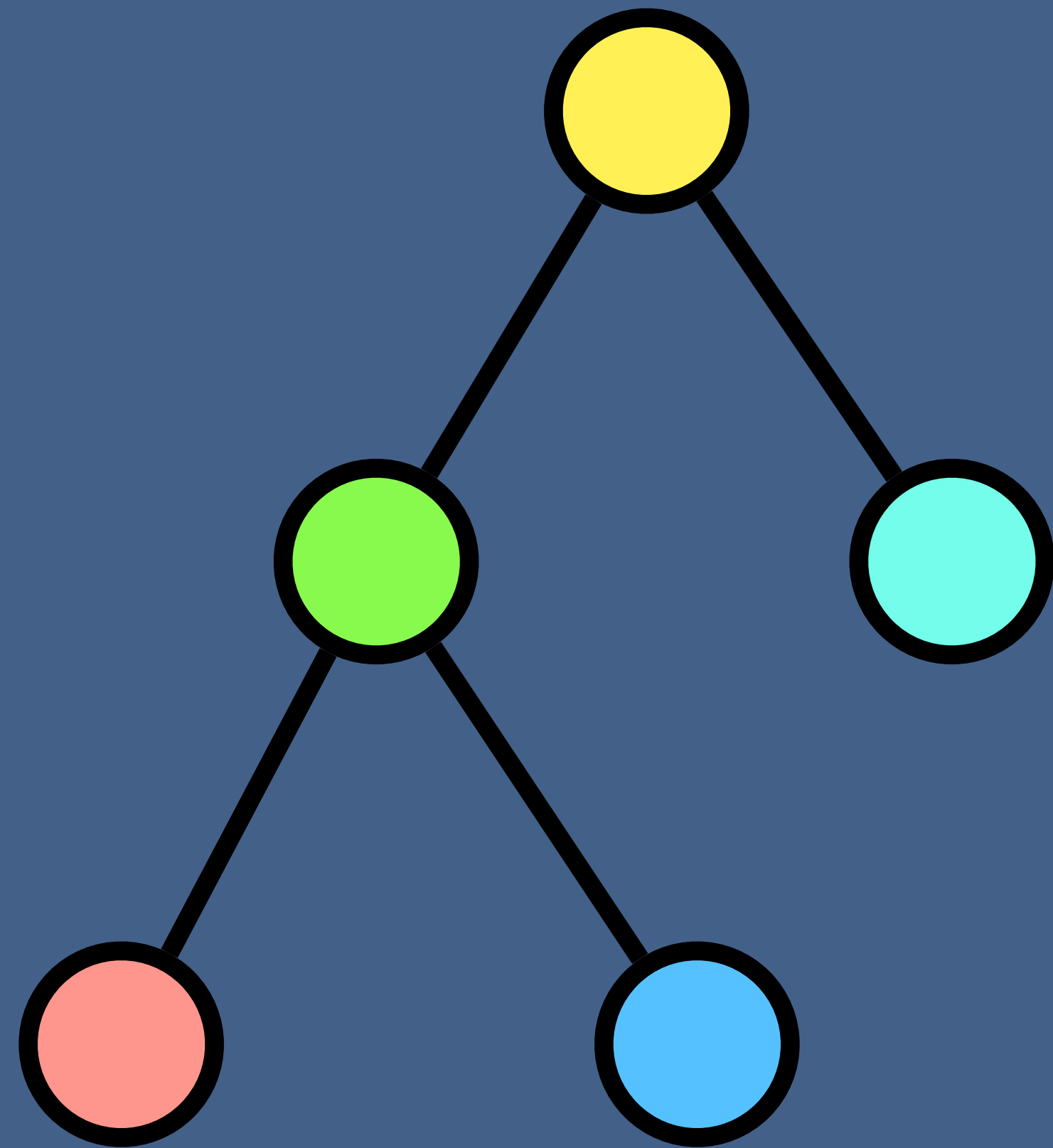
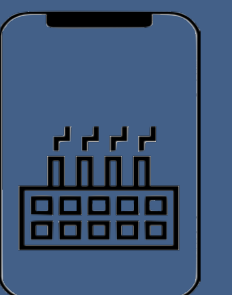
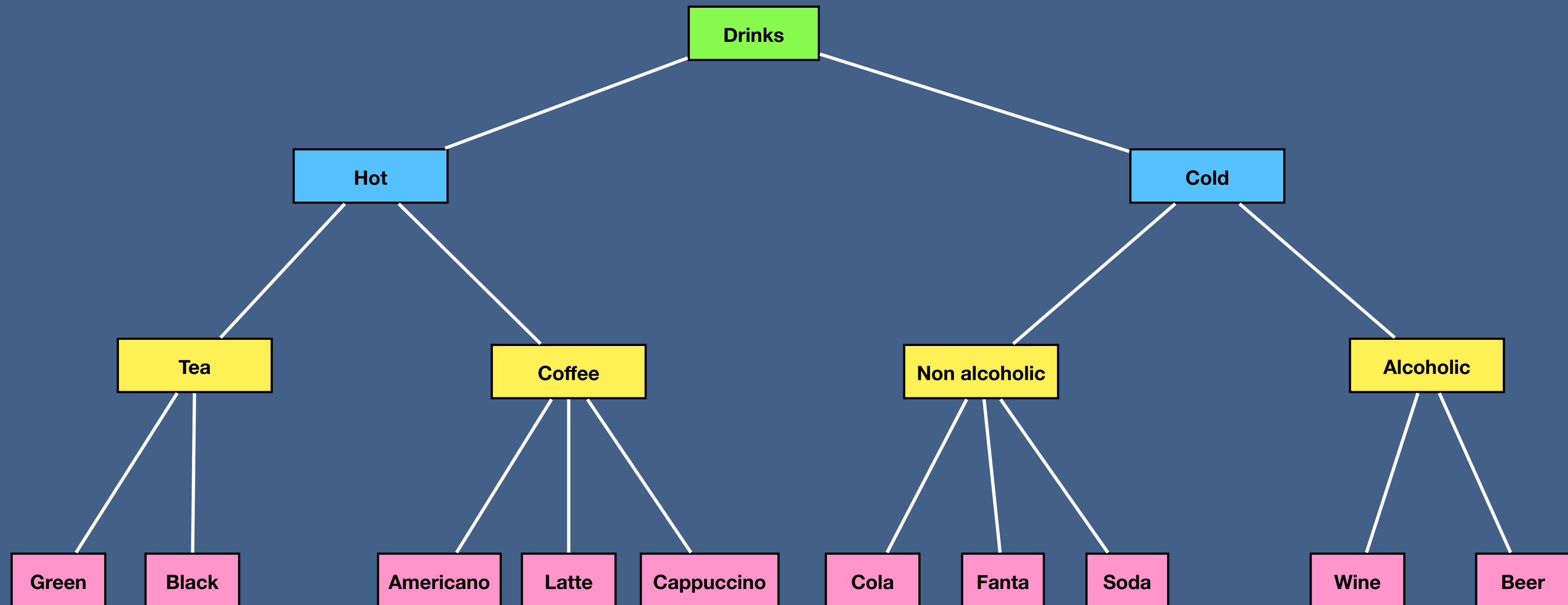


Tree / Binary Tree



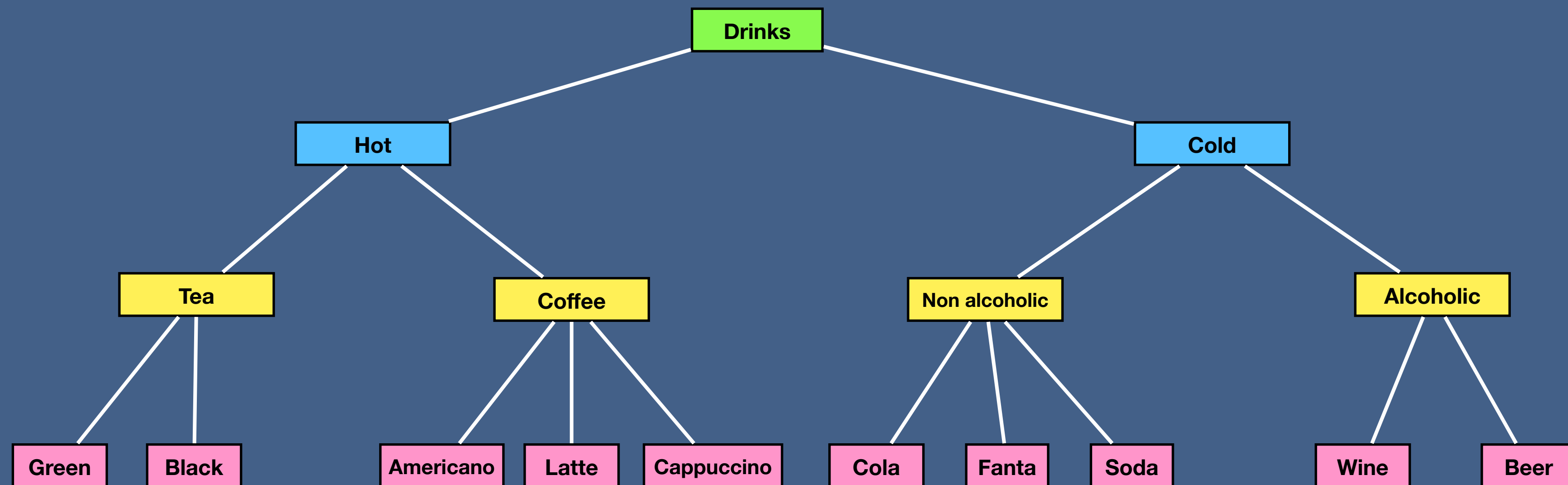
What is a Tree?

A tree is a nonlinear data structure with hierarchical relationships between its elements without having any cycle, it is basically reversed from a real life tree.



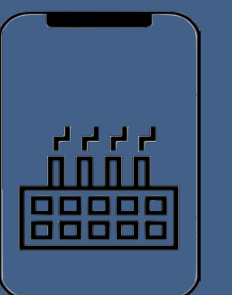
What is a Tree?

A tree is a nonlinear data structure with hierarchical relationships between its elements without having any cycle, it is basically reversed from a real life tree.



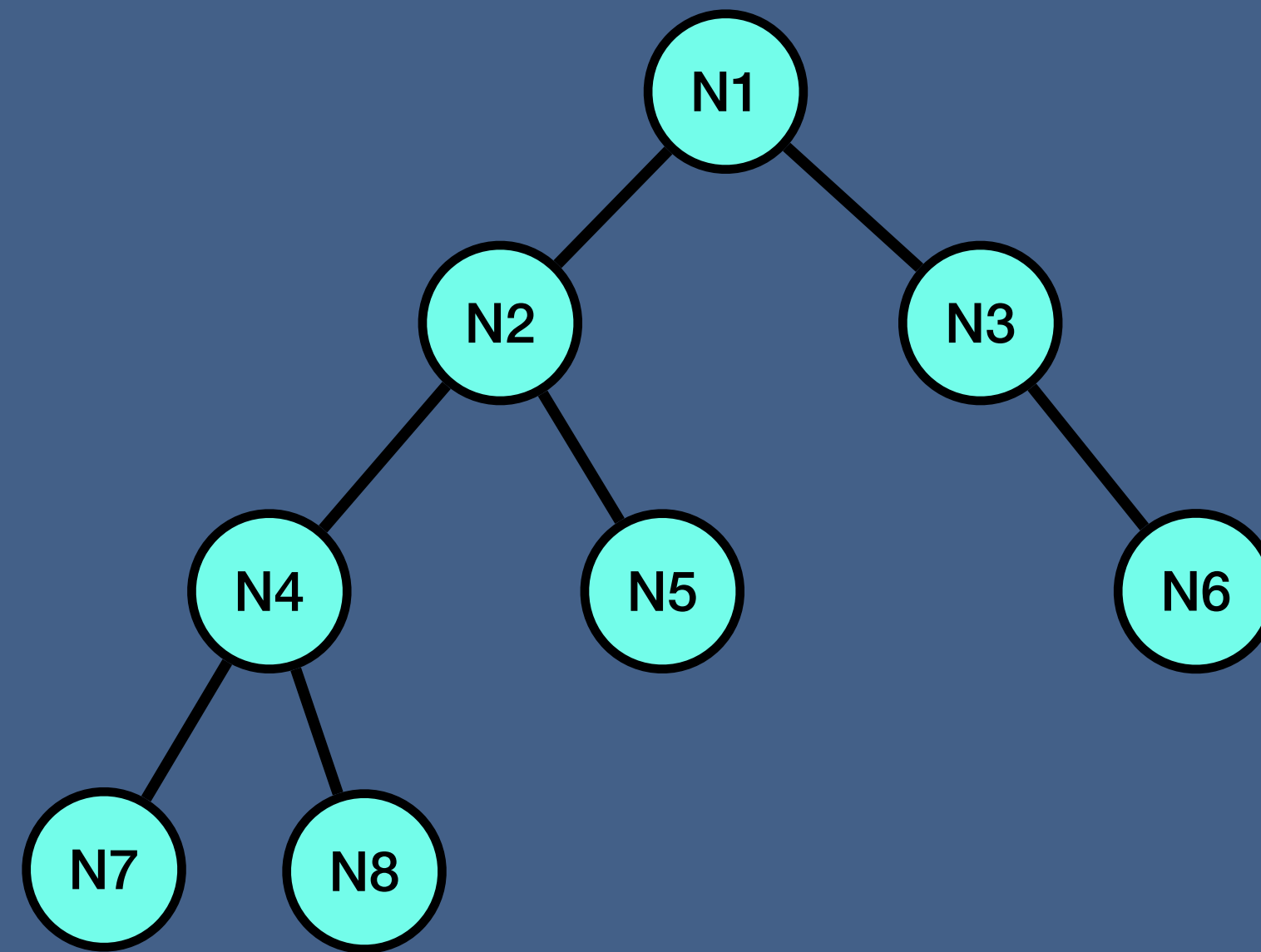
Properties:

- Represent hierarchical data
- Each node has two components : data and a link to its sub category
- Base category and sub categories under it



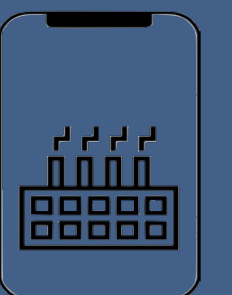
What is a Tree?

A tree is a nonlinear data structure with hierarchical relationships between its elements without having any cycle, it is basically reversed from a real life tree.



Tree Properties:

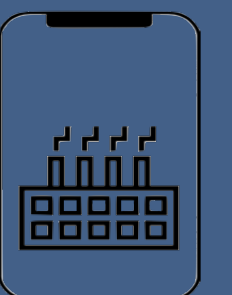
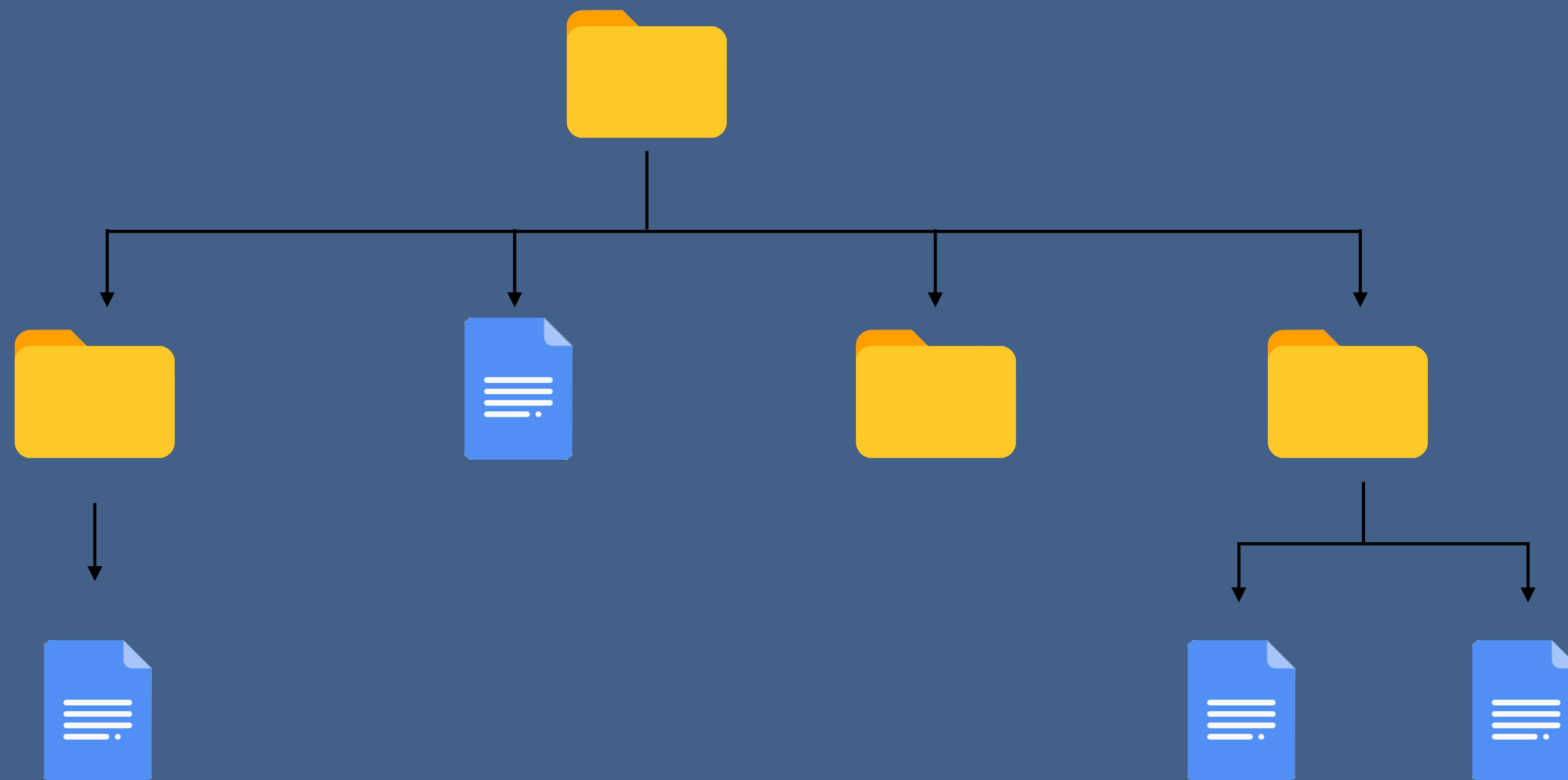
- Represent hierarchical data
- Each node has two components : data and a link to its sub category
- Base category and sub categories under it



Why Tree?

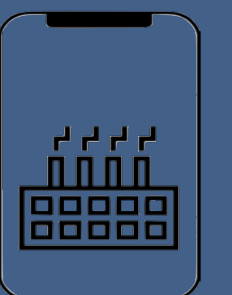
- Quicker and Easier access to the data
- Store hierarchical data, like folder structure, organization structure, XML/HTML data.

The file system on a computer



Why Tree?

- Quicker and Easier access to the data
- Store hierarchical data, like folder structure, organization structure, XML/HTML data.
- There are many different types of data structures which performs better in various situations
 - Binary Search Tree, AVL, Red Black Tree, Trie



Tree Terminology

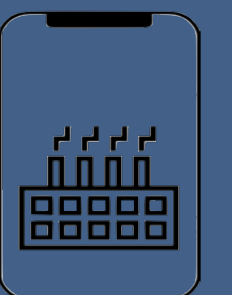
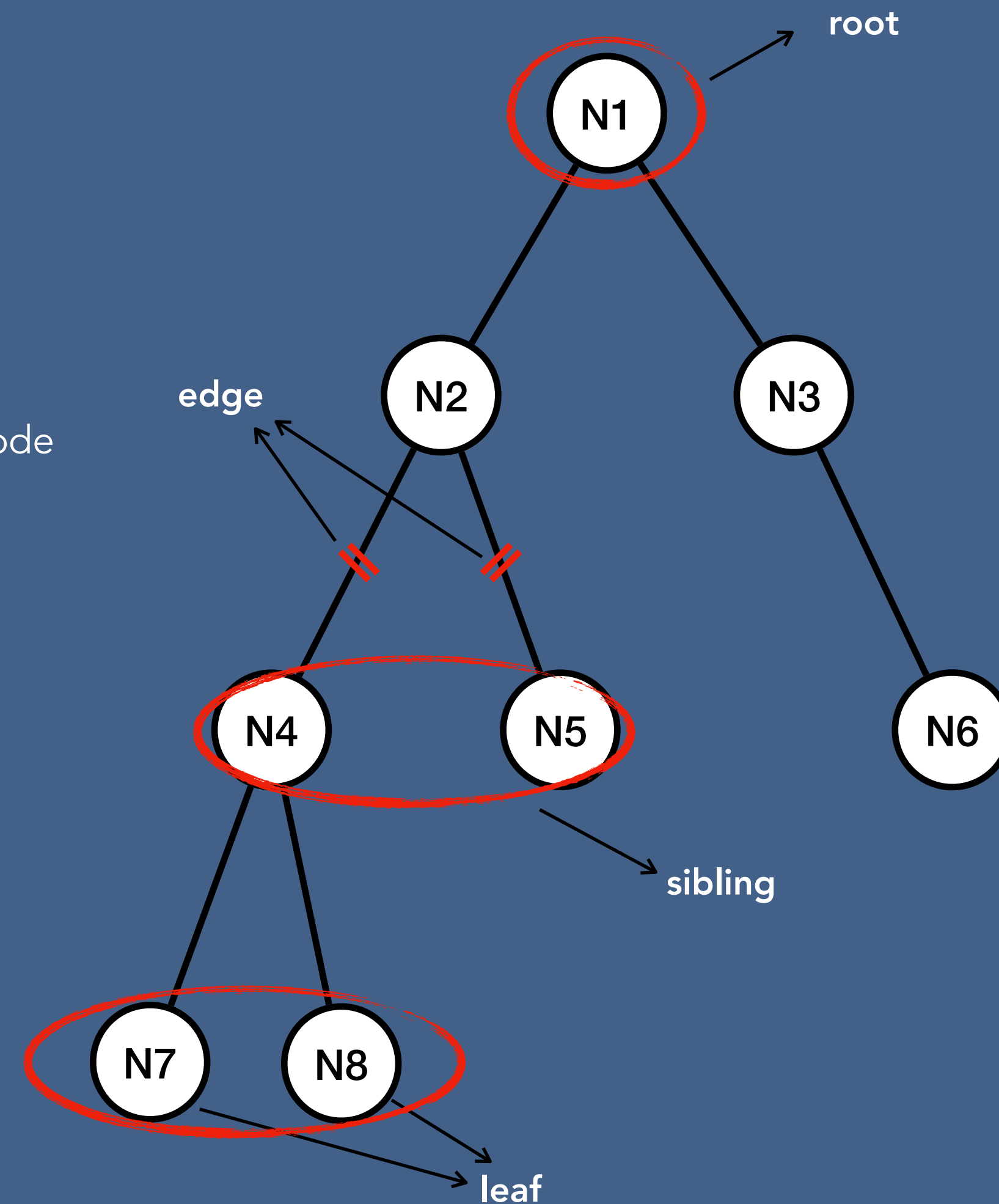
Root : top node without parent

Edge : a link between parent and child

Leaf : a node which does not have children

Sibling : children of same parent

Ancestor : parent, grandparent, great grandparent of a node



Tree Terminology

Root : top node without parent

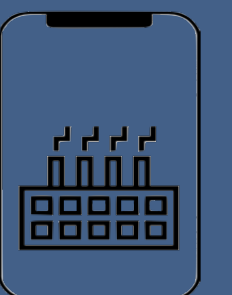
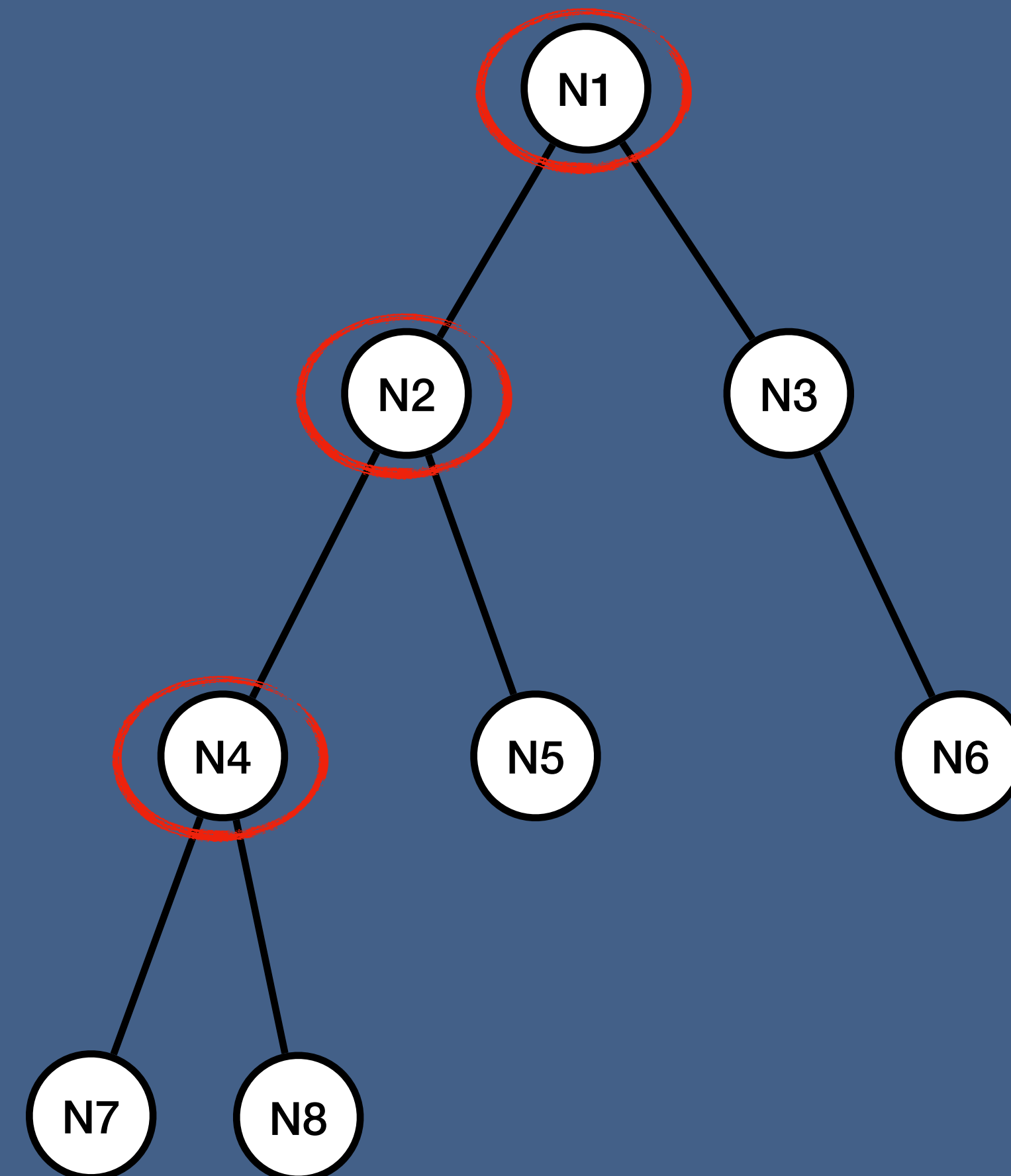
Edge : a link between parent and child

Leaf : a node which does not have children

Sibling : children of same parent

Ancestor : parent, grandparent, great grandparent of a node

ancestors of N7



Tree Terminology

Root : top node without parent

Edge : a link between parent and child

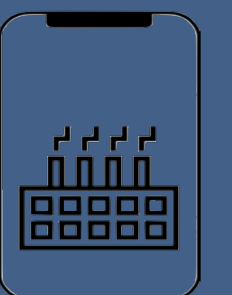
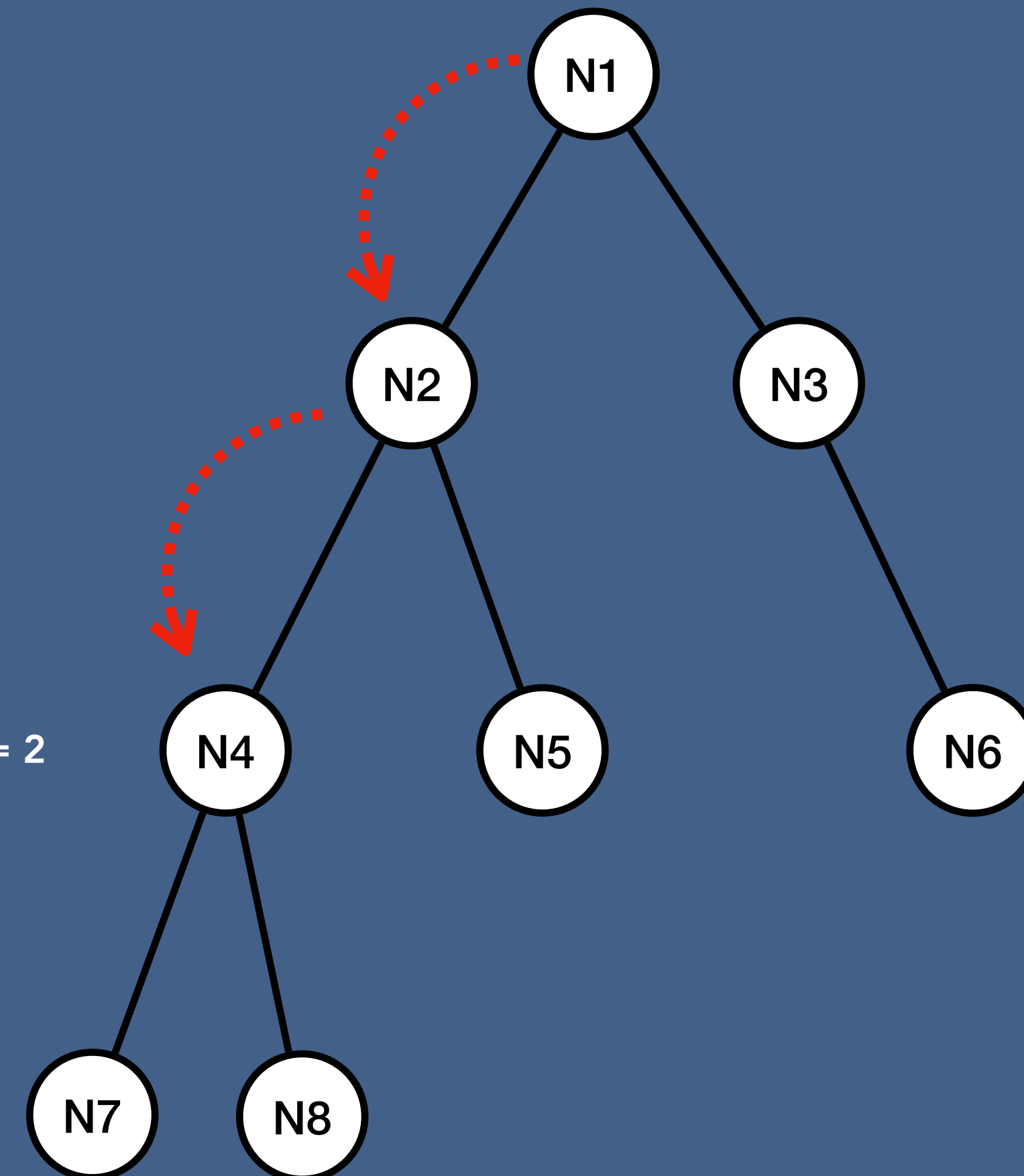
Leaf : a node which does not have children

Sibling : children of same parent

Ancestor : parent, grandparent, great grandparent of a node

Depth of node : a length of the path from root to node

Depth of N4 = 2



Tree Terminology

Root : top node without parent

Edge : a link between parent and child

Leaf : a node which does not have children

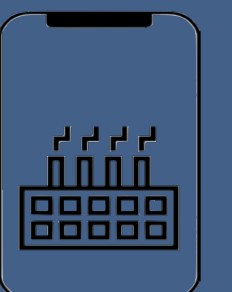
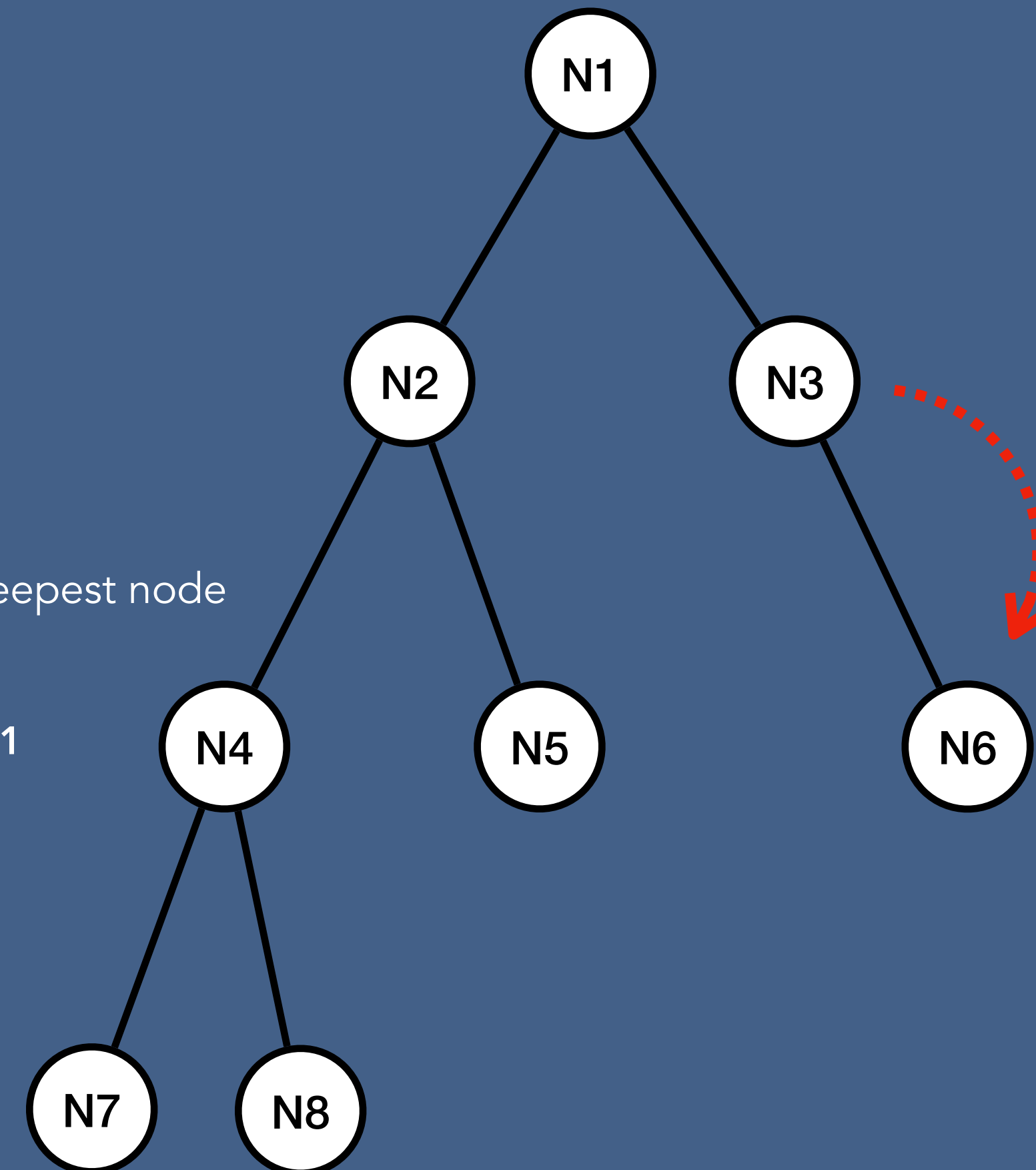
Sibling : children of same parent

Ancestor : parent, grandparent, great grandparent of a node

Depth of node : a length of the path from root to node

Height of node : a length of the path from the node to the deepest node

Heigh of N3 = 1



Tree Terminology

Root : top node without parent

Edge : a link between parent and child

Leaf : a node which does not have children

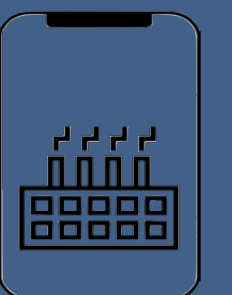
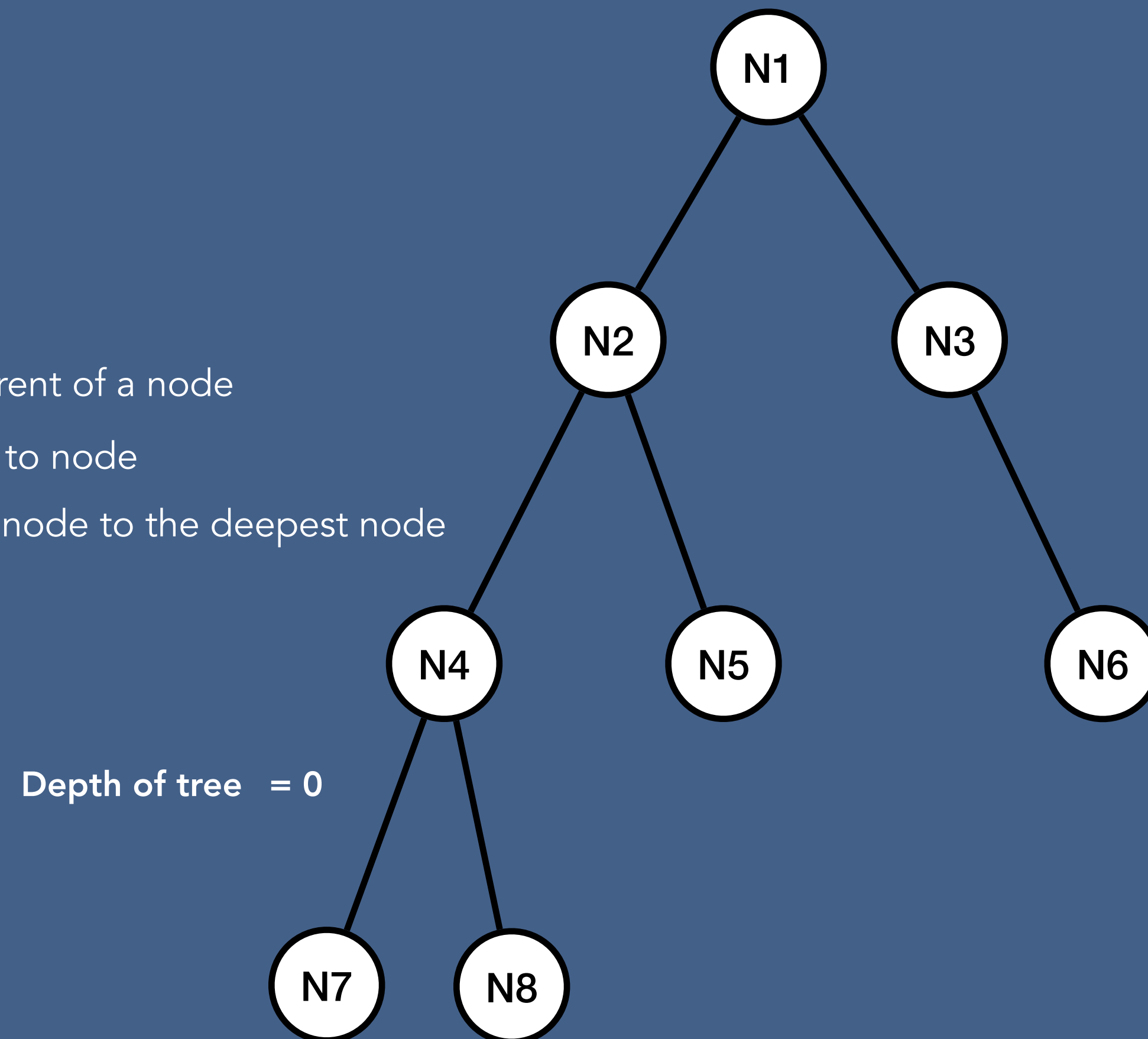
Sibling : children of same parent

Ancestor : parent, grandparent, great grandparent of a node

Depth of node : a length of the path from root to node

Height of node : a length of the path from the node to the deepest node

Depth of tree : depth of root node



Tree Terminology

Root : top node without parent

Edge : a link between parent and child

Leaf : a node which does not have children

Sibling : children of same parent

Ancestor : parent, grandparent, great grandparent of a node

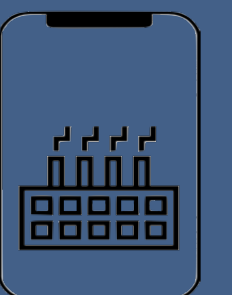
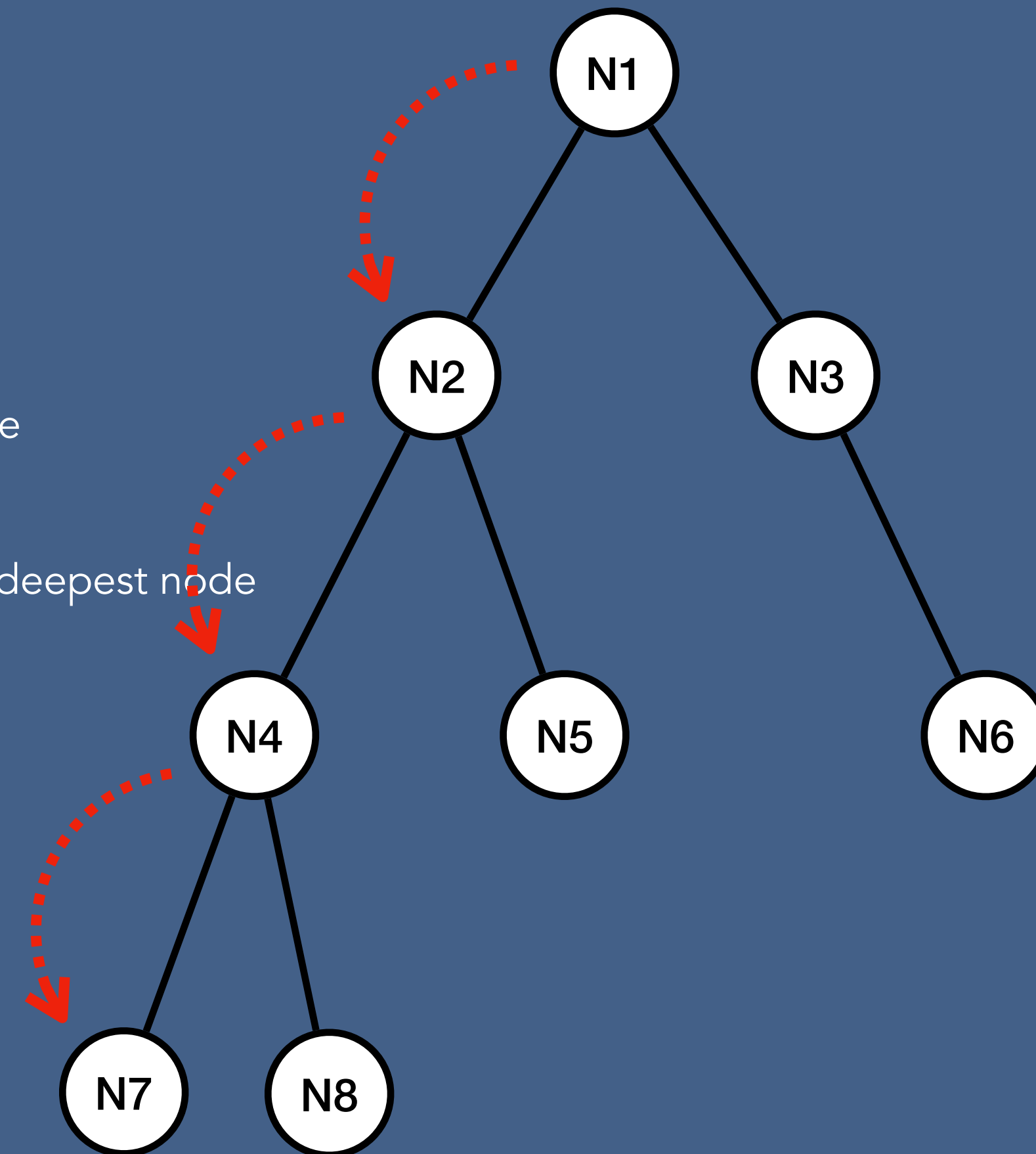
Depth of node : a length of the path from root to node

Height of node : a length of the path from the node to the deepest node

Depth of tree : depth of root node

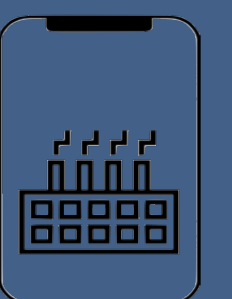
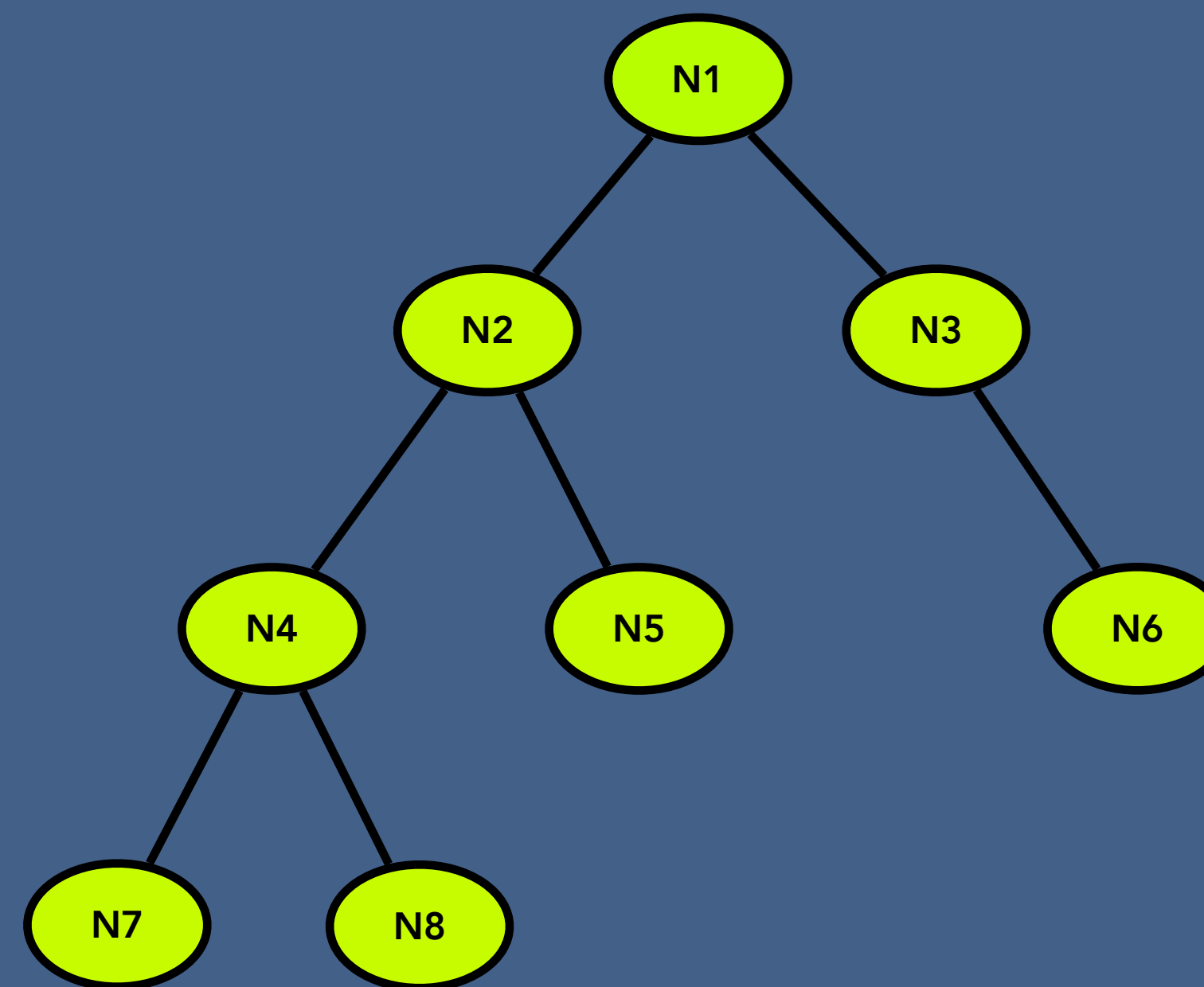
Height of tree : height of root node

Height of tree = 3



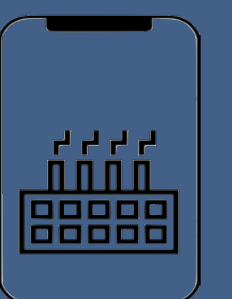
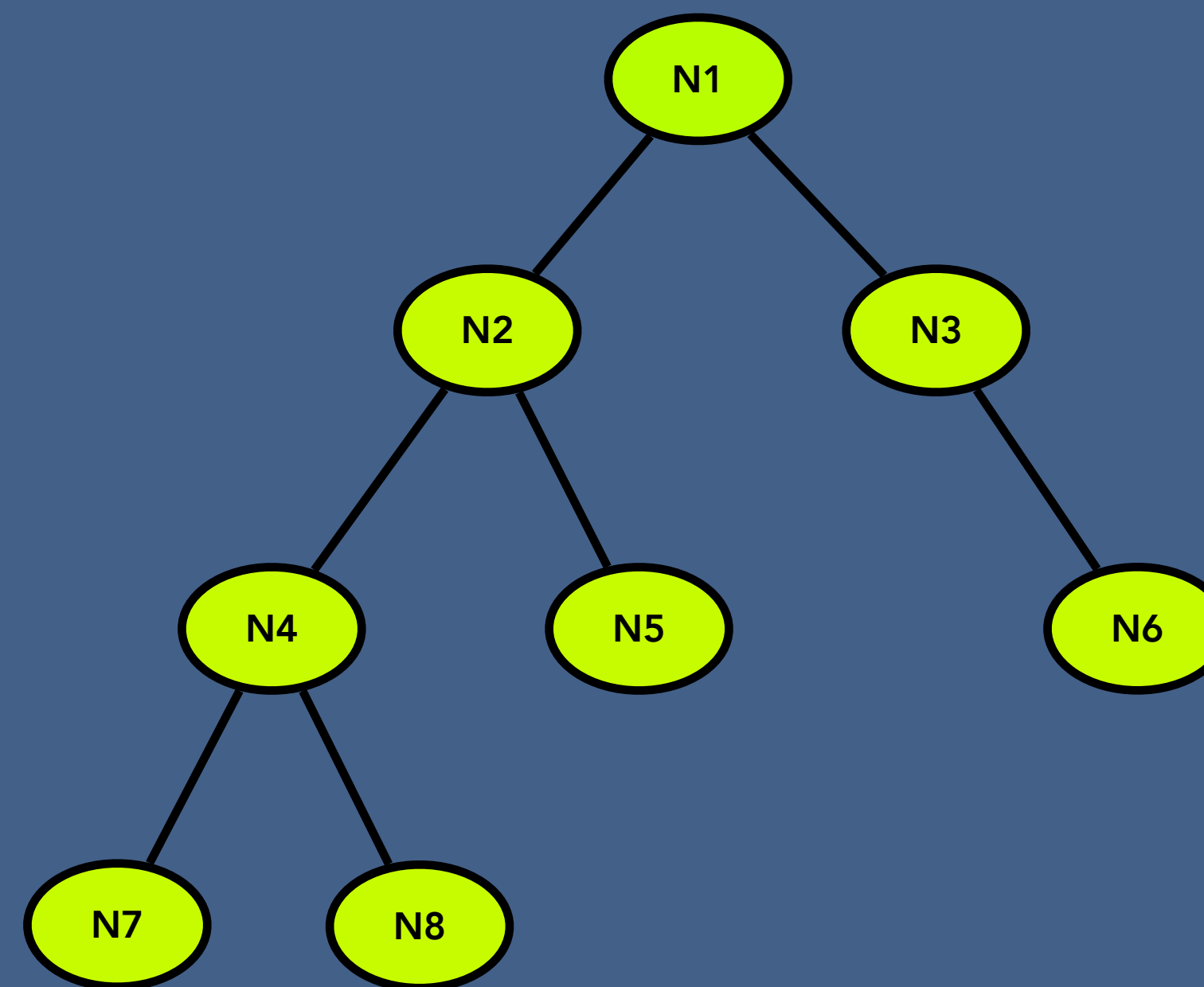
Binary Tree

- Binary trees are the data structures in which each node has at most two children, often referred to as the left and right children
- Binary tree is a family of data structure (BST, Heap tree, AVL, red black trees, Syntax tree)



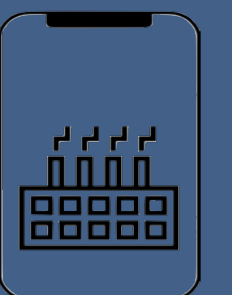
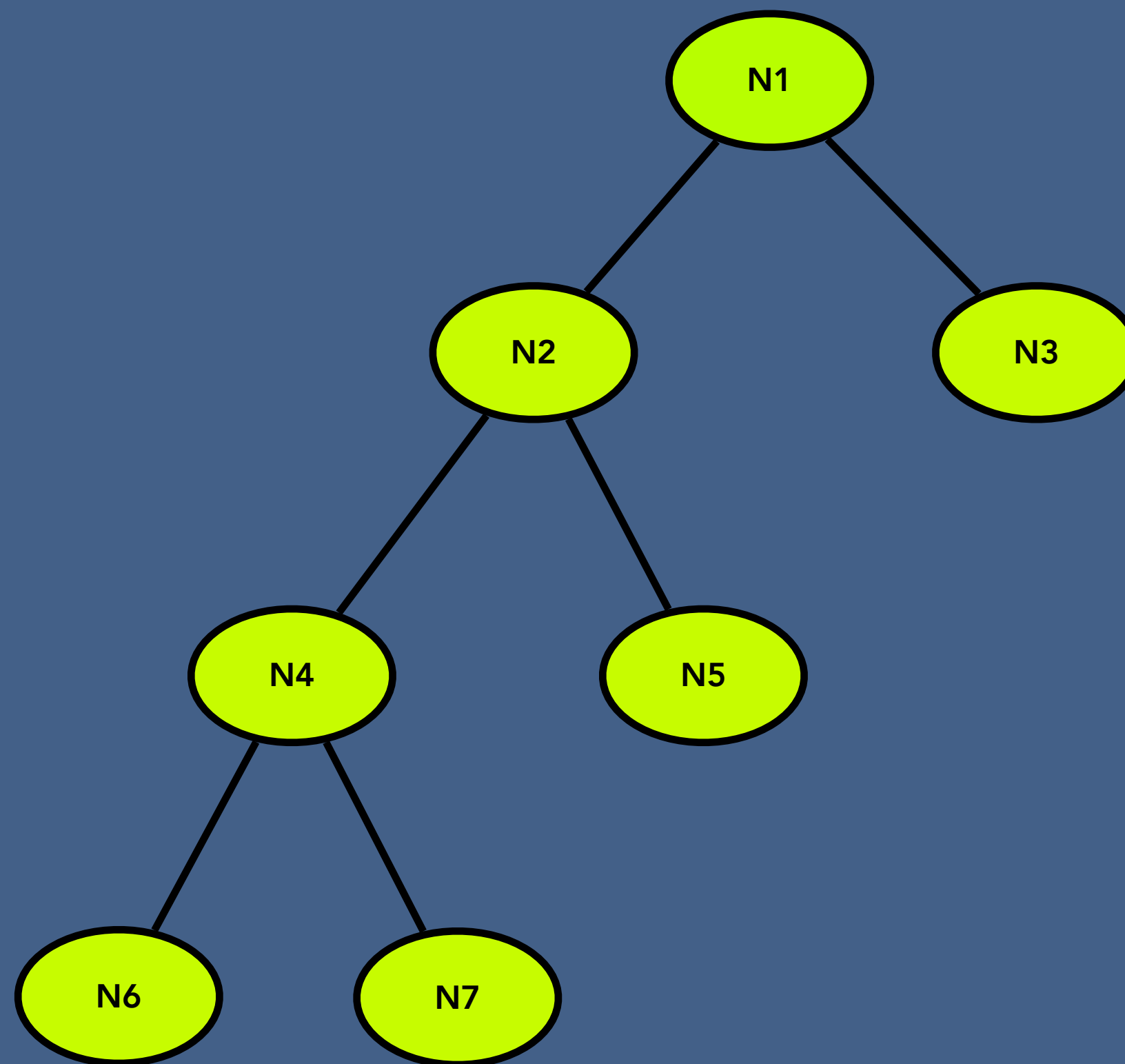
Why Binary Tree?

- Binary trees are a prerequisite for more advanced trees like BST, AVL, Red Black Trees
- Huffman coding problem, heap priority problem and expression parsing problems can be solved efficiently using binary trees,



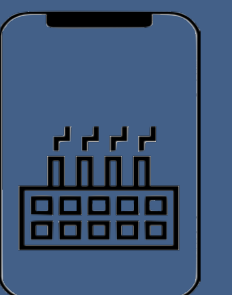
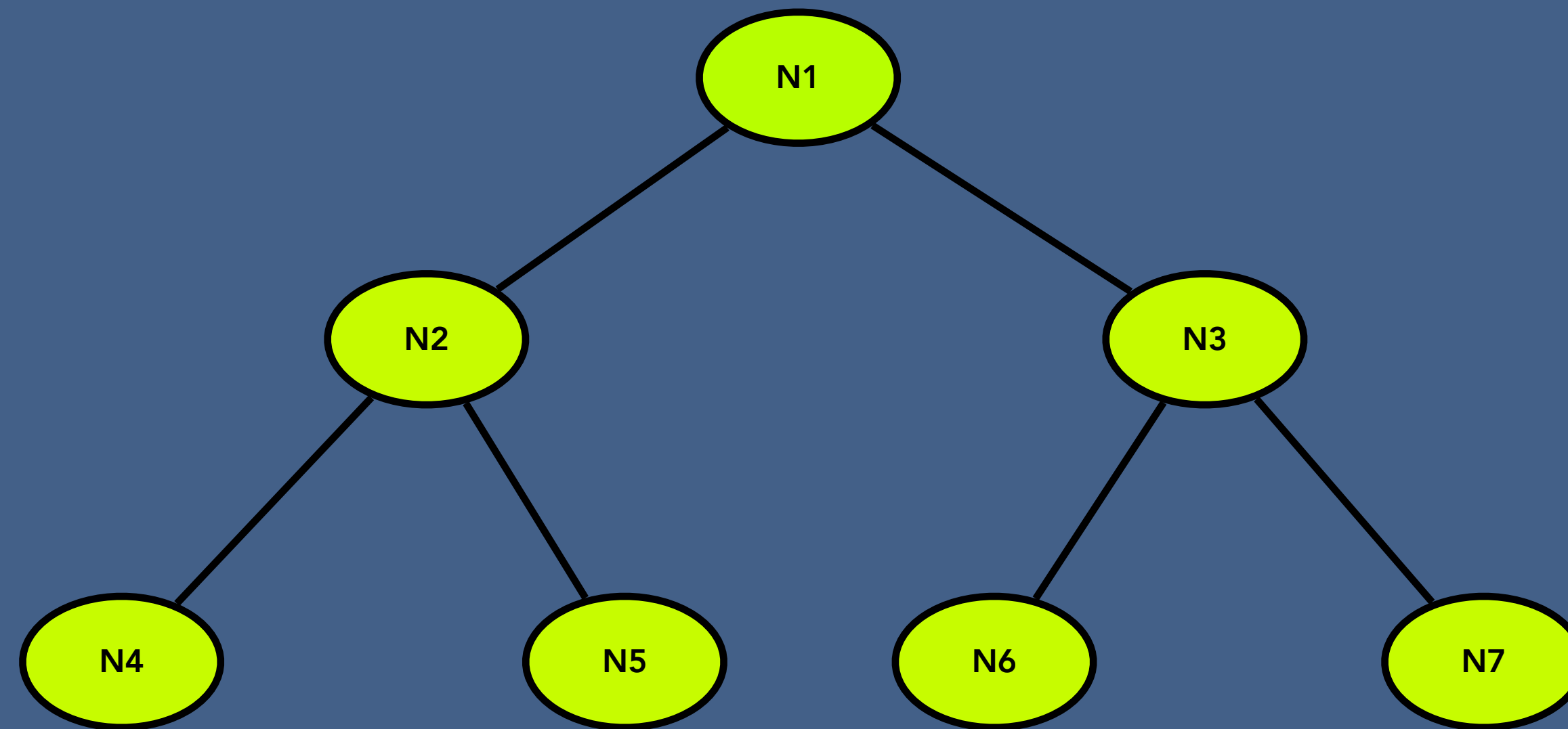
Types of Binary Tree

Full Binary Tree



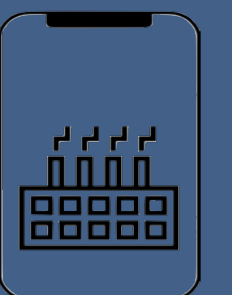
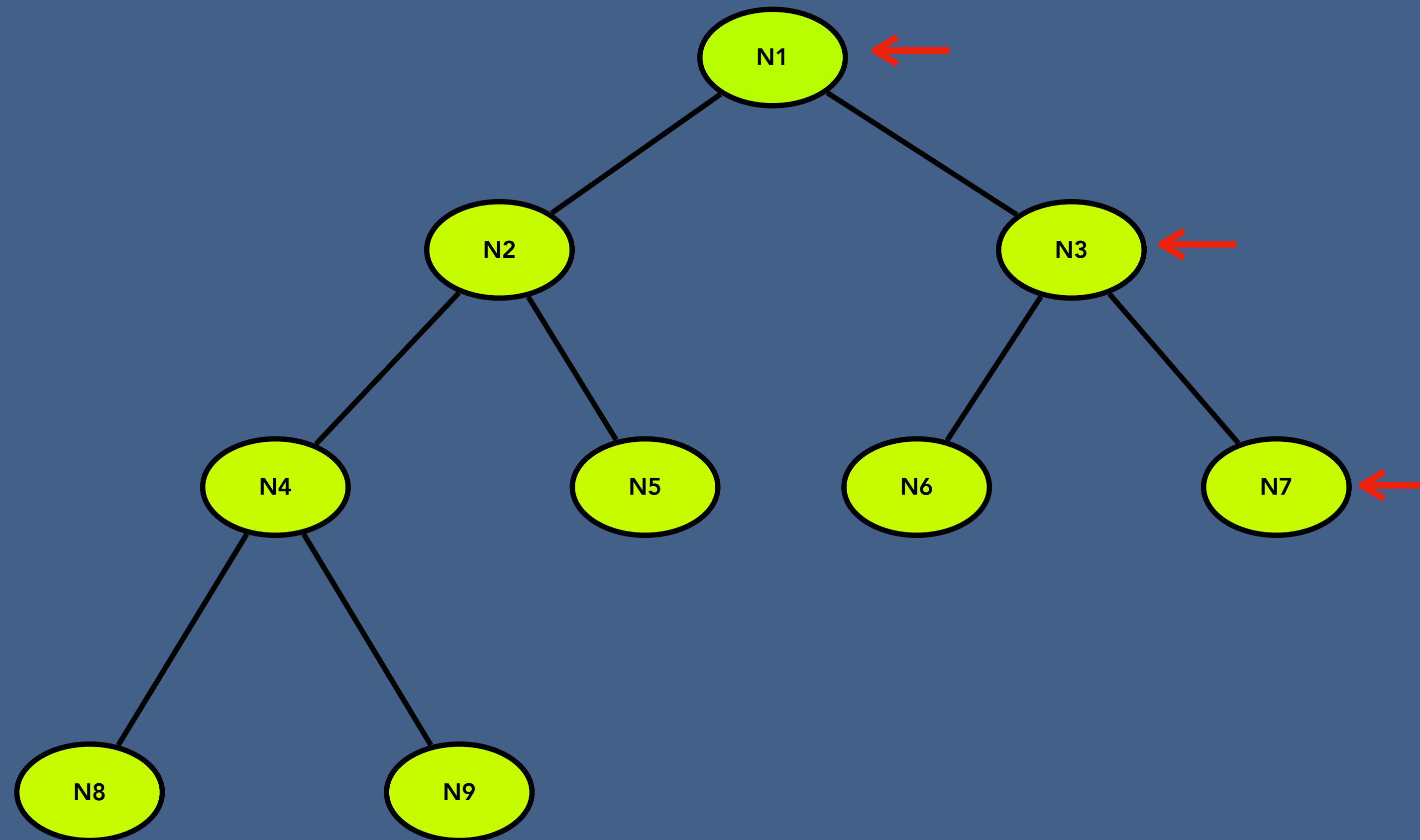
Types of Binary Tree

Perfect Binary Tree



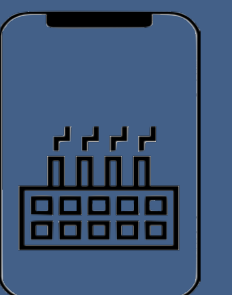
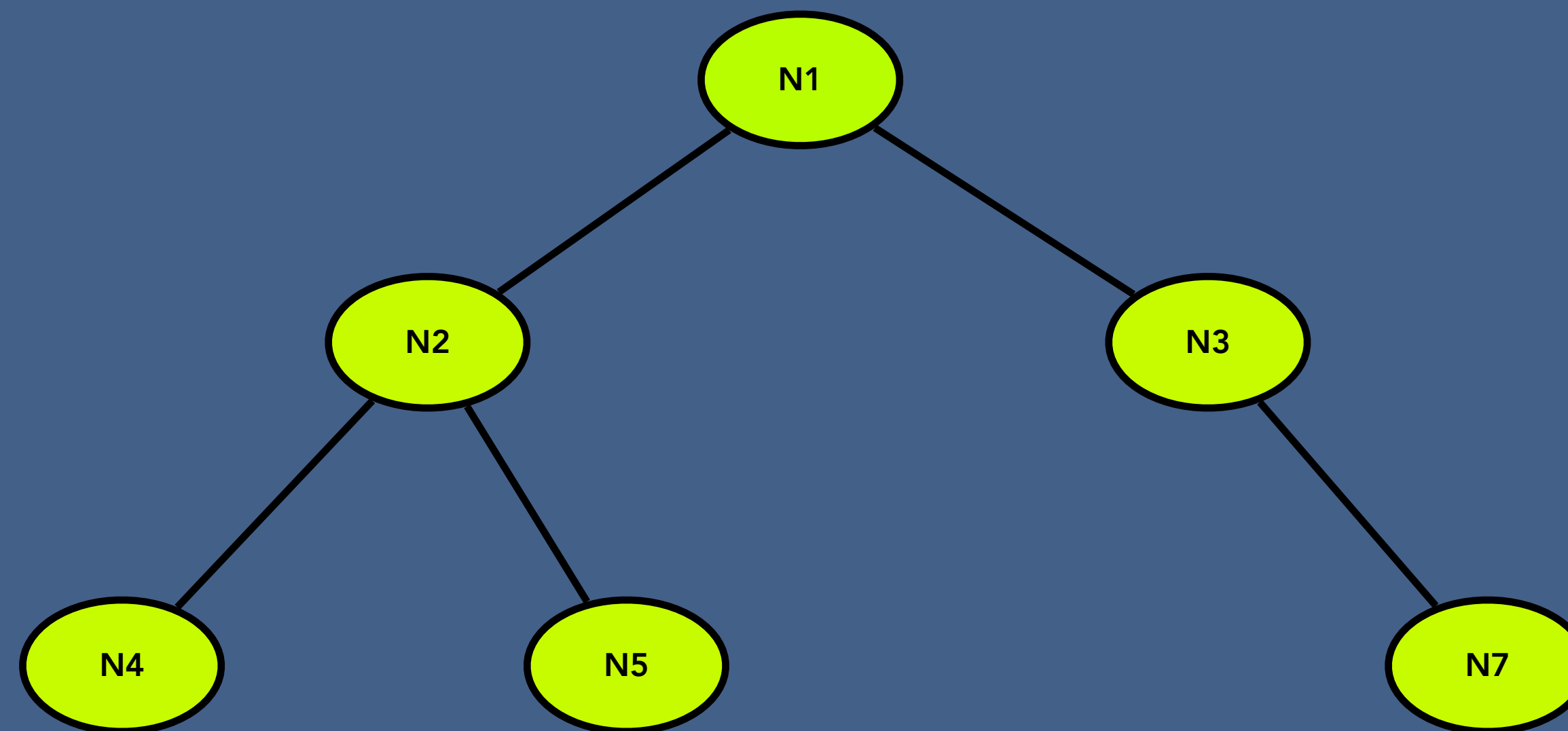
Types of Binary Tree

Complete Binary Tree



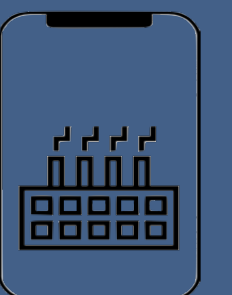
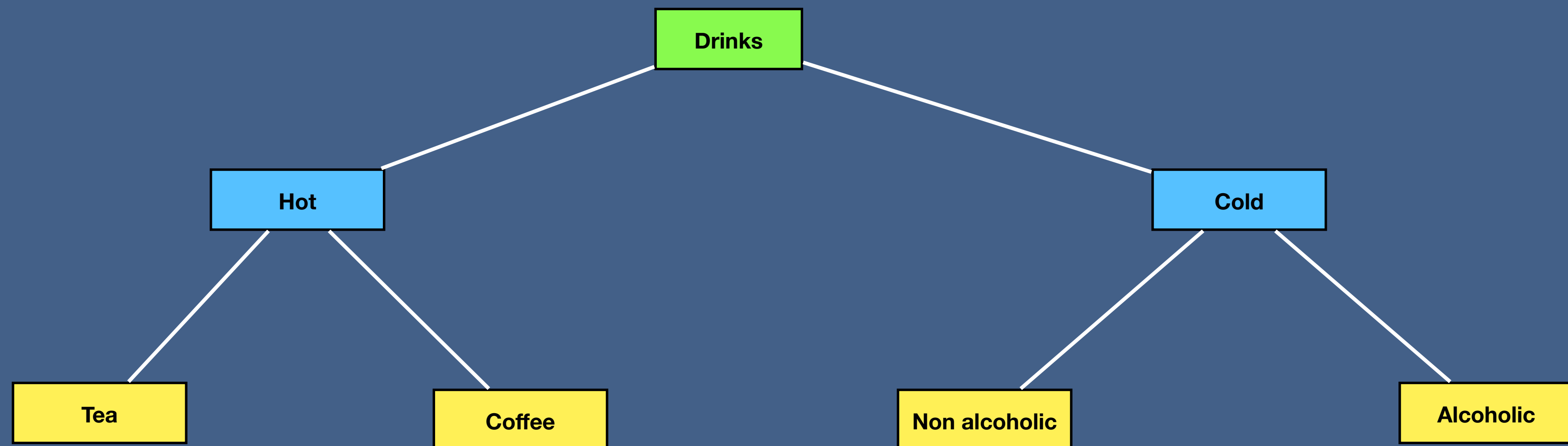
Types of Binary Tree

Balanced Binary Tree



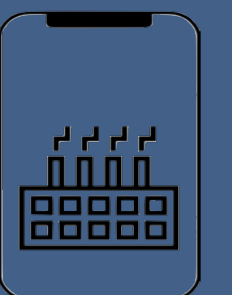
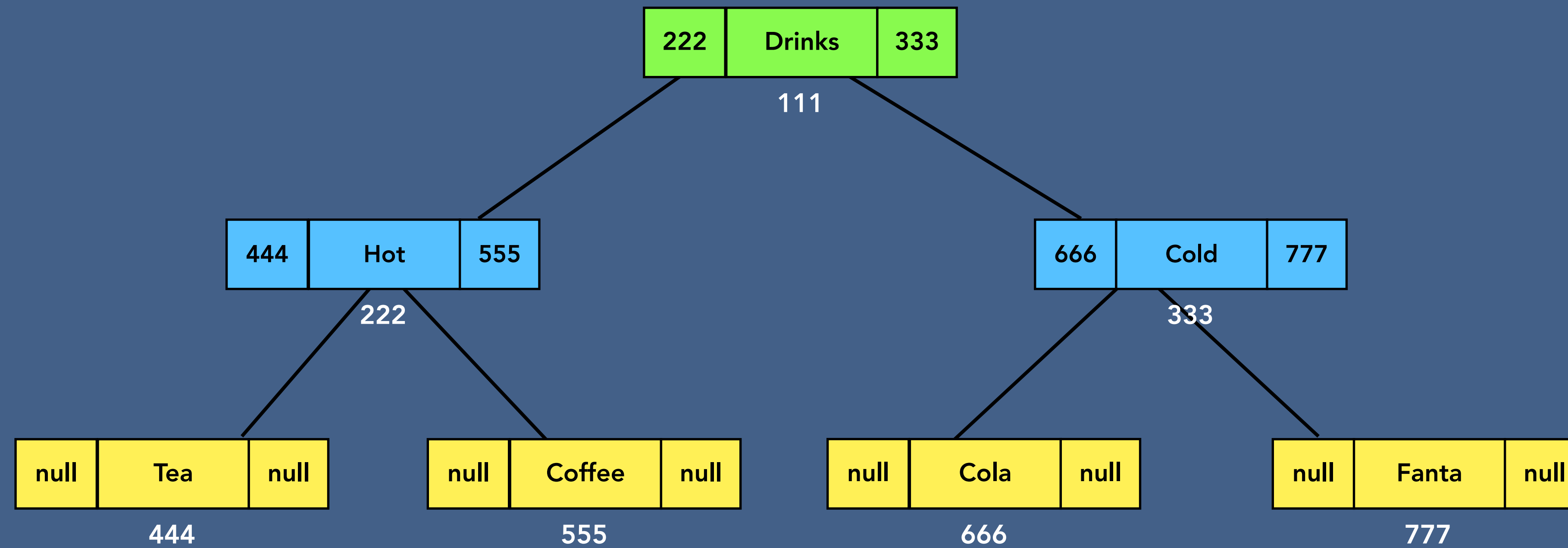
Binary Tree

- Linked List
- Array



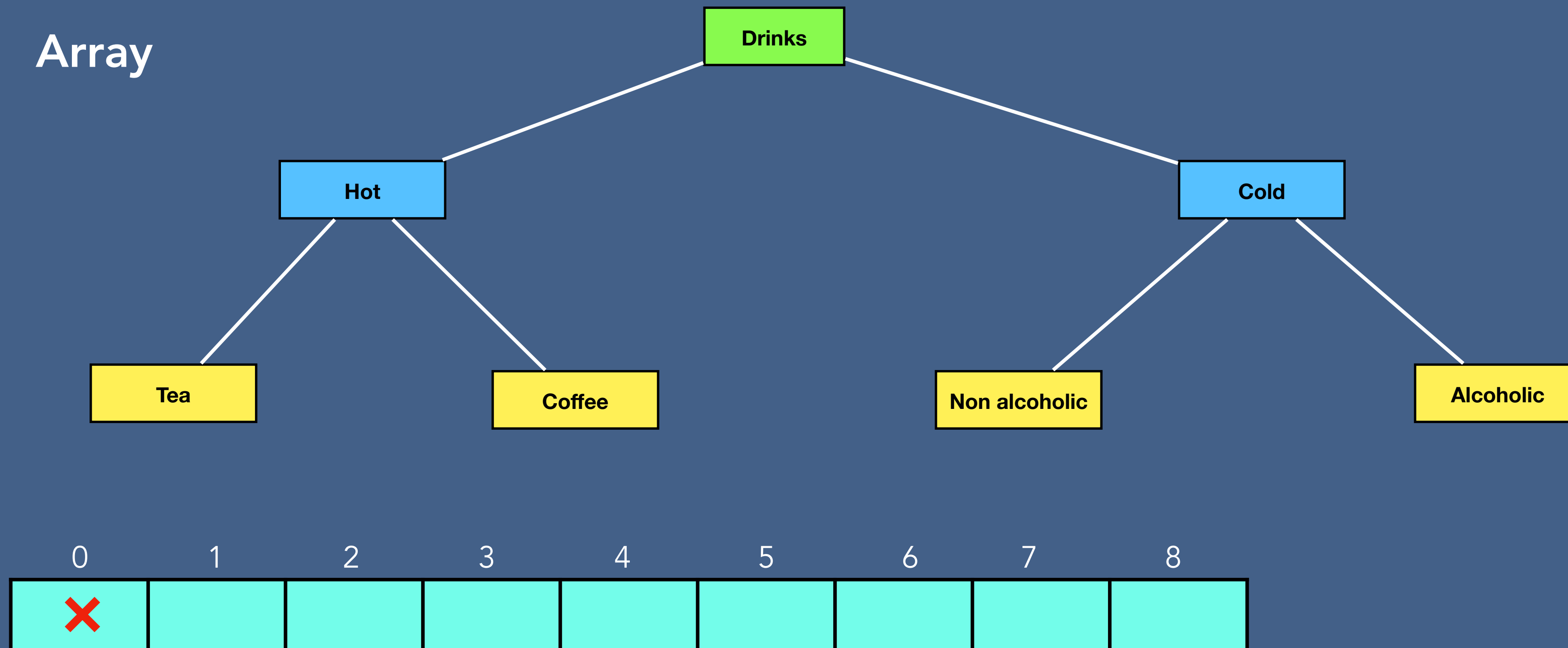
Binary Tree

Linked List



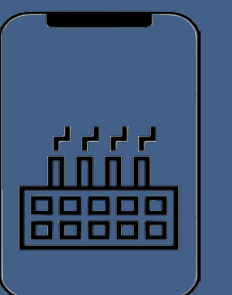
Binary Tree

Array



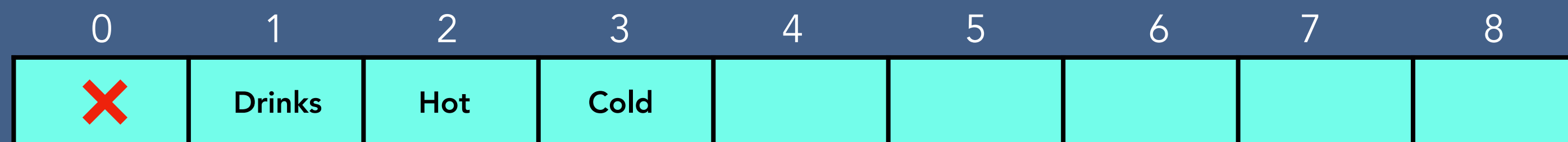
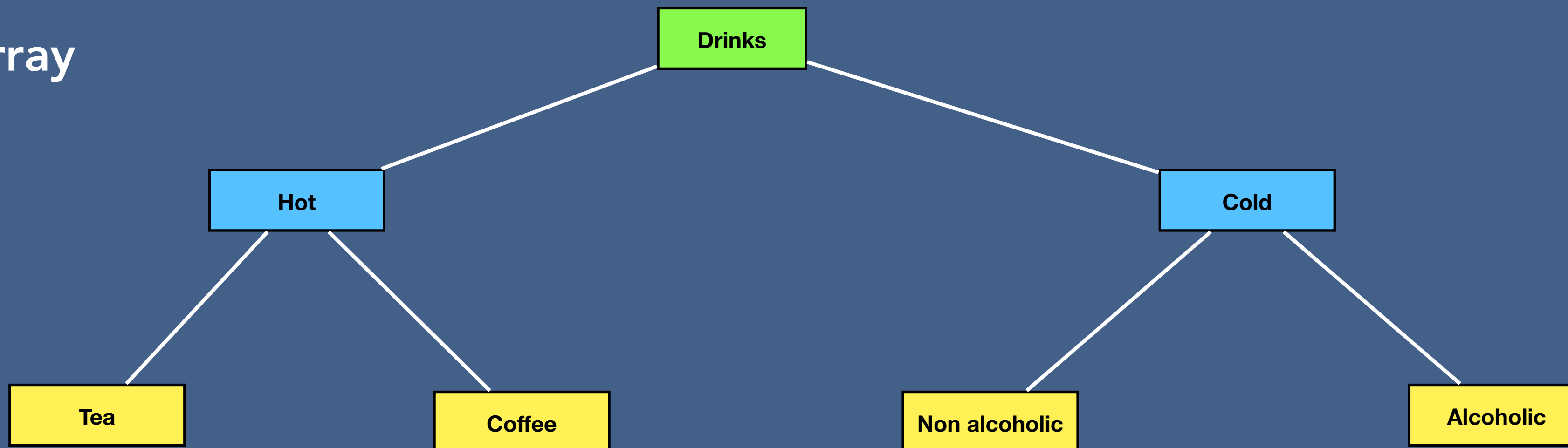
Left child = $\text{cell}[2x]$

Right child = $\text{cell}[2x+1]$



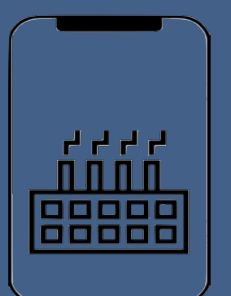
Binary Tree

Array



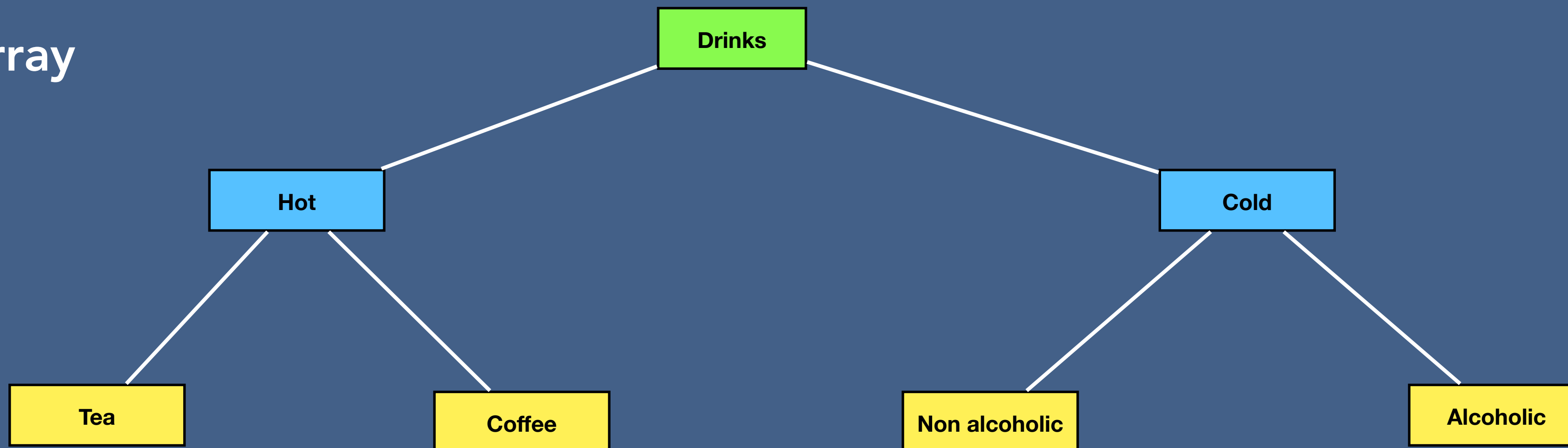
Left child = $\text{cell}[2x]$ $\longrightarrow$ $x = 1$, $\text{cell}[2 \times 1 = 2]$

Right child = $\text{cell}[2x+1]$ $\longrightarrow$ $x = 1$, $\text{cell}[2 \times 1 + 1 = 3]$



Binary Tree

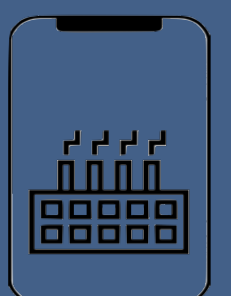
Array



0	1	2	3	4	5	6	7	8
×	Drinks	Hot	Cold	Tea	Coffee			

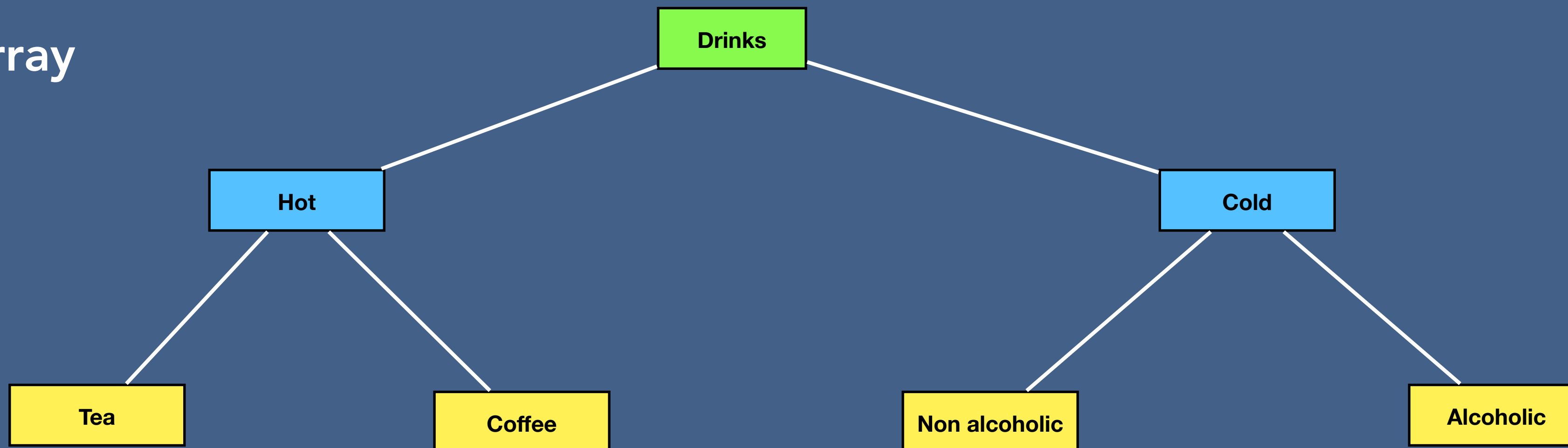
Left child = $\text{cell}[2x]$ $\longrightarrow$ $x = 2$, $\text{cell}[2 \times 2 = 4]$

Right child = $\text{cell}[2x+1]$ $\longrightarrow$ $x = 2$, $\text{cell}[2 \times 2 + 1 = 5]$



Binary Tree

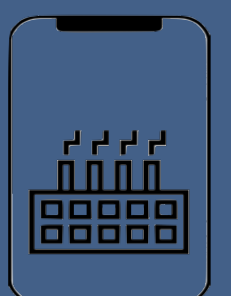
Array



0	1	2	3	4	5	6	7	8
×	Drinks	Hot	Cold	Tea	Coffee	Non alcoholic	Alcoholic	

Left child = $\text{cell}[2x]$ $\longrightarrow$ $x = 3$, $\text{cell}[2 \times 3 = 6]$

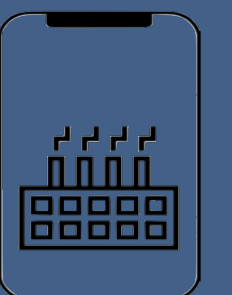
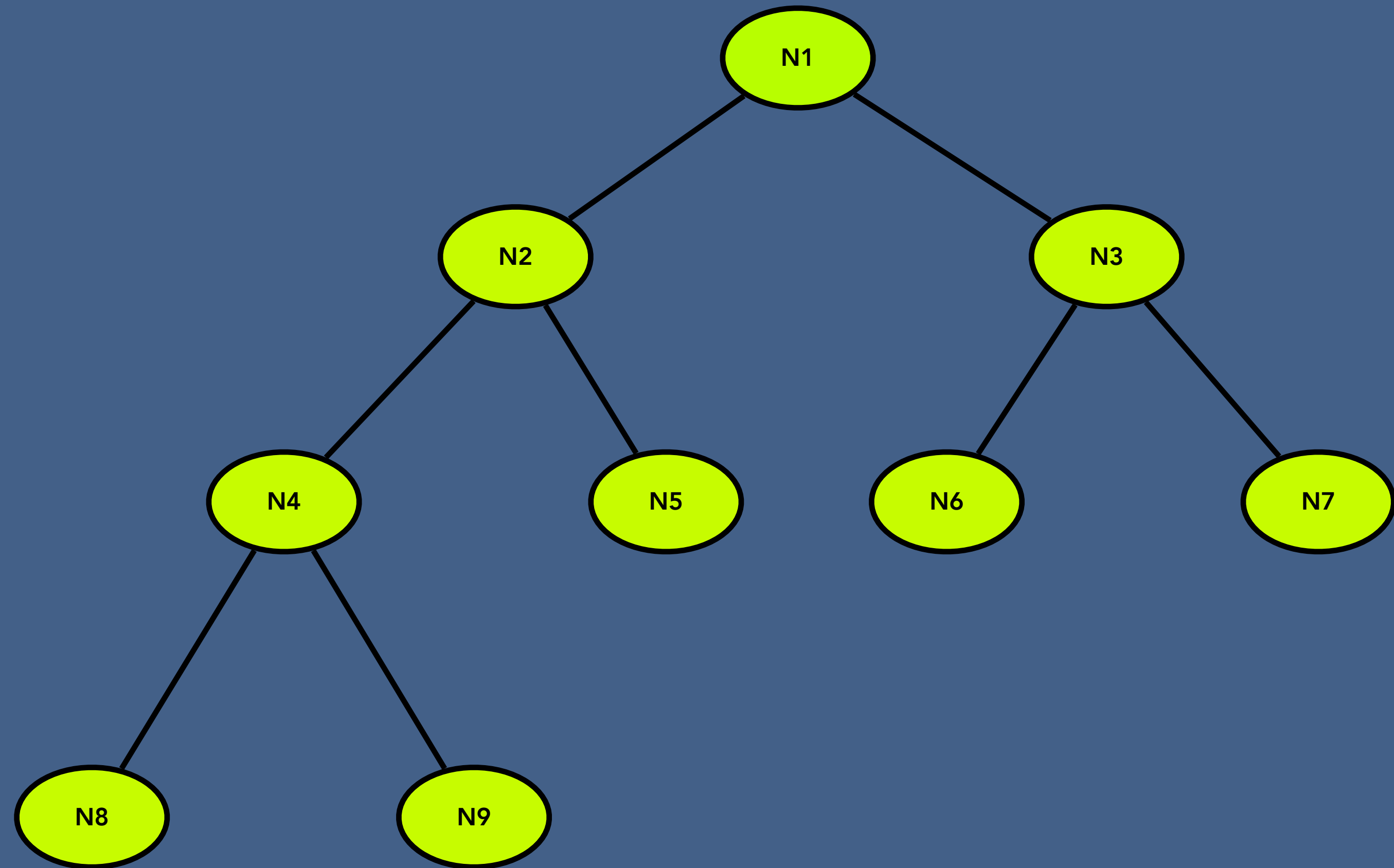
Right child = $\text{cell}[2x+1]$ $\longrightarrow$ $x = 3$, $\text{cell}[2 \times 3 + 1 = 7]$



Binary Tree using Linked List

- Creation of Tree
- Insertion of a node
- Deletion of a node
- Search for a value
- Traverse all nodes
- Deletion of tree

```
newTree = Tree()
```



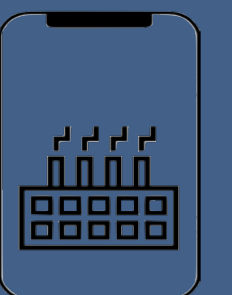
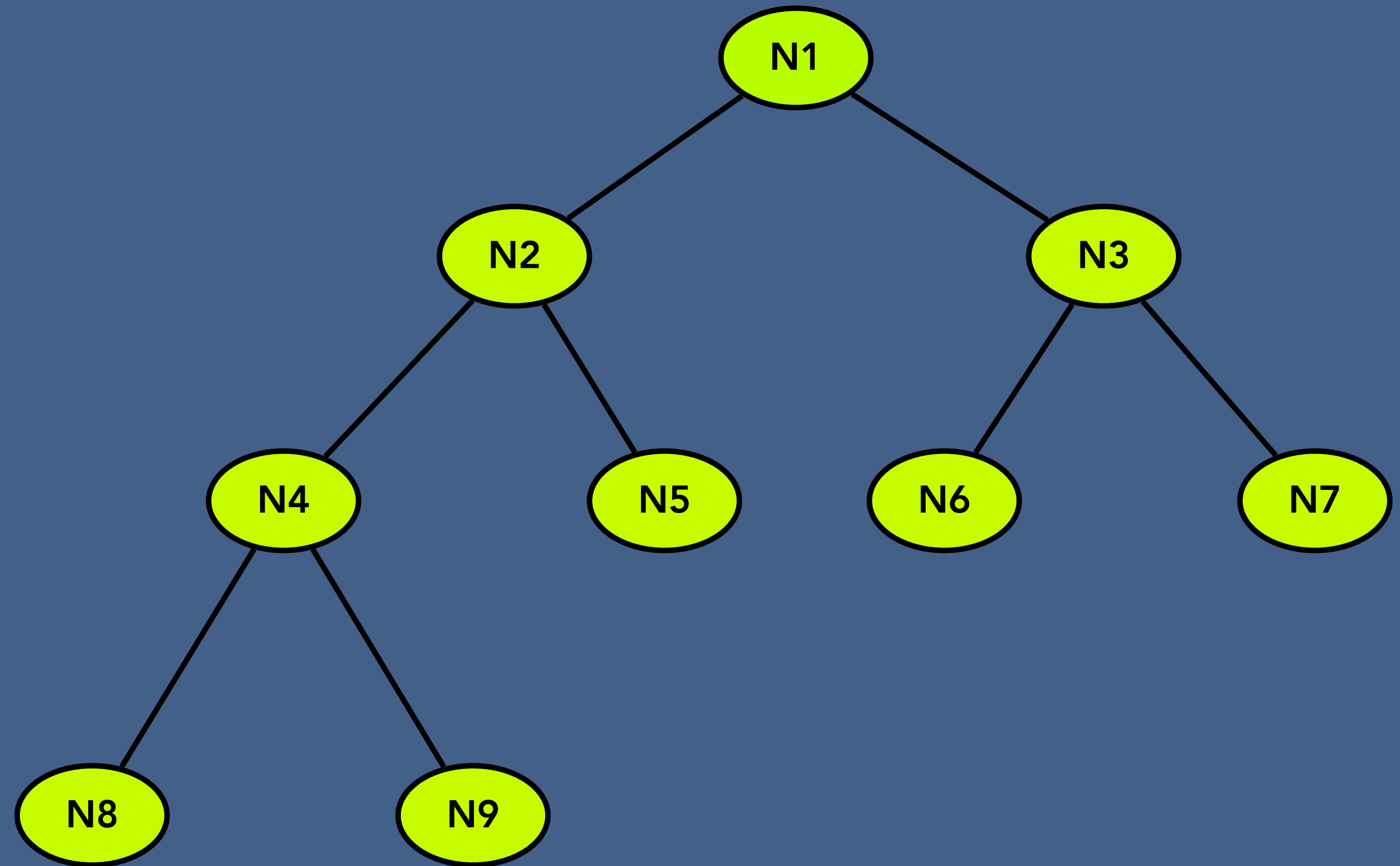
Binary Tree - Traversal

Depth first search

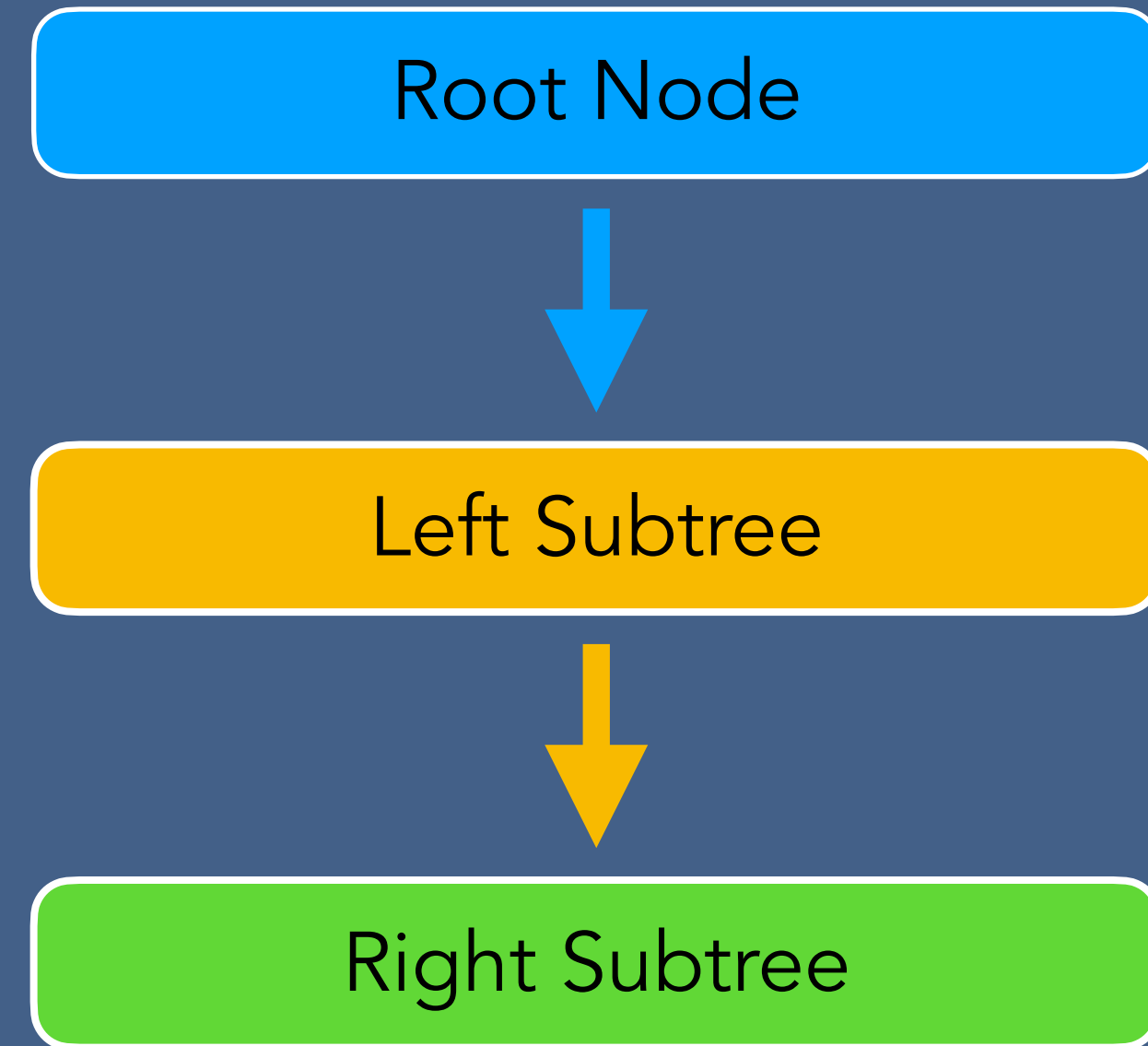
- Preorder traversal
- Inorder traversal
- Post order traversal

Breadth first search

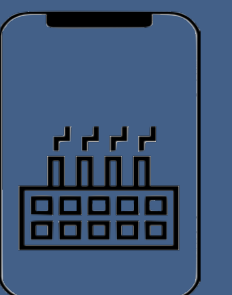
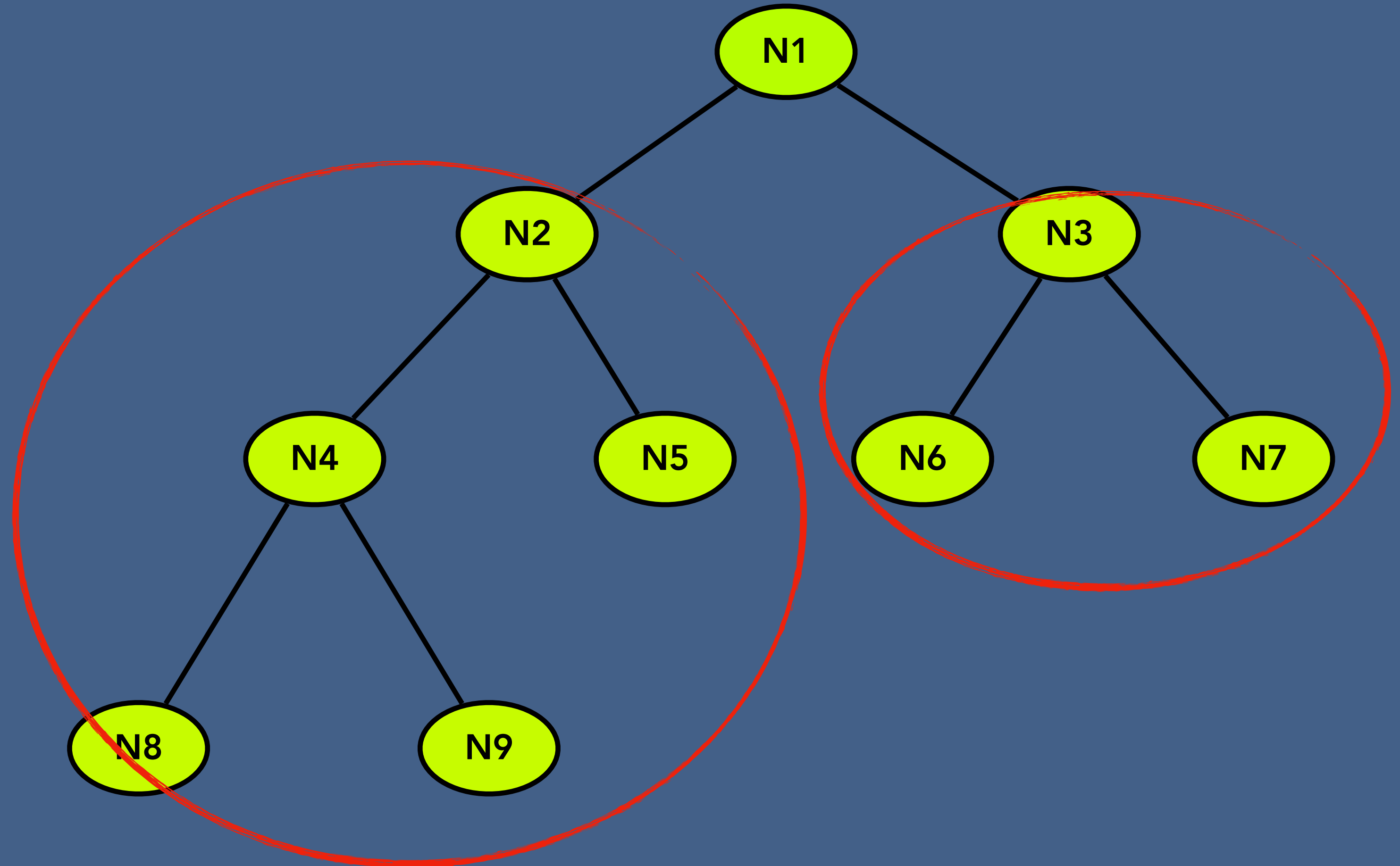
- Level order traversal



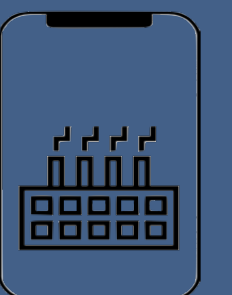
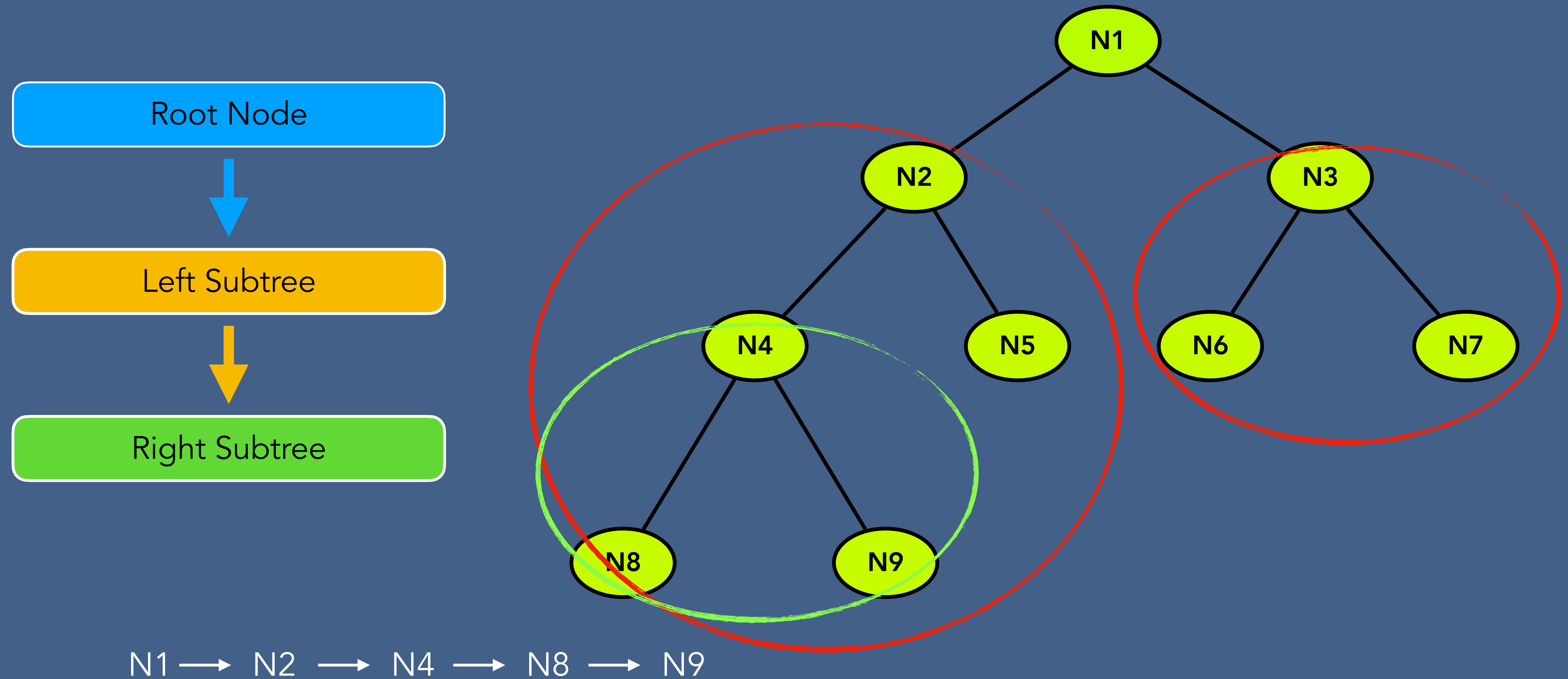
Binary Tree - PreOrder Traversal



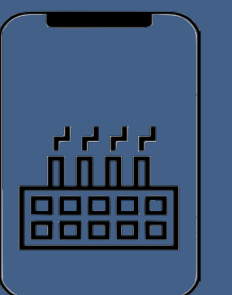
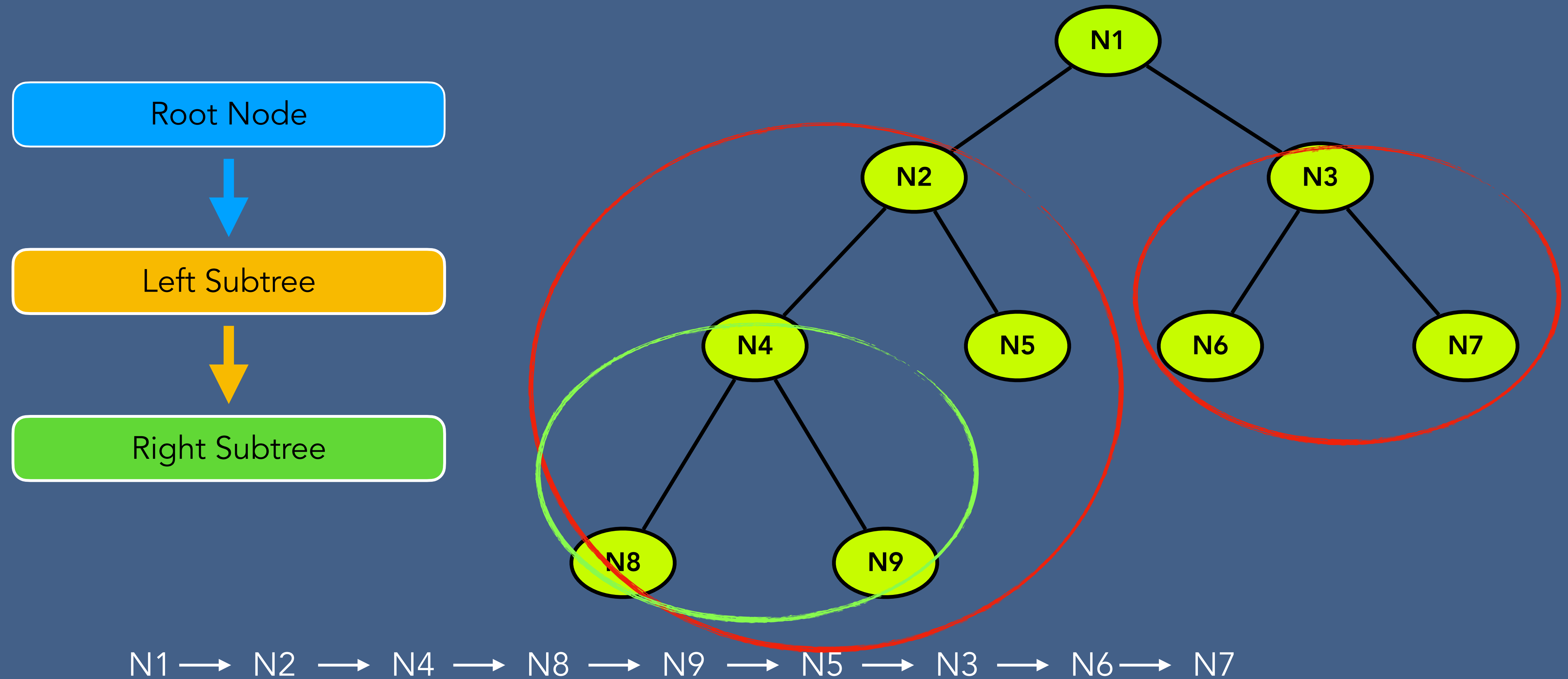
N1 → N2



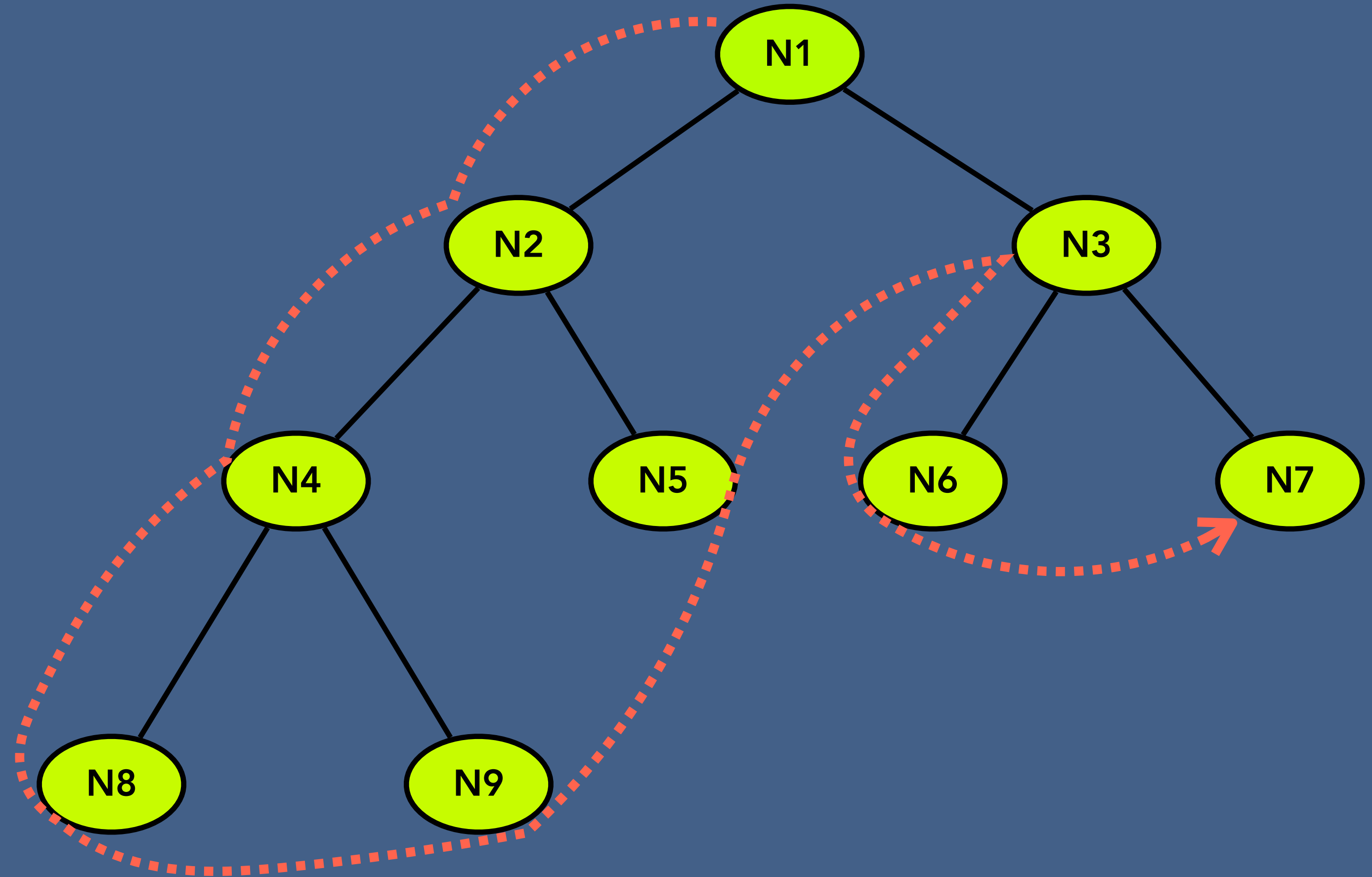
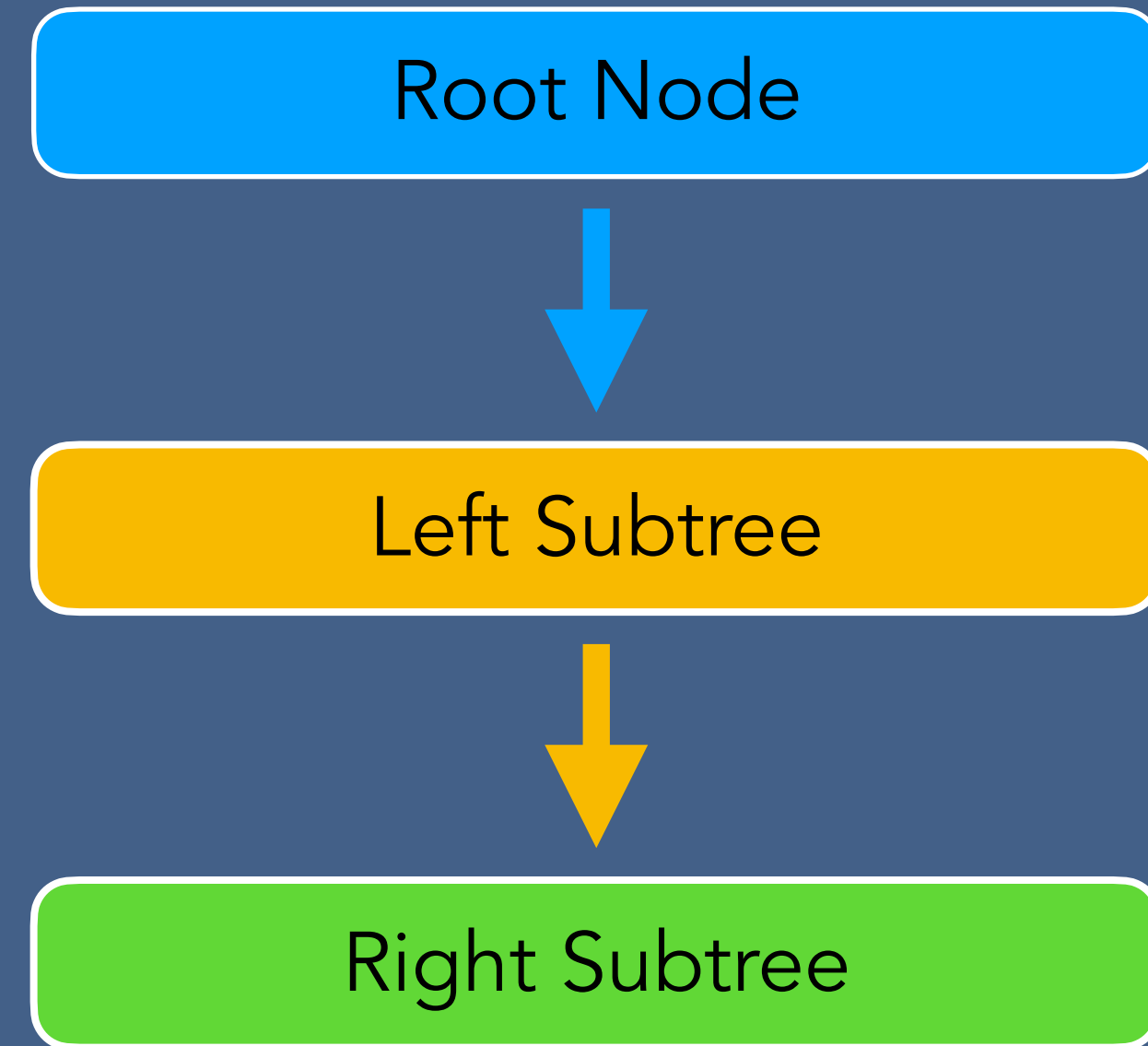
Binary Tree - PreOrder Traversal



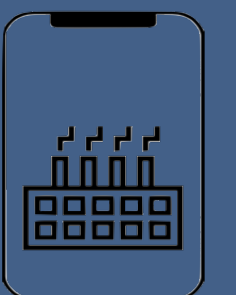
Binary Tree - PreOrder Traversal



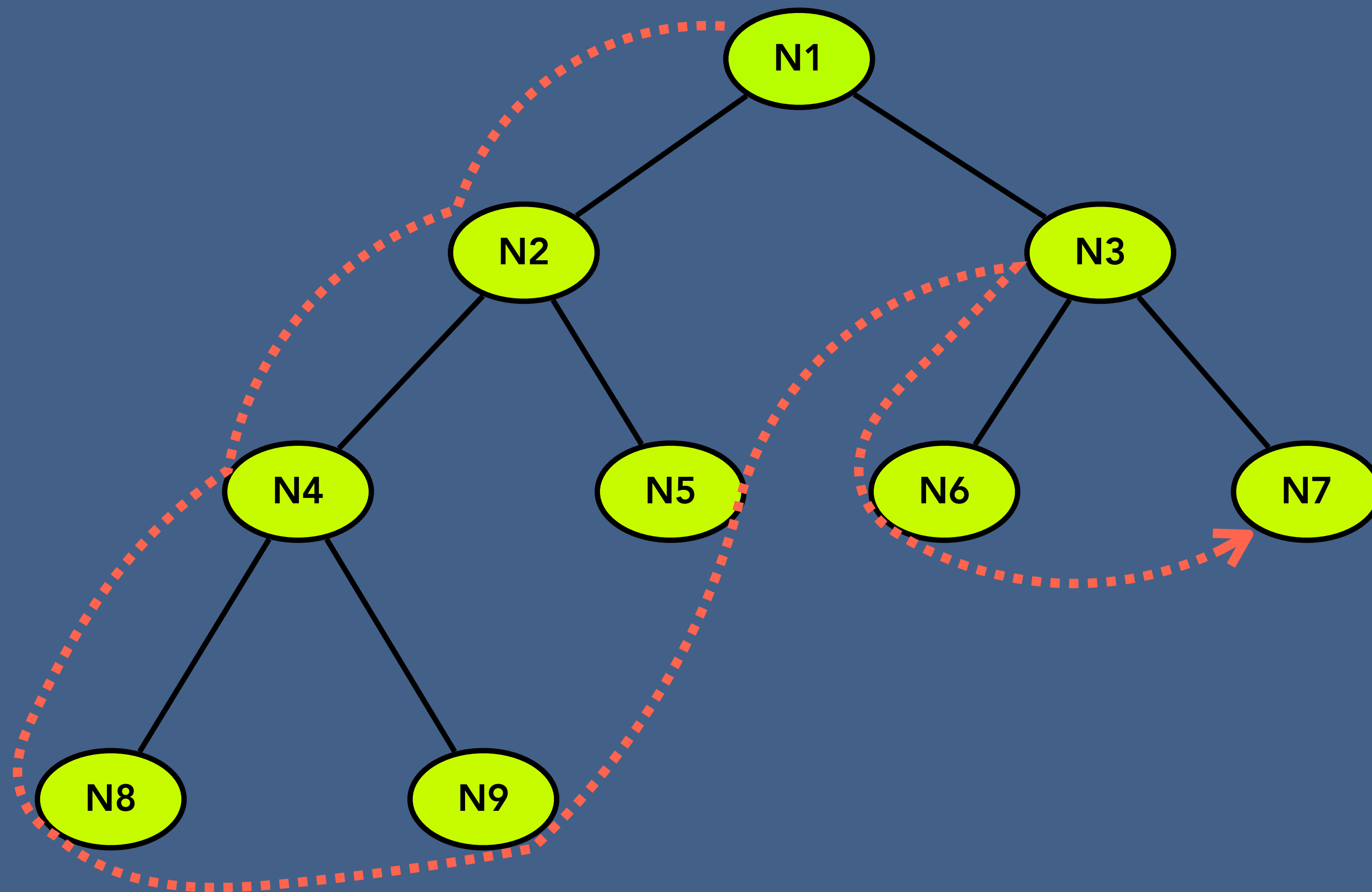
Binary Tree - PreOrder Traversal



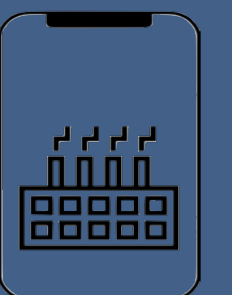
N1 → N2 → N4 → N8 → N9 → N5 → N3 → N6 → N7



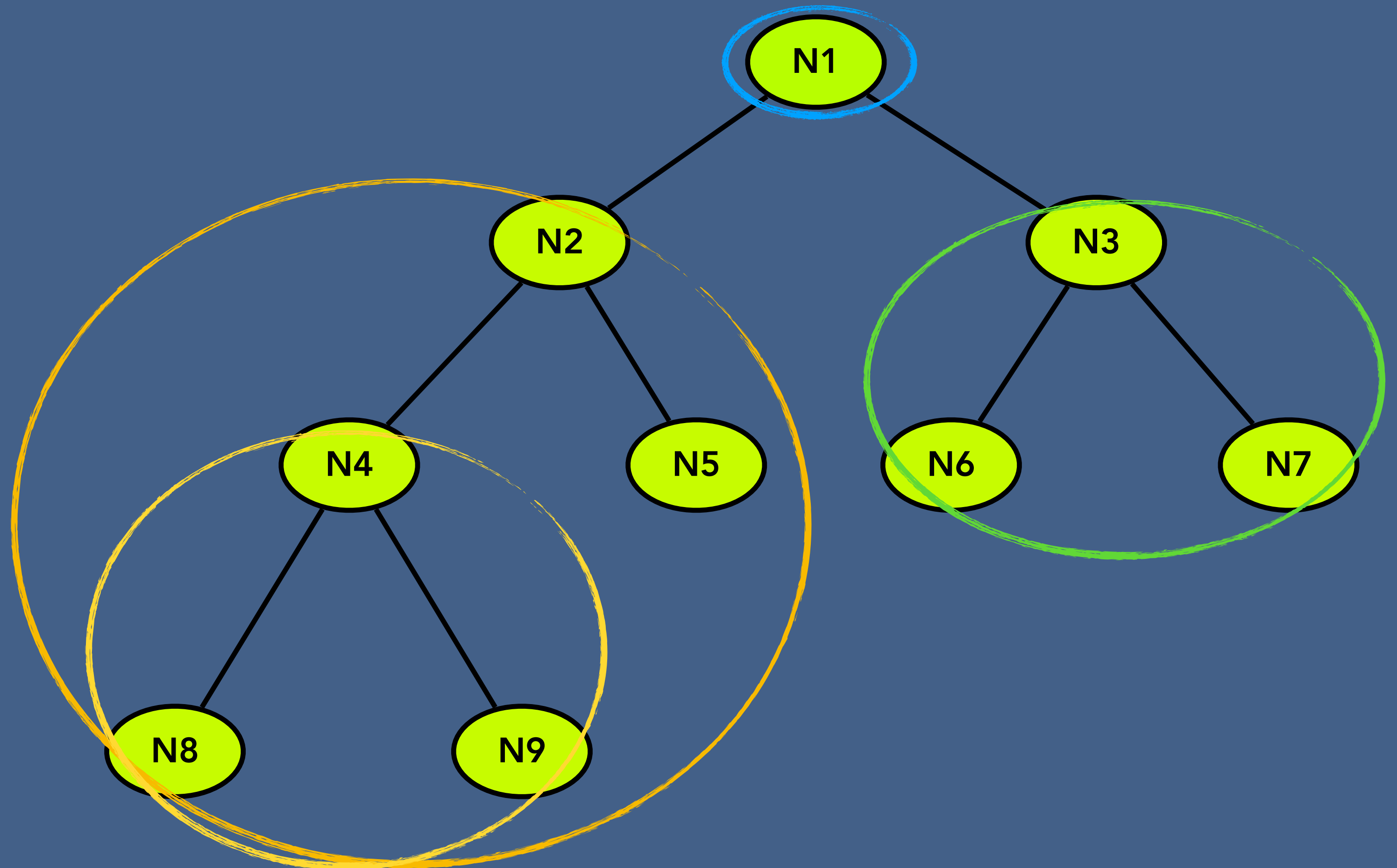
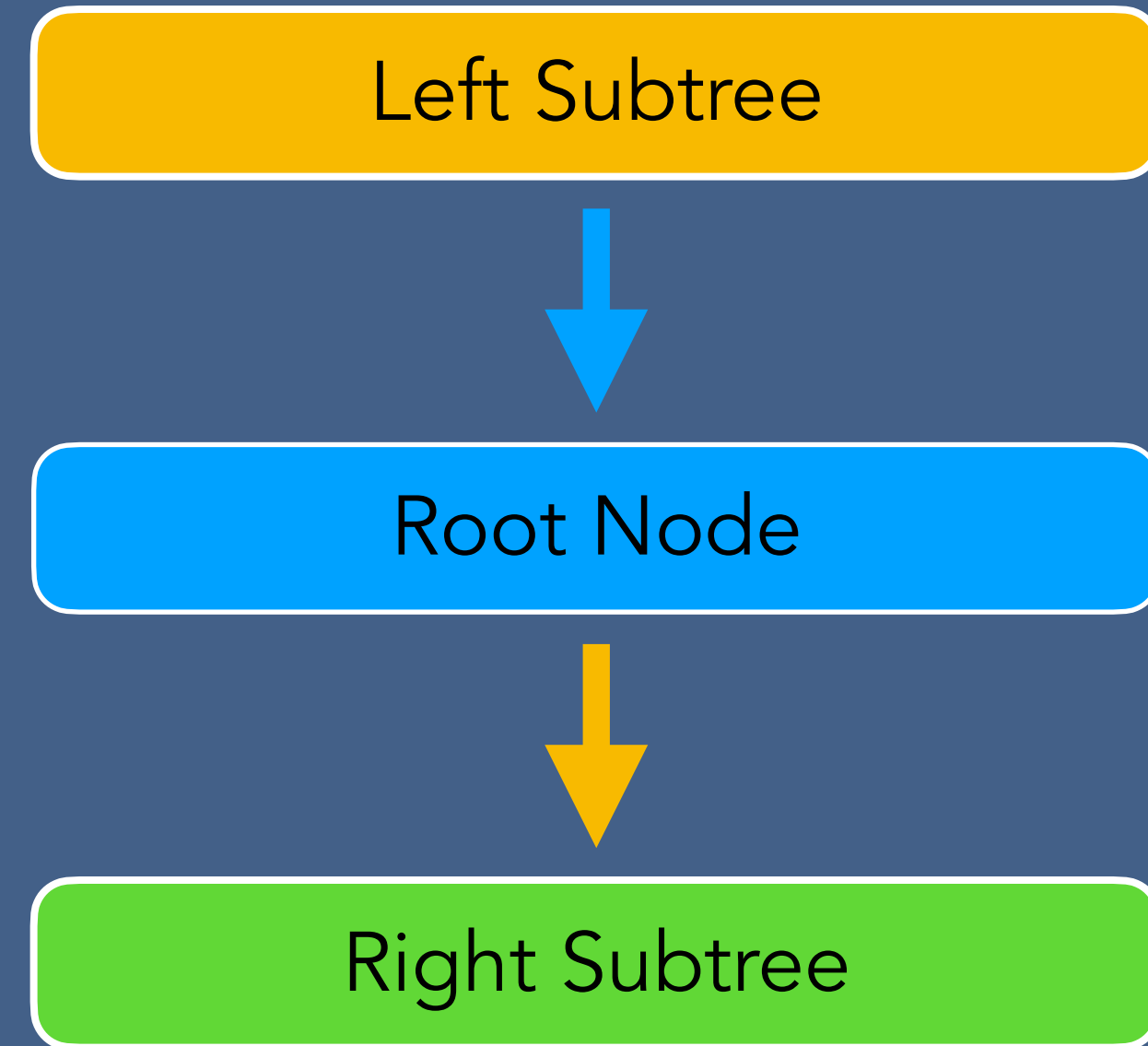
Binary Tree - PreOrder Traversal



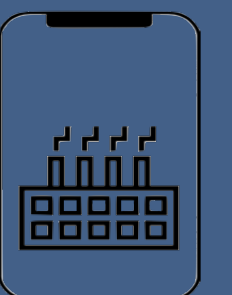
N1 → N2 → N4 → N8 → N9 → N5 → N3 → N6 → N7



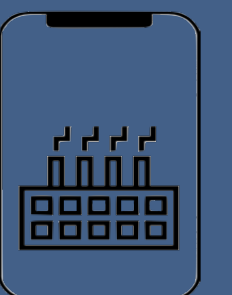
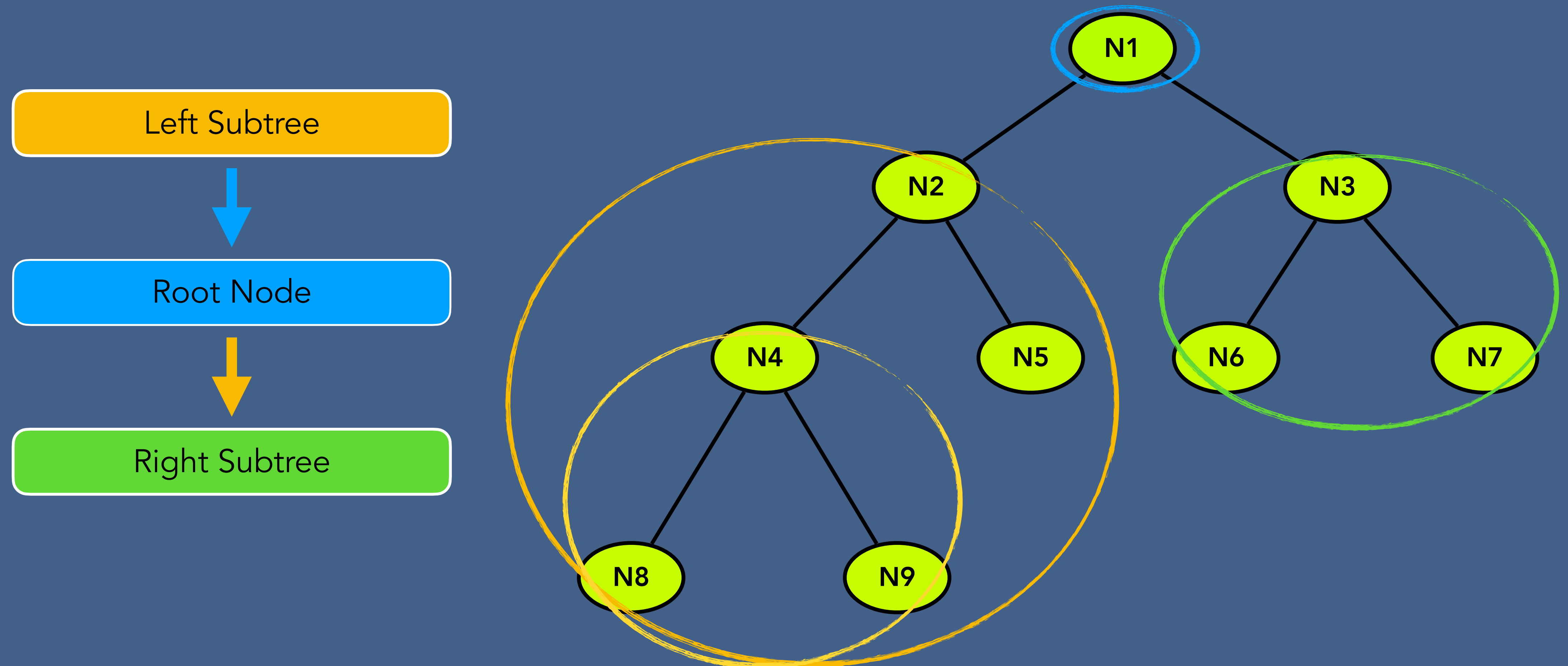
Binary Tree - InOrder Traversal



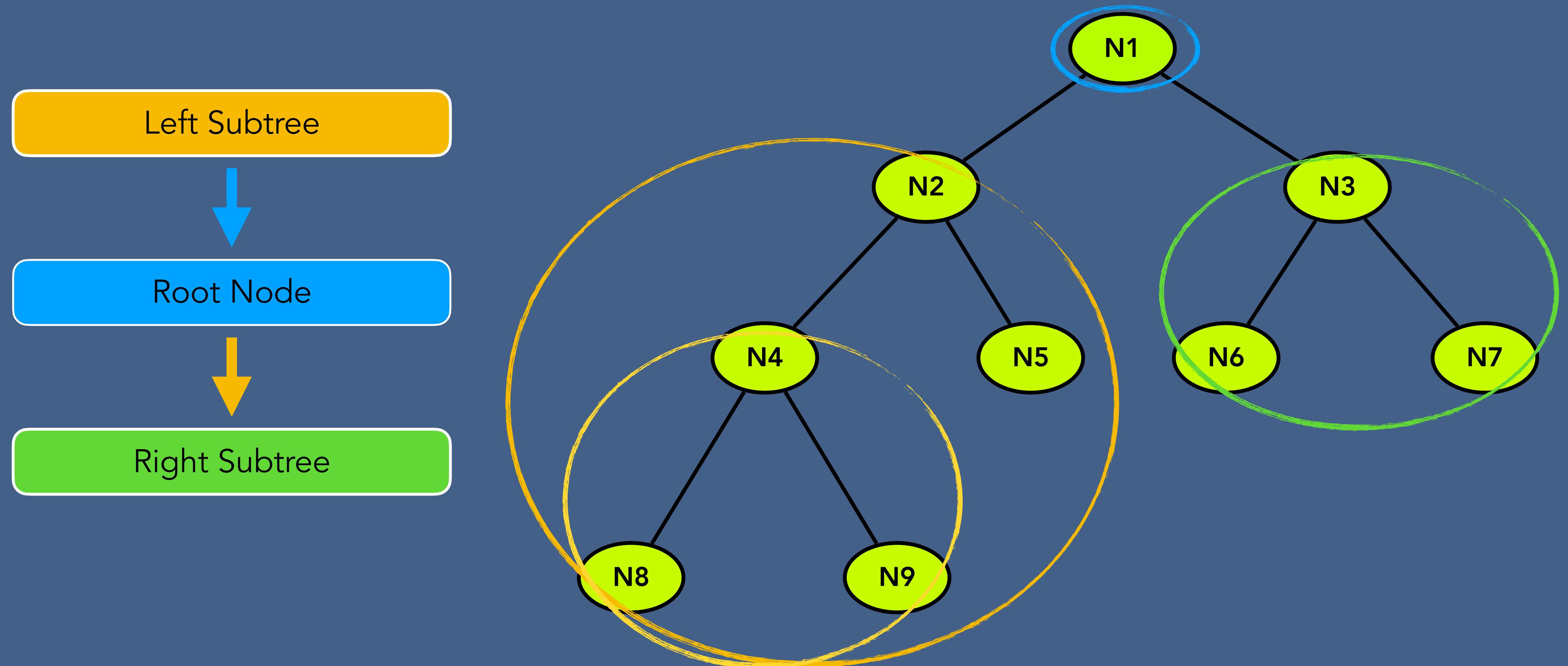
N8 → N4



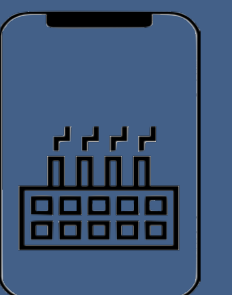
Binary Tree - InOrder Traversal



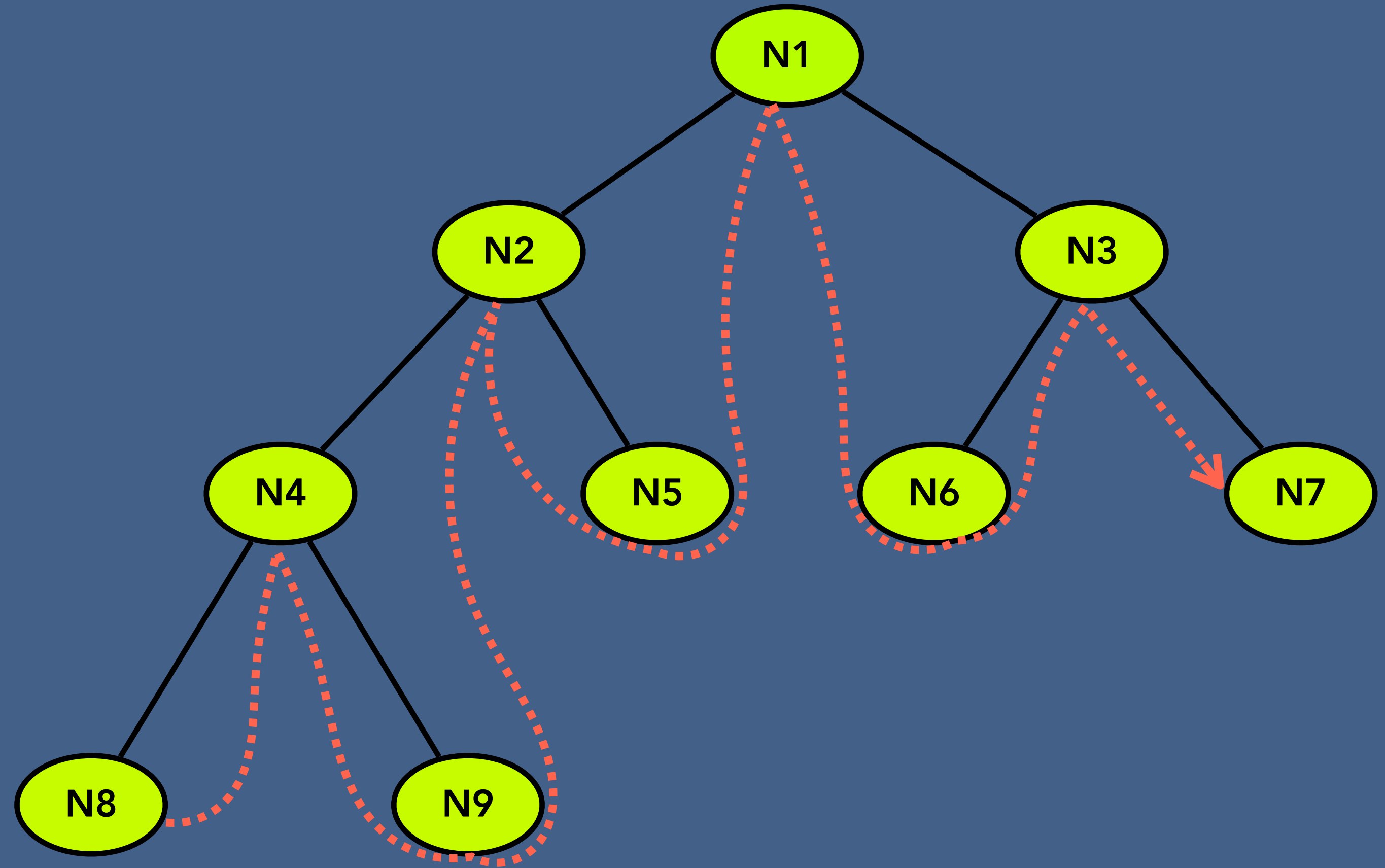
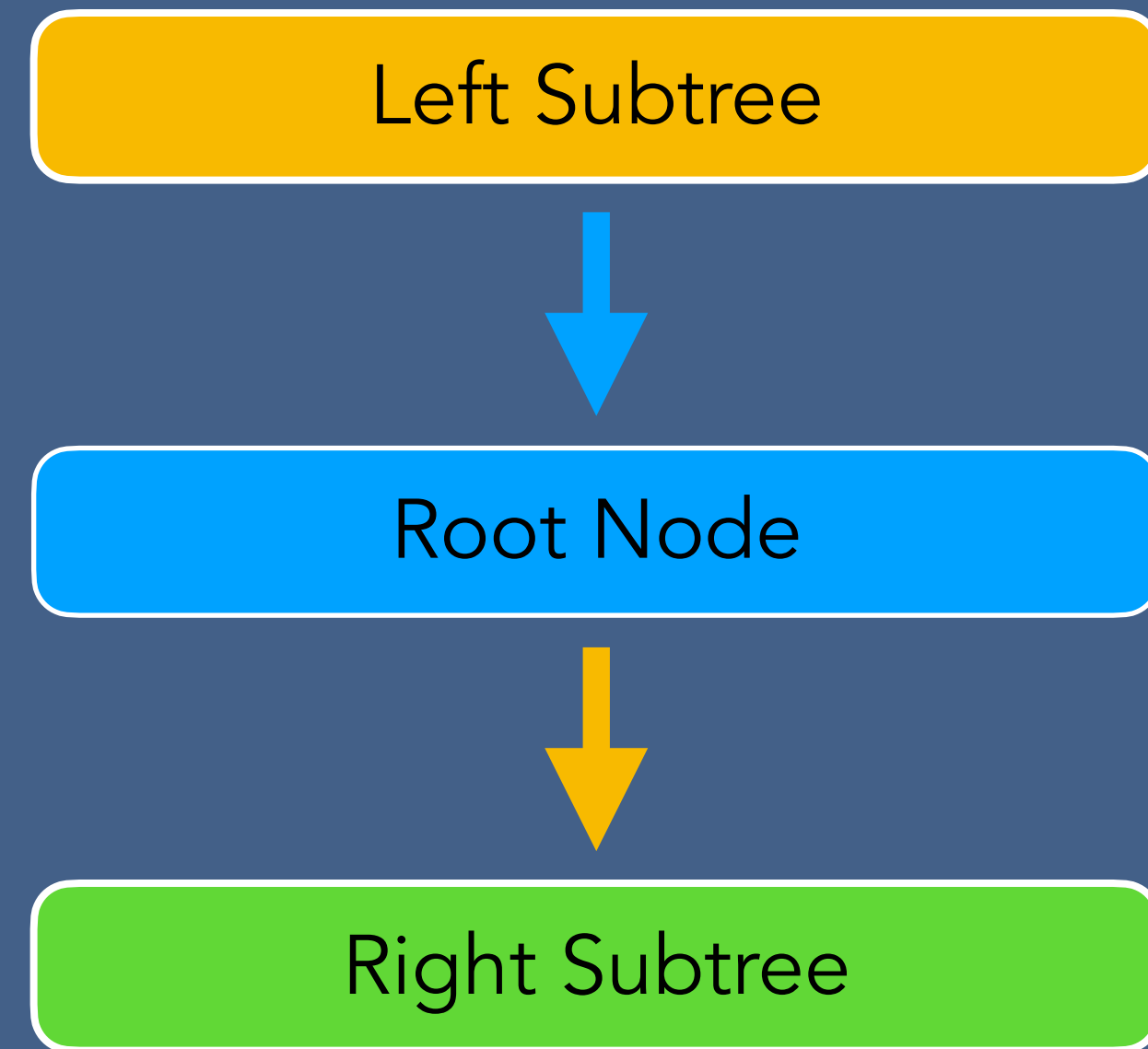
Binary Tree - InOrder Traversal



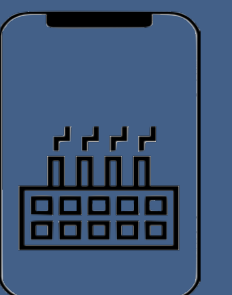
N9 → N4 → N8 → N2 → N5 → N1 → N6 → N3 → N7



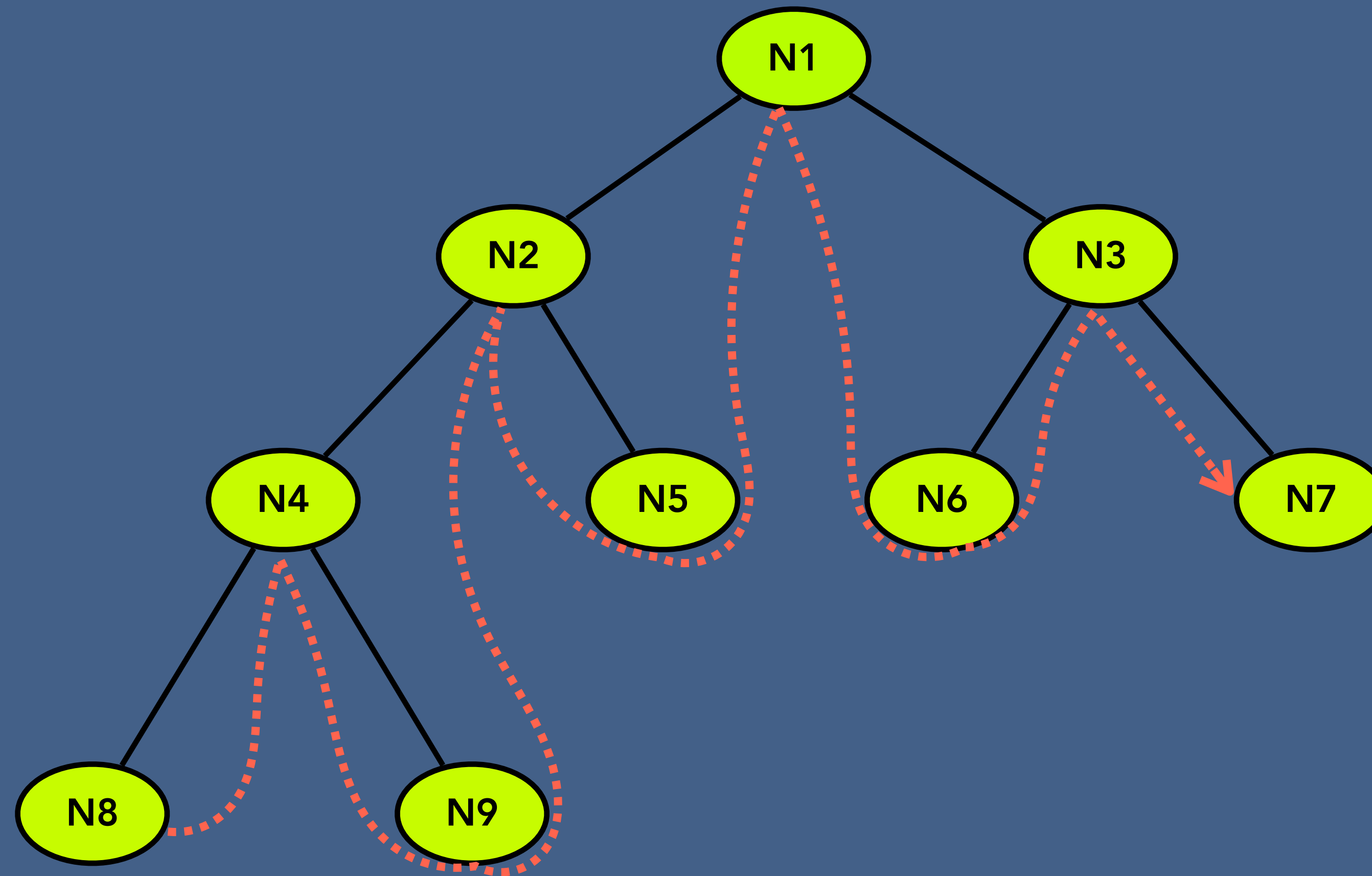
Binary Tree - InOrder Traversal



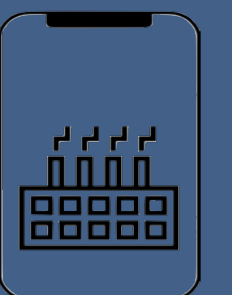
N9 → N4 → N8 → N2 → N5 → N1 → N6 → N3 → N7



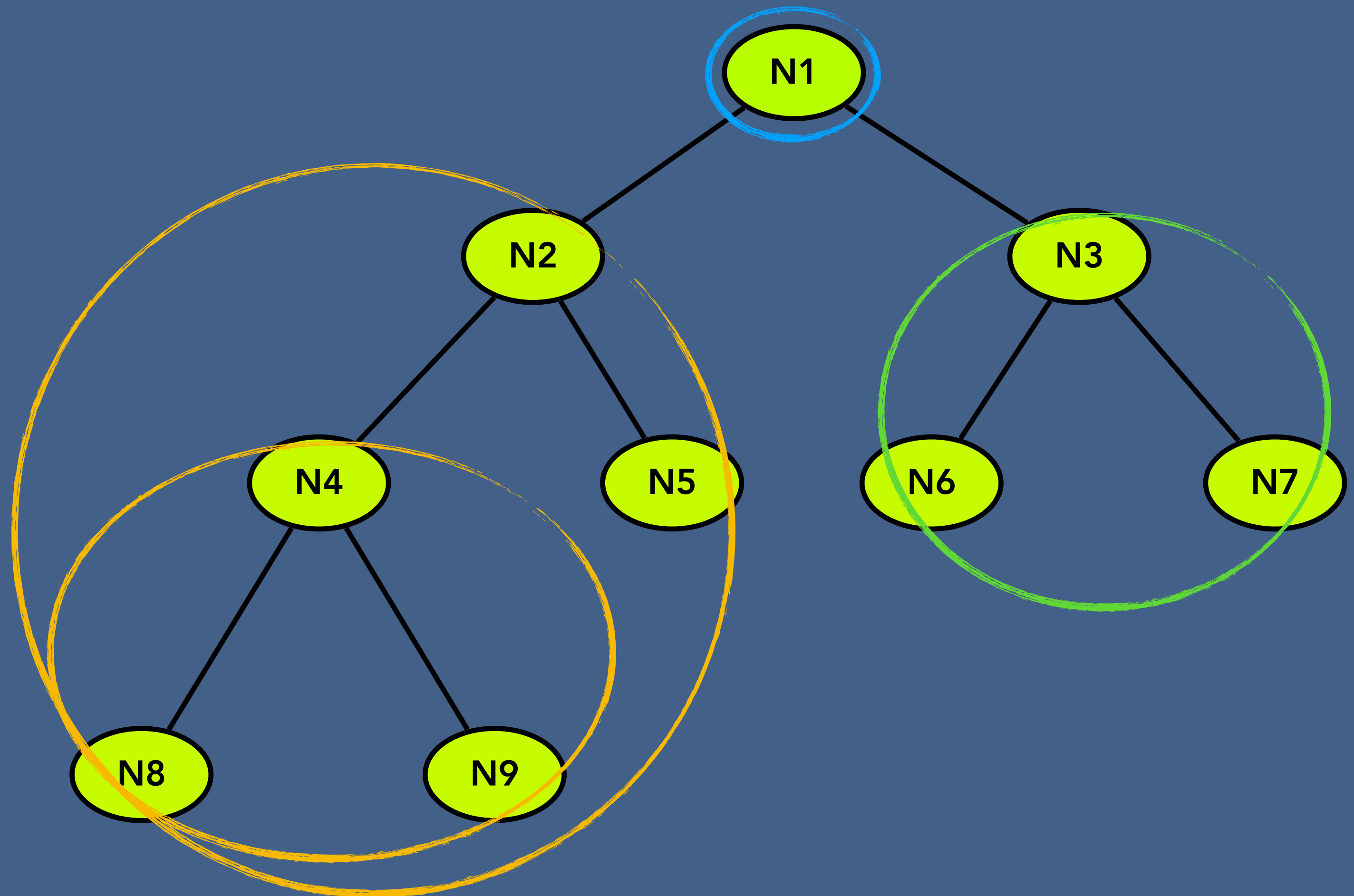
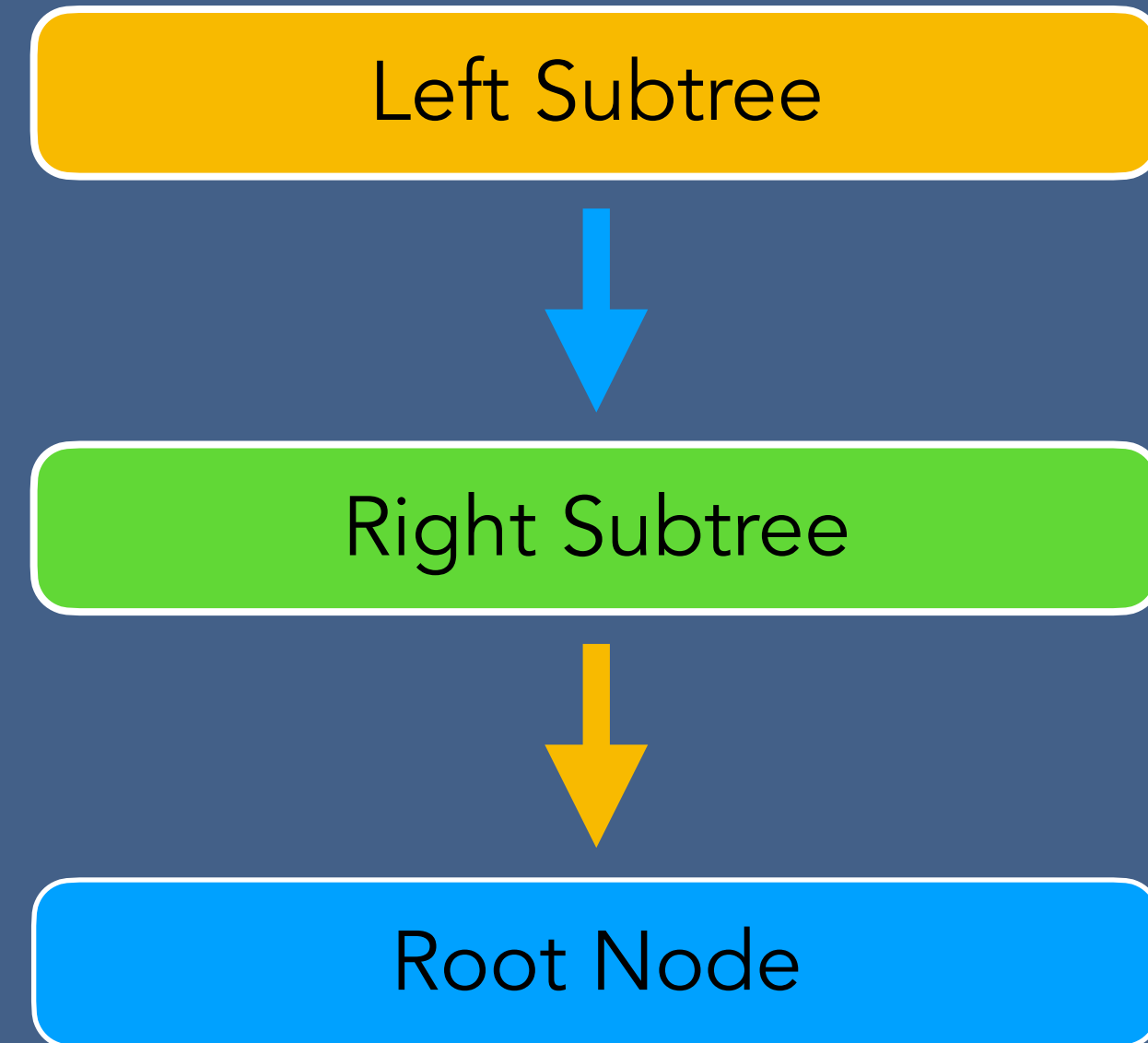
Binary Tree - InOrder Traversal



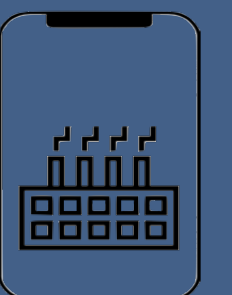
N8 → N4 → N9 → N2 → N5 → N1 → N6 → N3 → N7



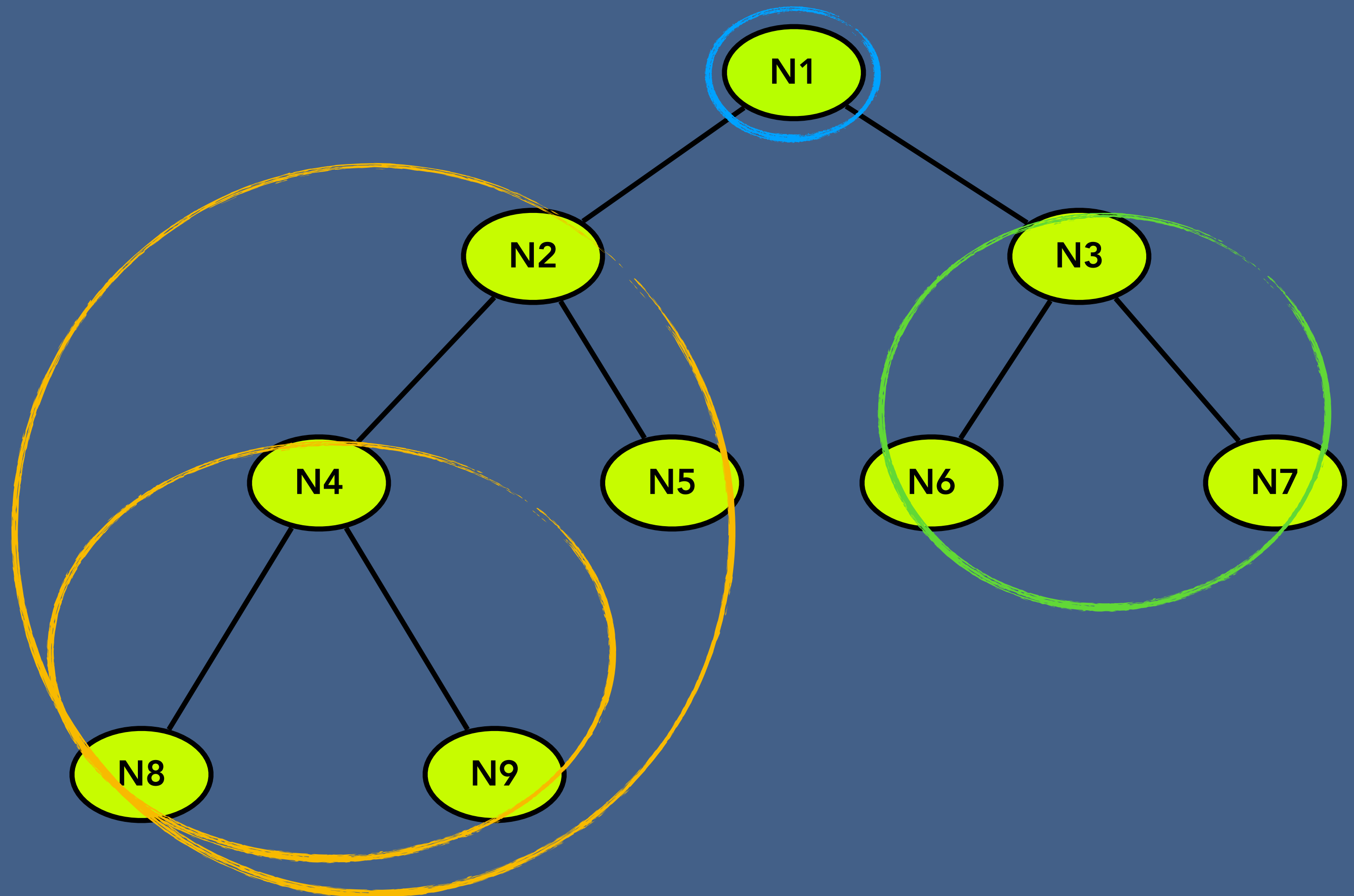
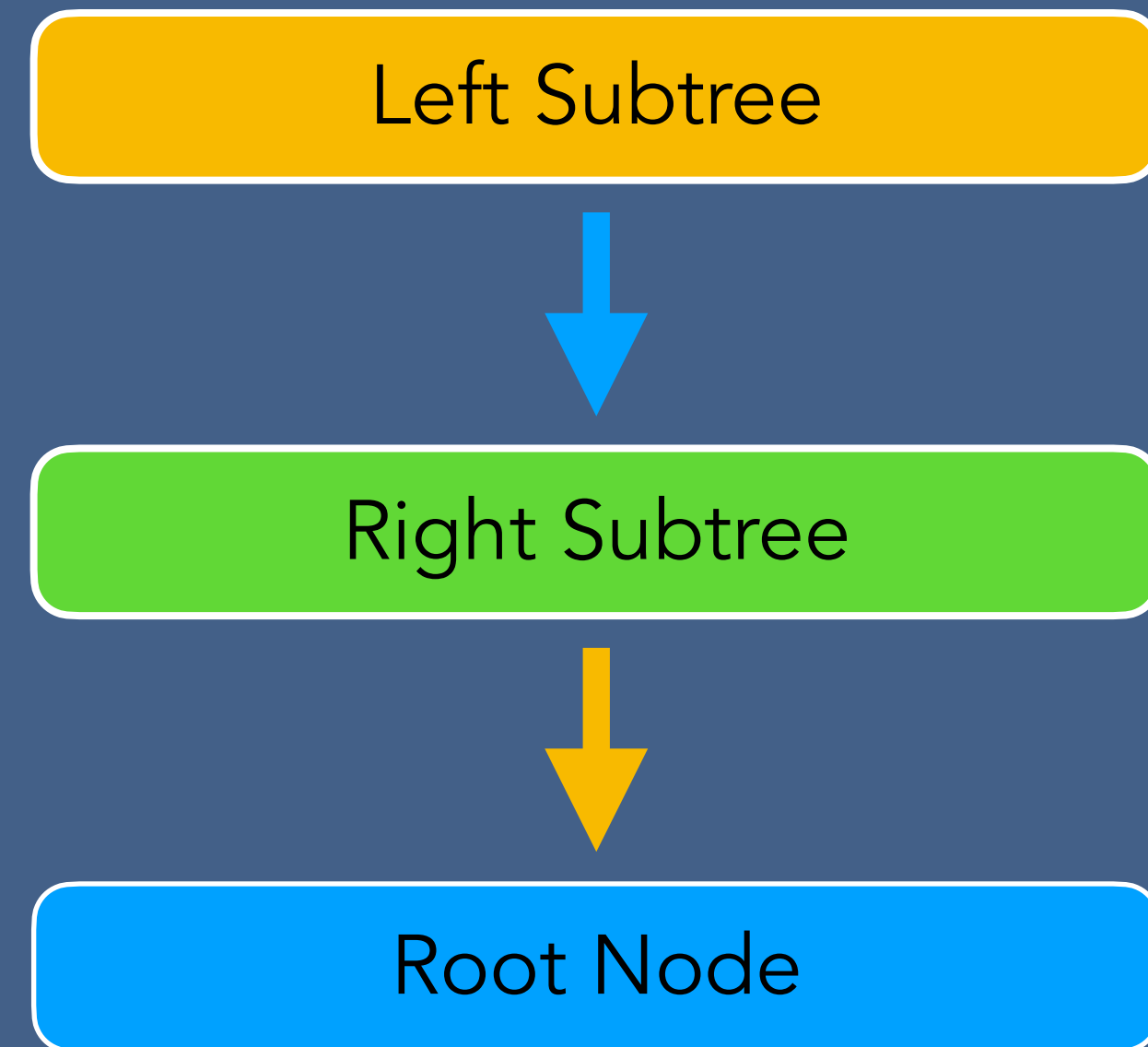
Binary Tree - Post Traversal



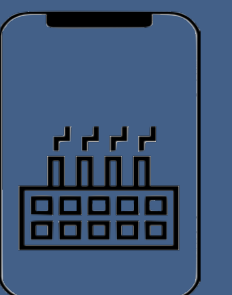
N8 → N9



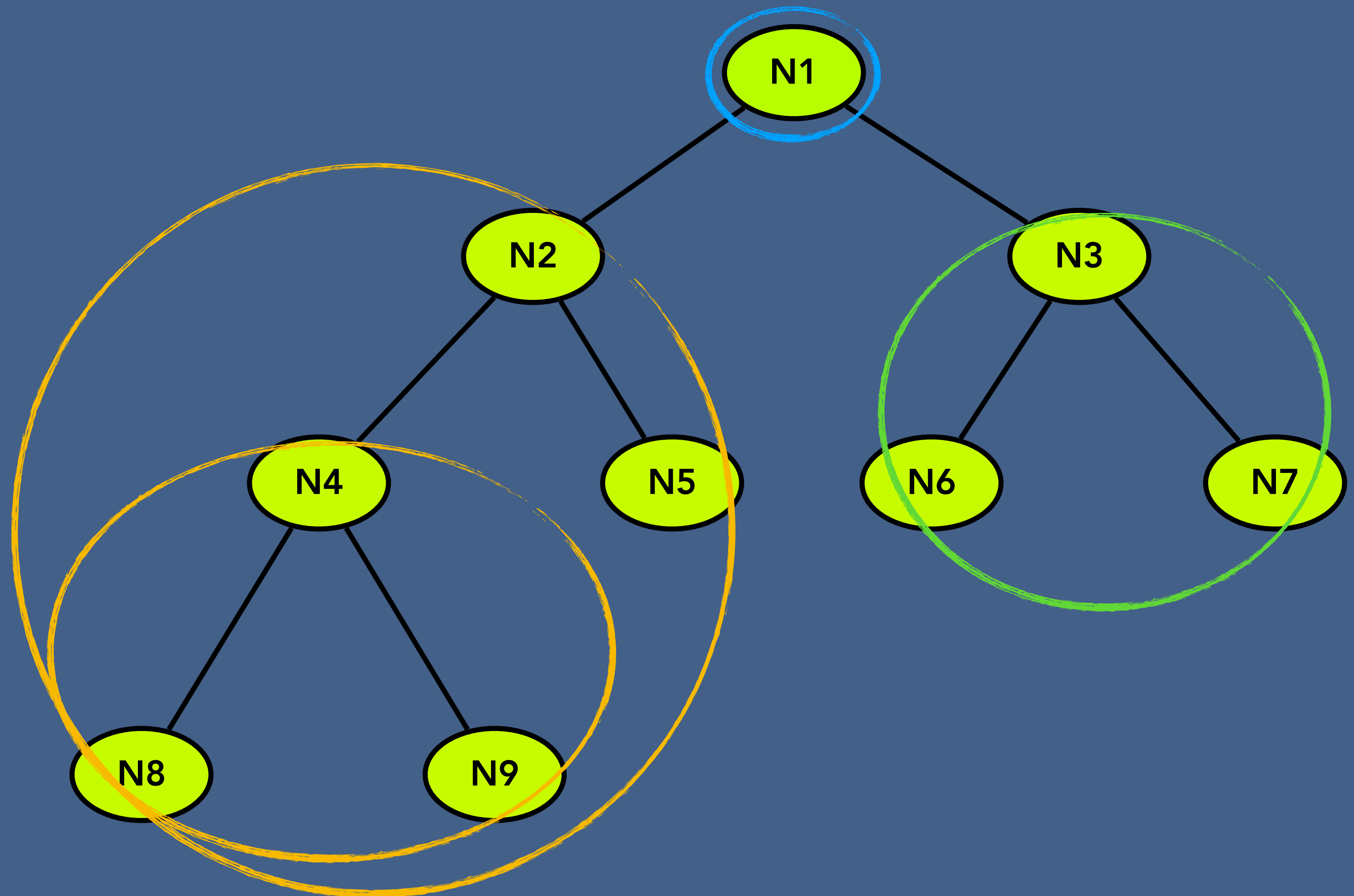
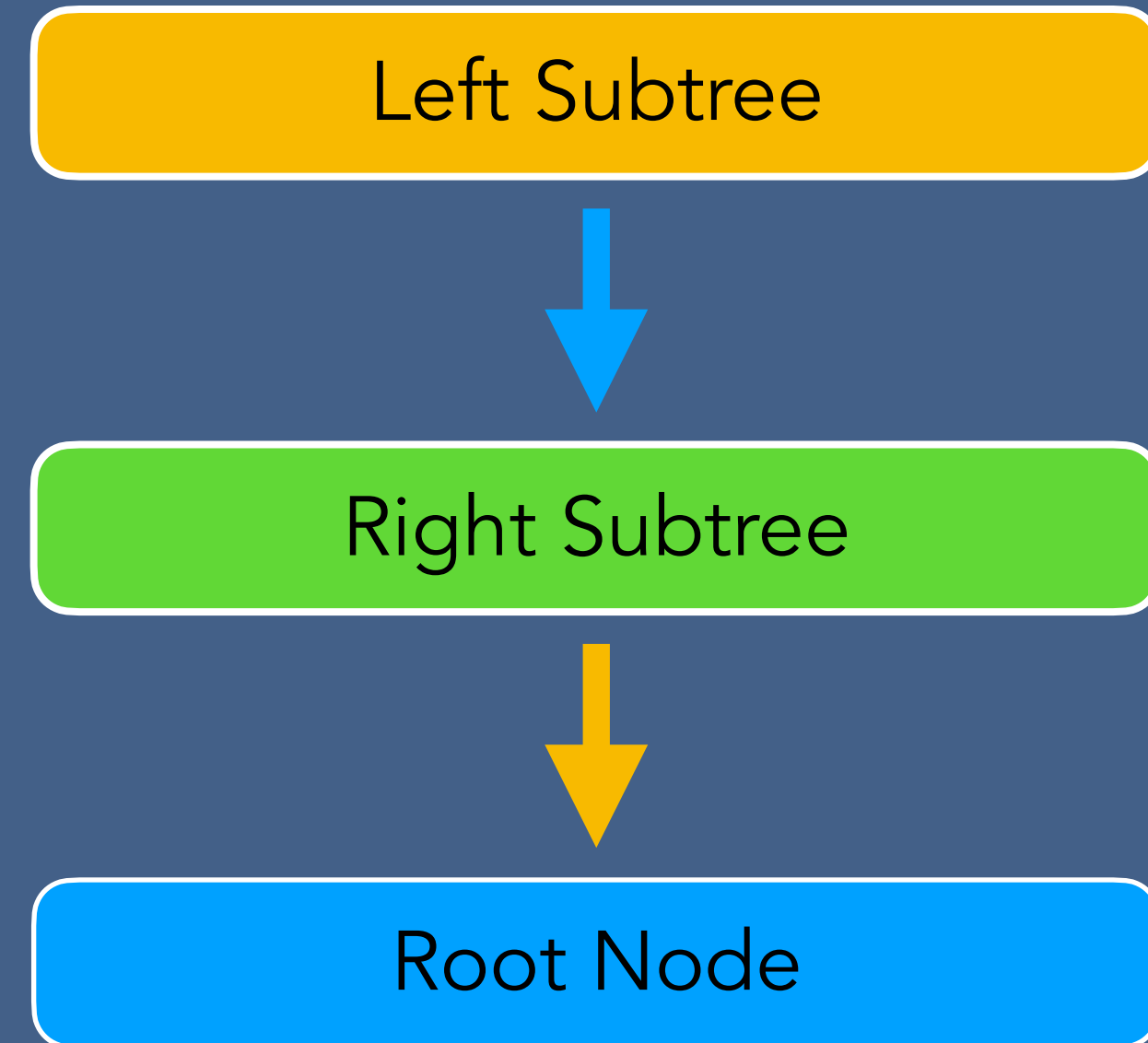
Binary Tree - Post Traversal



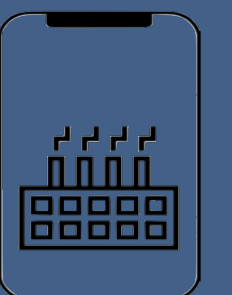
N8 → N9 → N4 → N5 → N2



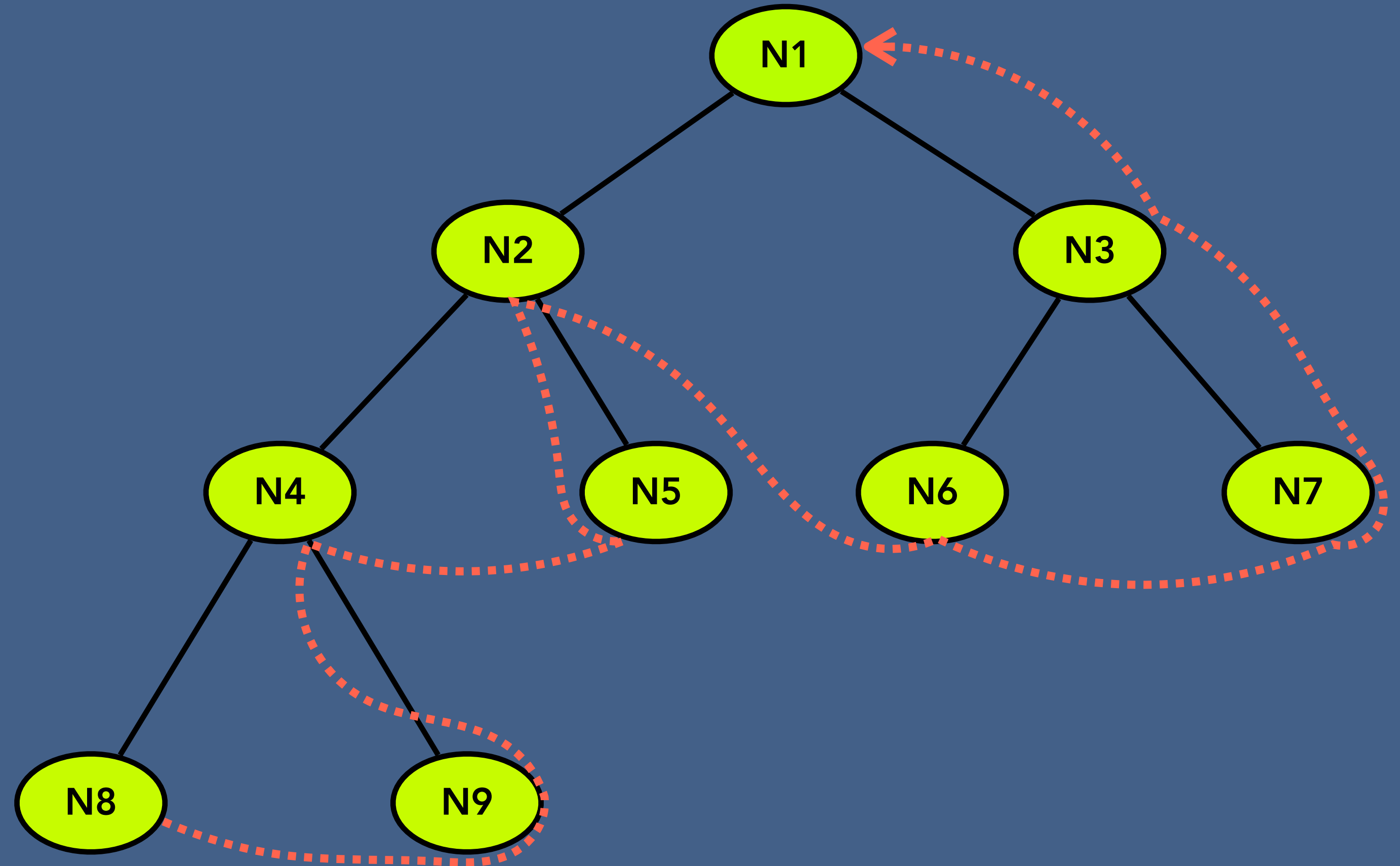
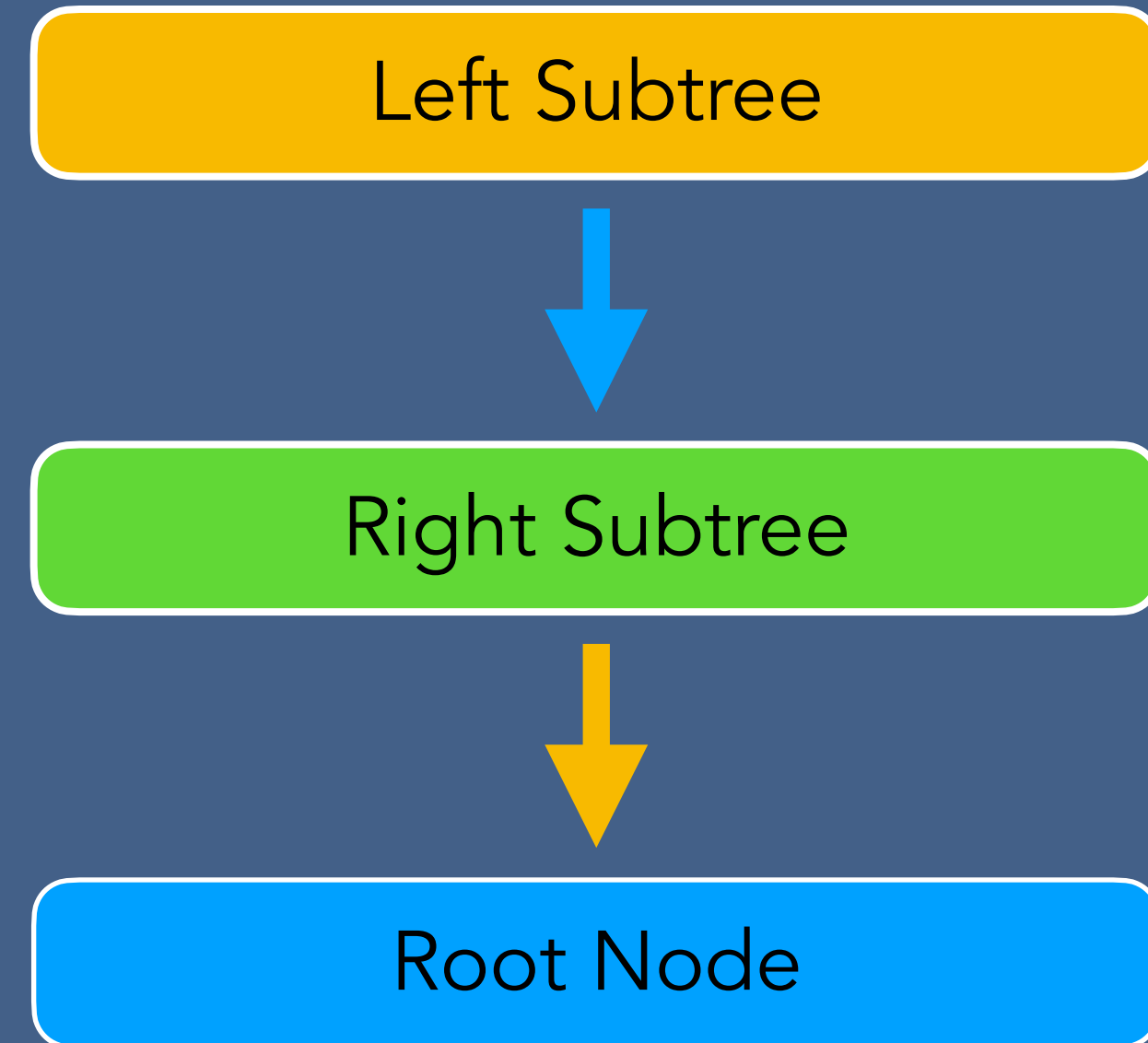
Binary Tree - Post Traversal



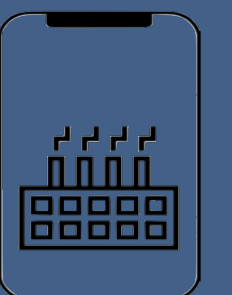
N8 → N9 → N4 → N5 → N2 → N6 → N7 → N3 → N1



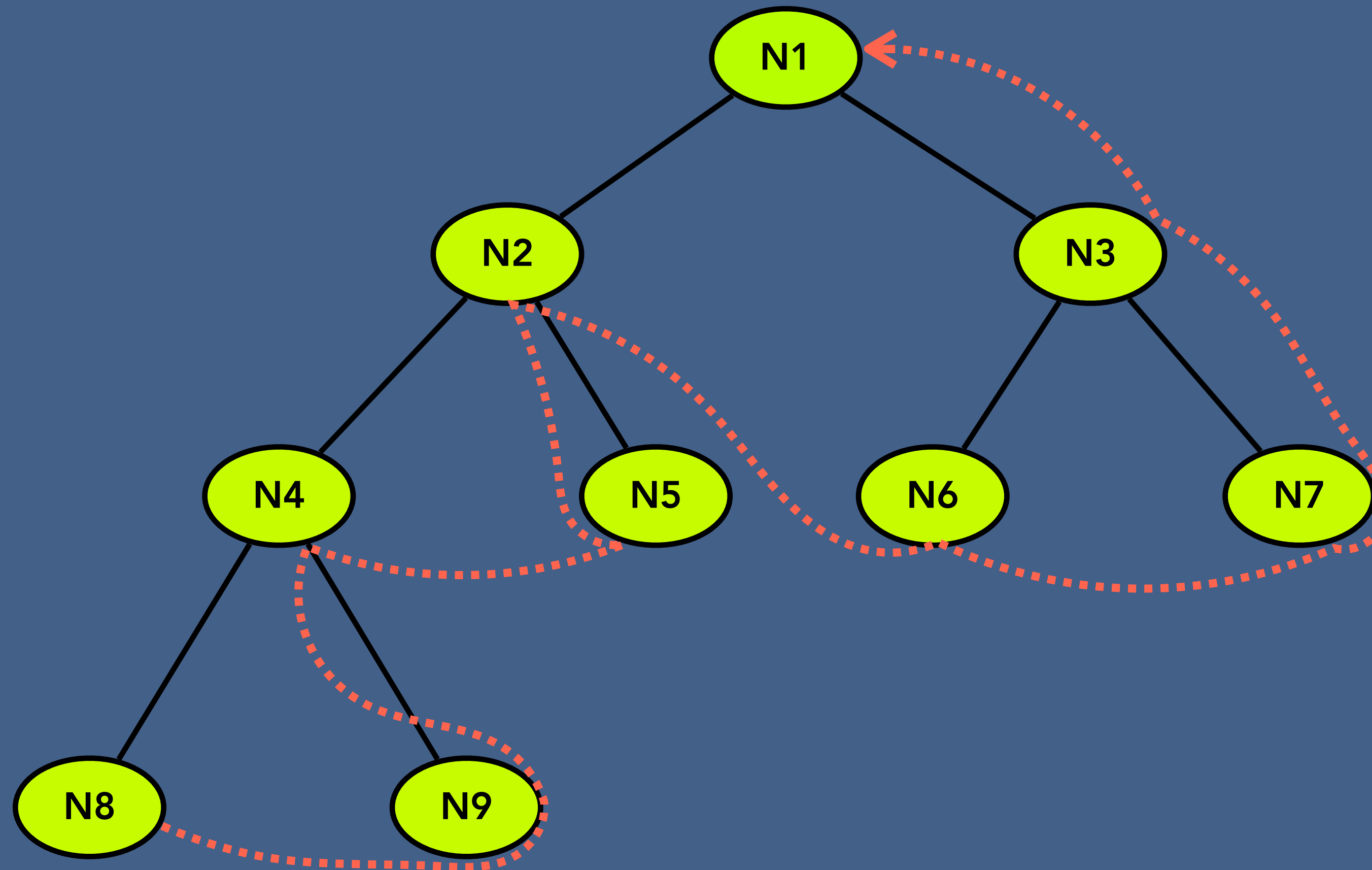
Binary Tree - Post Traversal



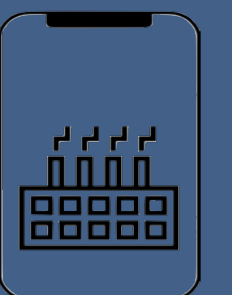
N8 → N9 → N4 → N5 → N2 → N6 → N7 → N3 → N1



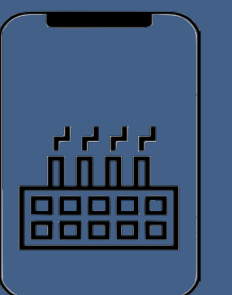
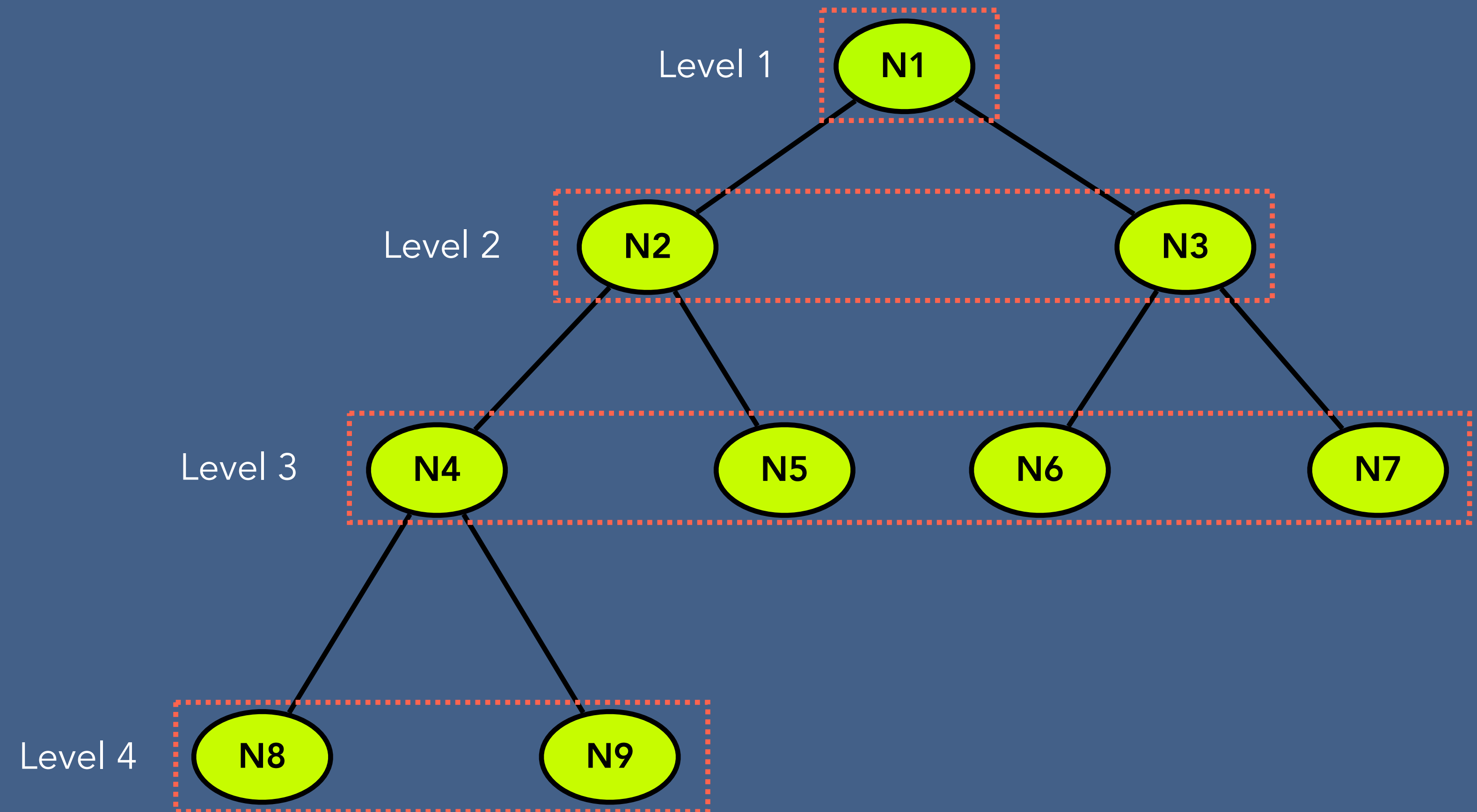
Binary Tree - Post Traversal



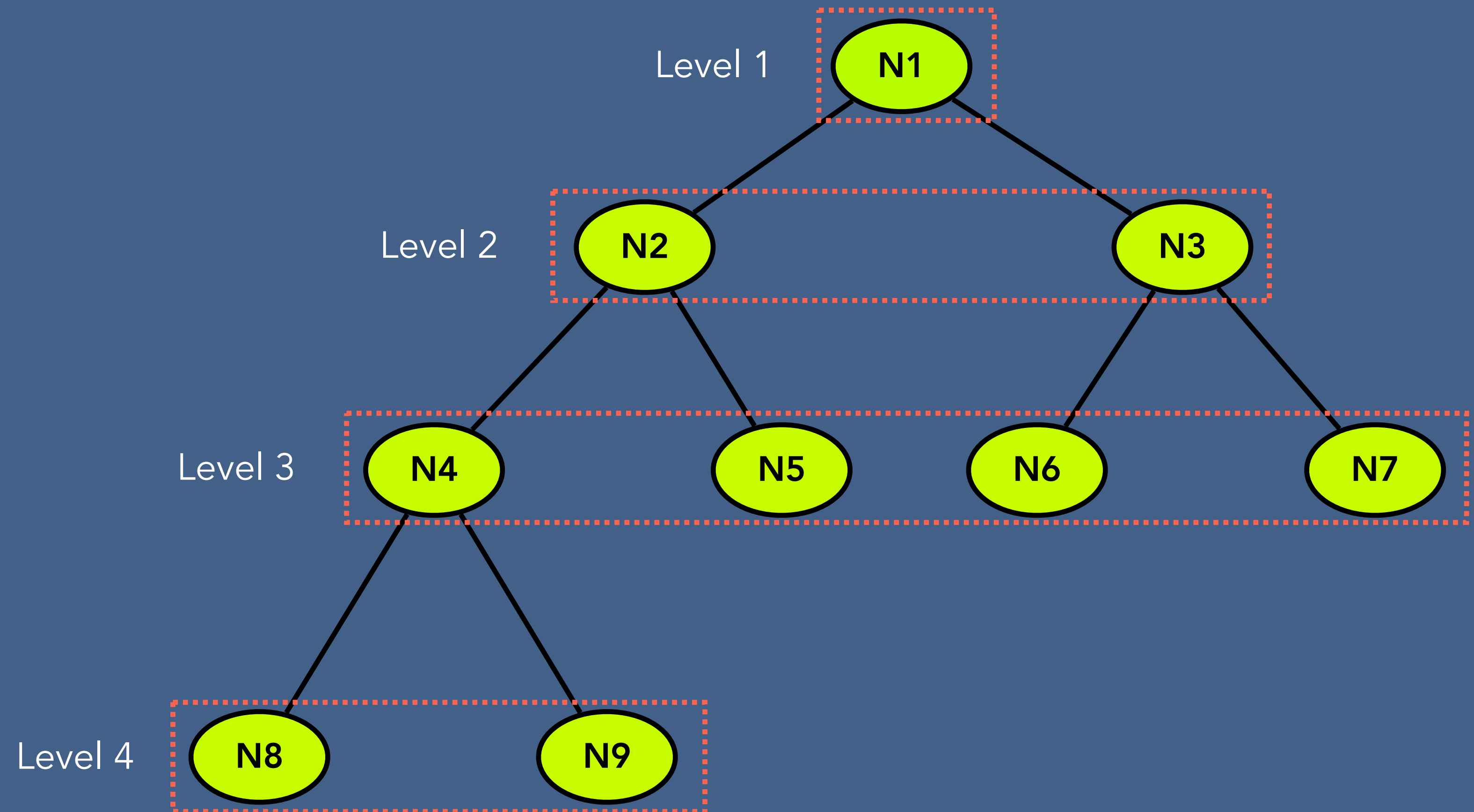
N8 → N9 → N4 → N5 → N2 → N6 → N7 → N3 → N1



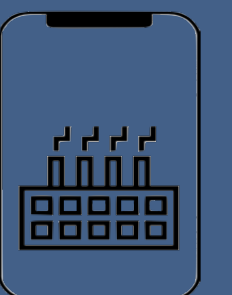
Binary Tree - LevelOrder Traversal



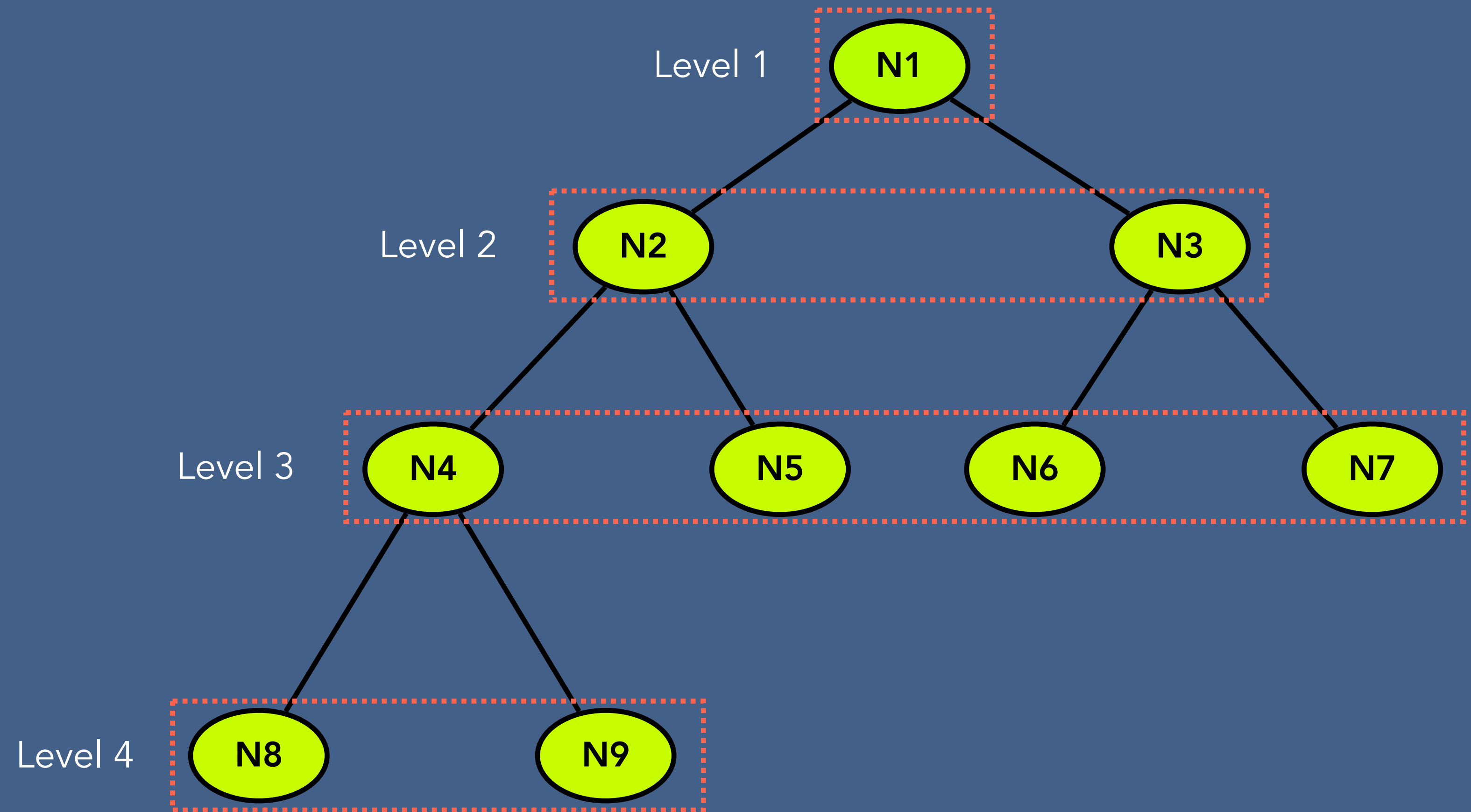
Binary Tree - LevelOrder Traversal



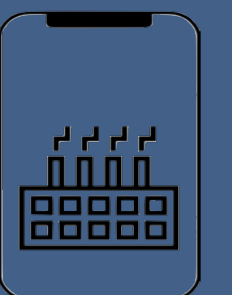
N1 → N2 → N3 → N4 → N5



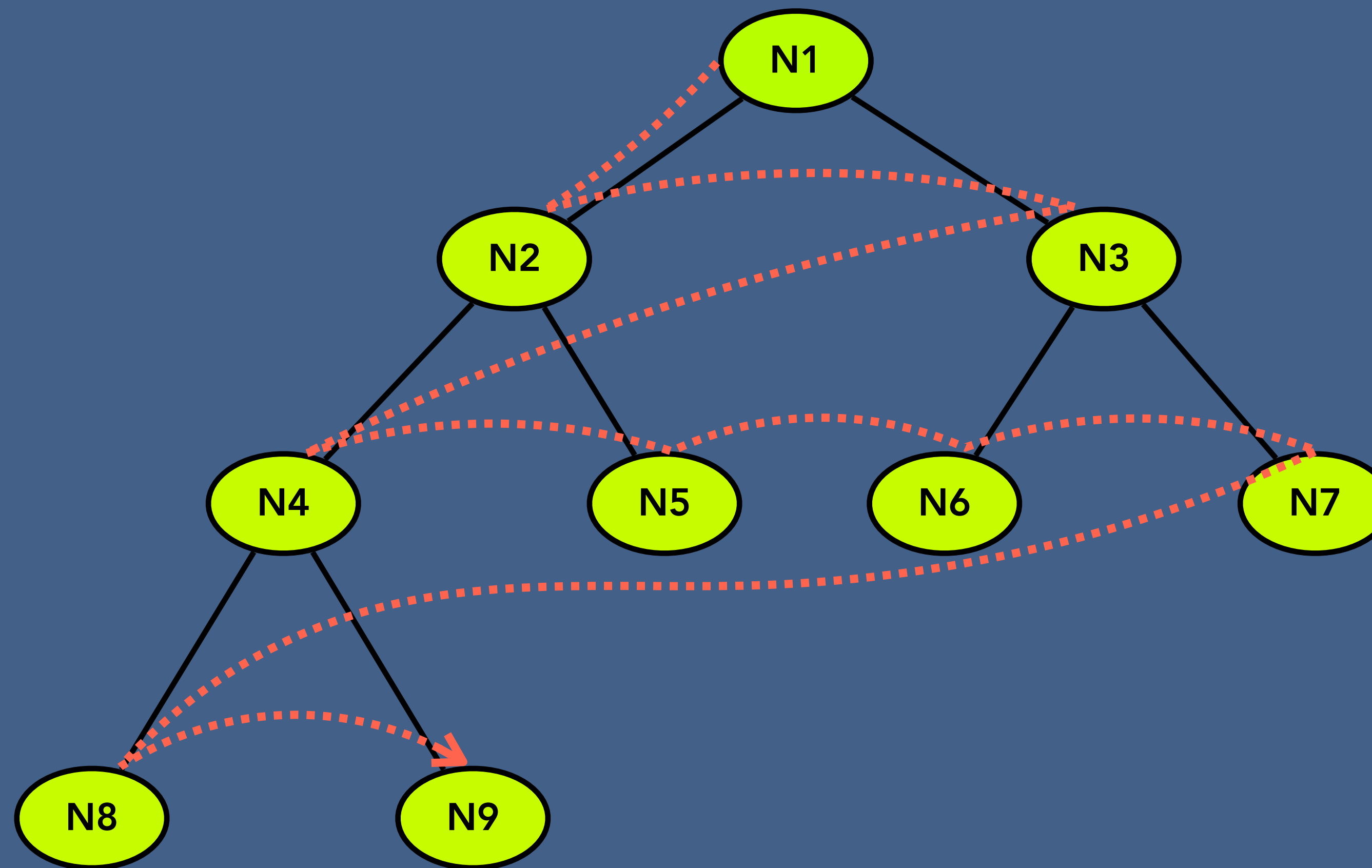
Binary Tree - LevelOrder Traversal



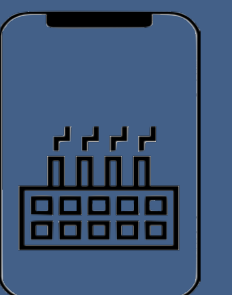
N1 → N2 → N3 → N4 → N5 → N6 → N7 → N8 → N9



Binary Tree - LevelOrder Traversal



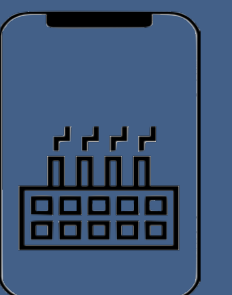
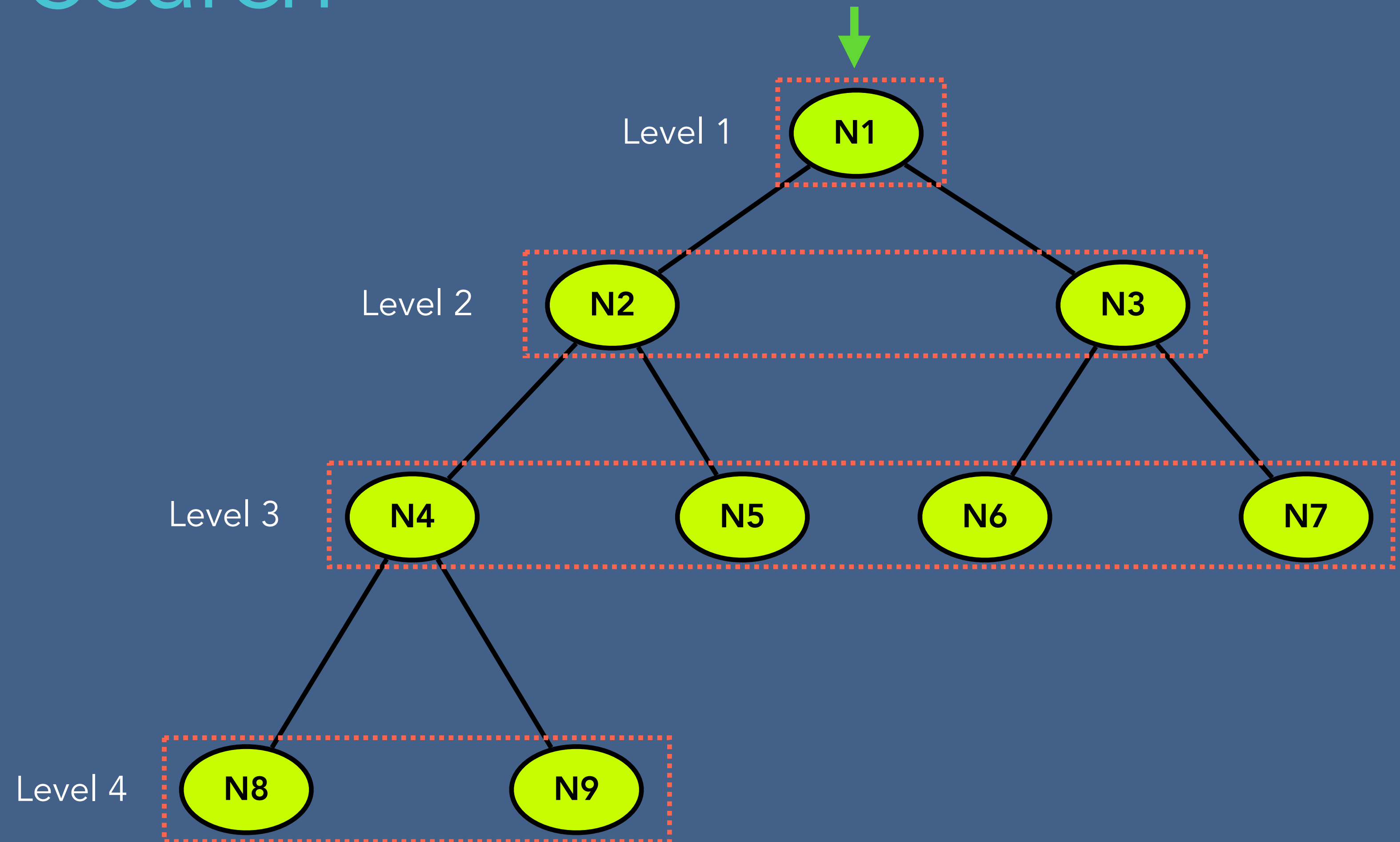
N1 → N2 → N3 → N4 → N5 → N6 → N7 → N8 → N9



Binary Tree - Search

Level Order Traversal

N5 → Success

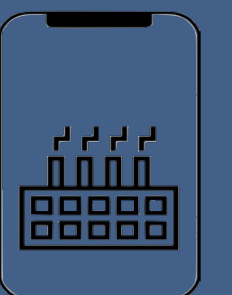
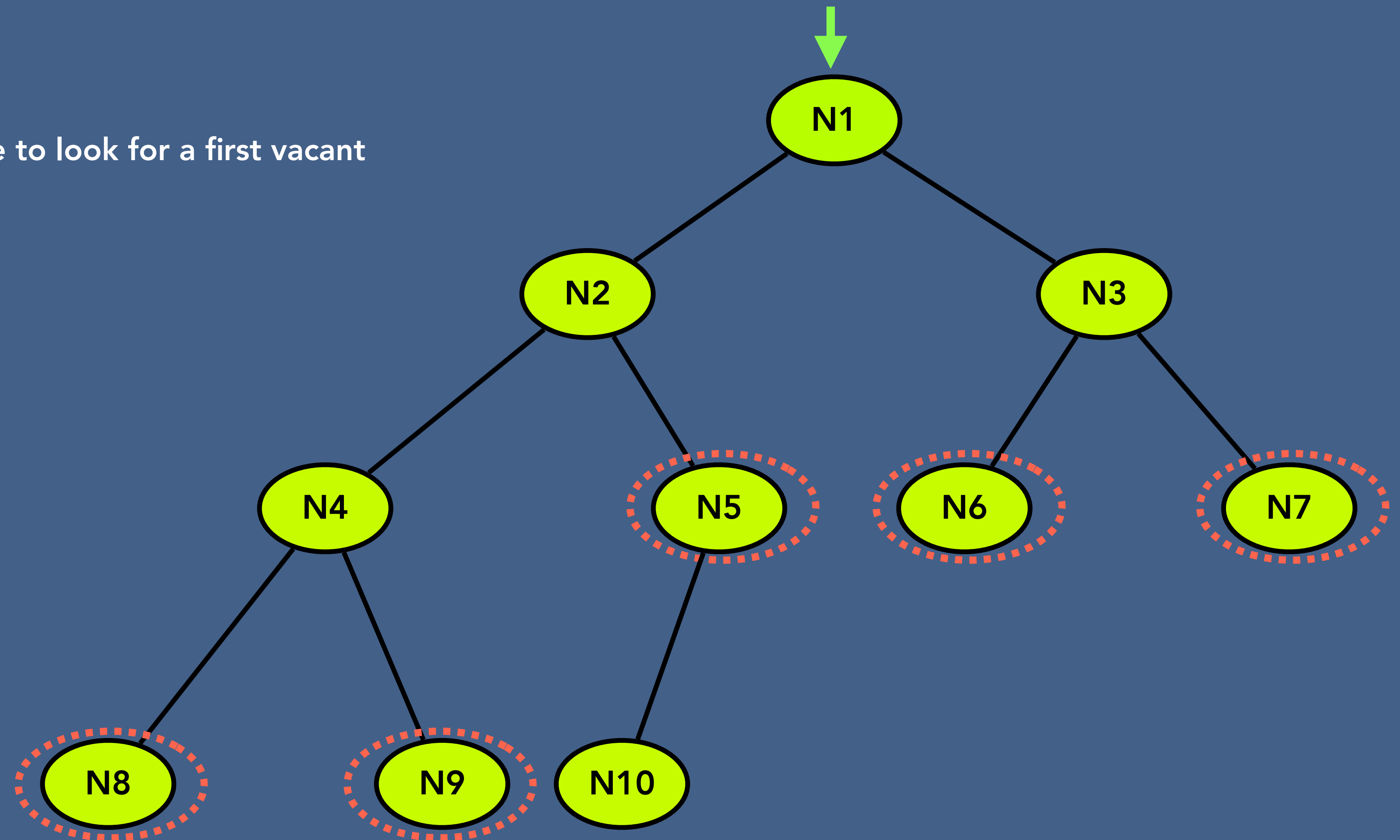


Binary Tree - Insert a Node

- A root node is null
- The tree exists and we have to look for a first vacant place

Level Order Traversal

newNode



Binary Tree - Delete a Node

Level Order Traversal

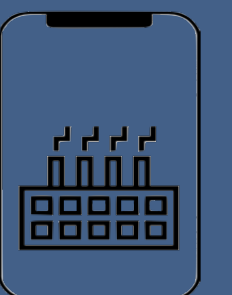
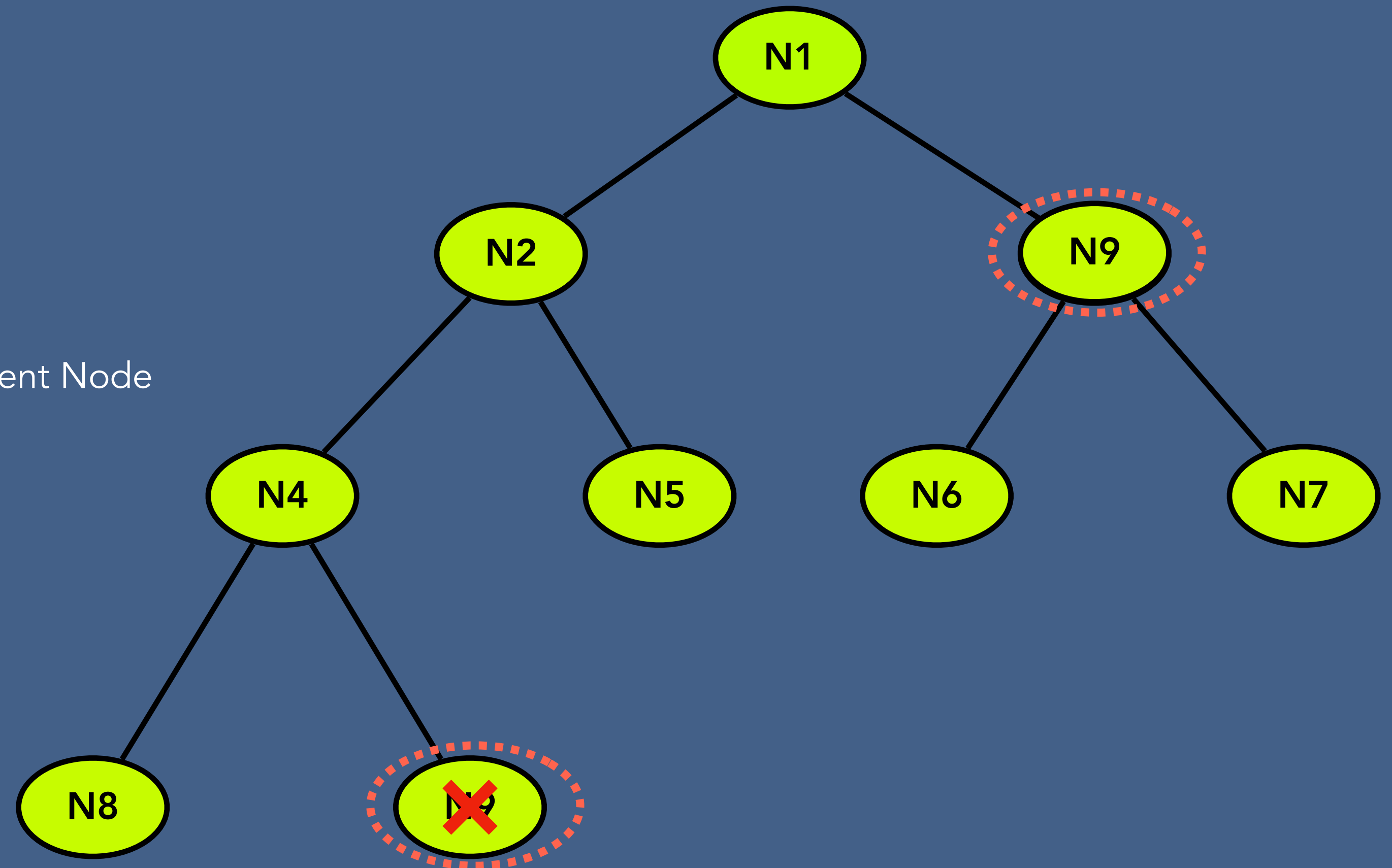
N3

Step 1 - Find the Node

Step 2 - Find Deepest Node

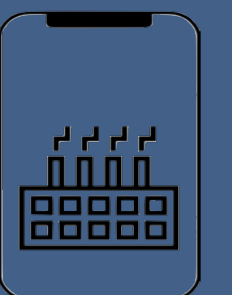
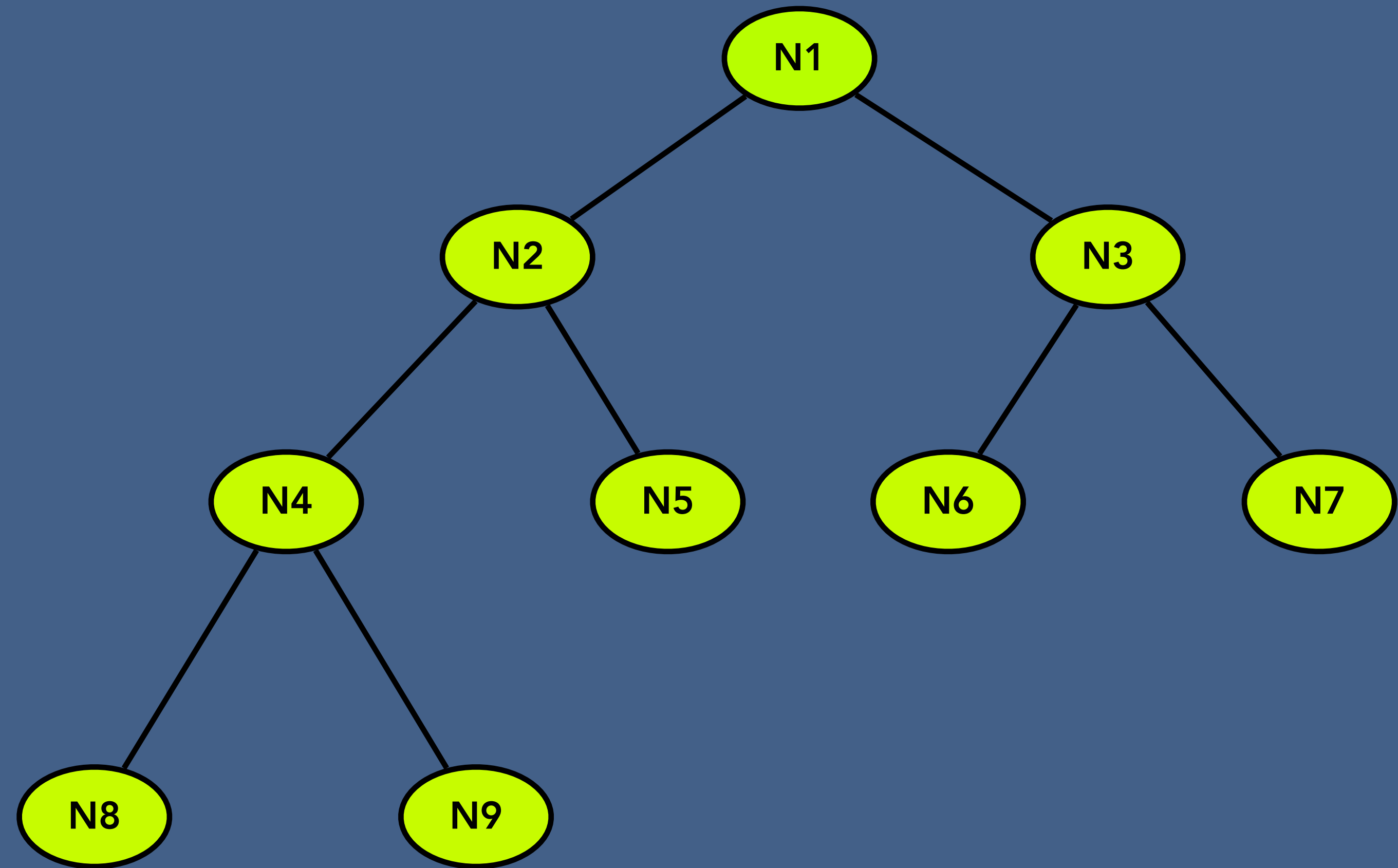
Step 3 - Set Deepest Node's value to Current Node

Step 4 - Delete Deepest Node



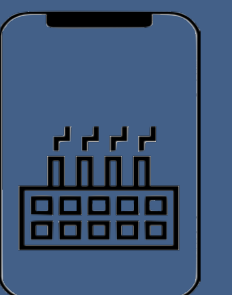
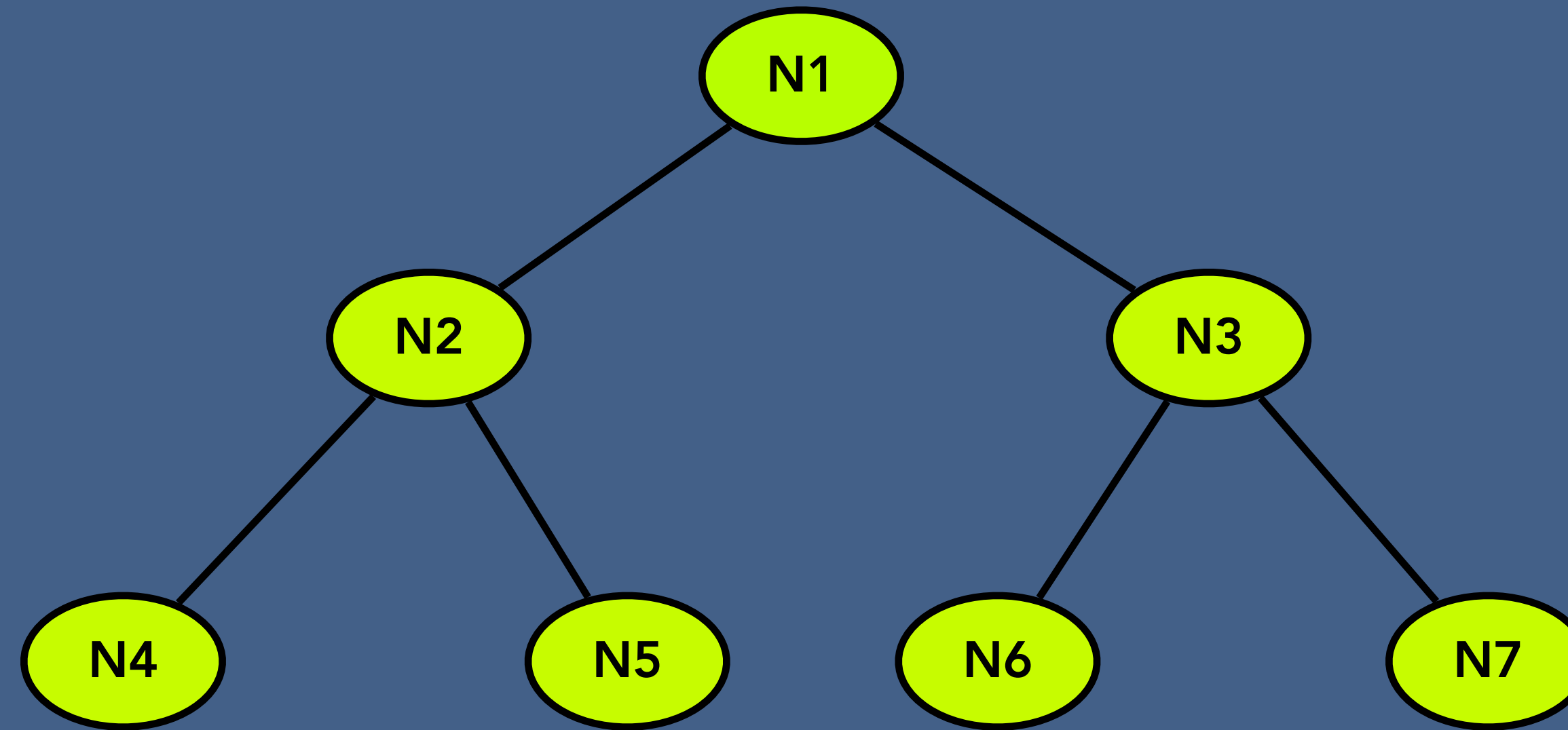
Binary Tree - Delete Binary Tree

rootNode = Null

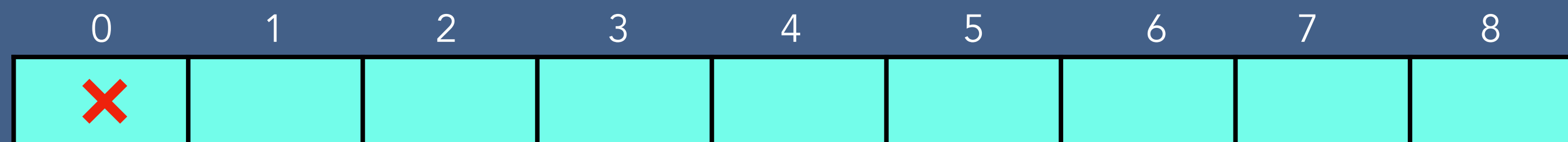
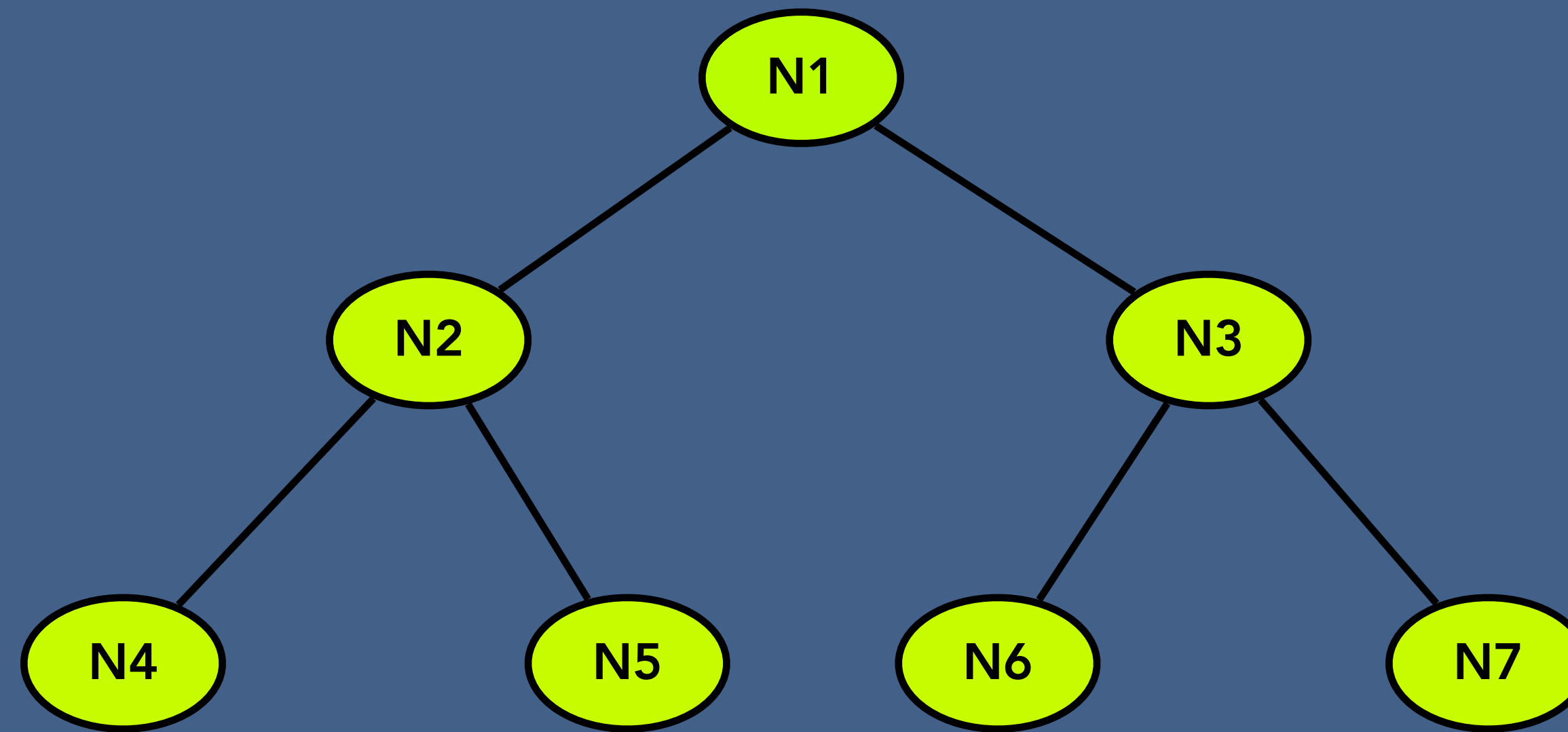


Binary Tree using Array

- Creation of Tree
- Insertion of a node
- Deletion of a node
- Search for a value
- Traverse all nodes
- Deletion of tree

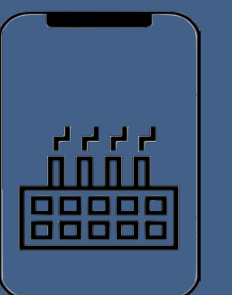


Binary Tree using Array

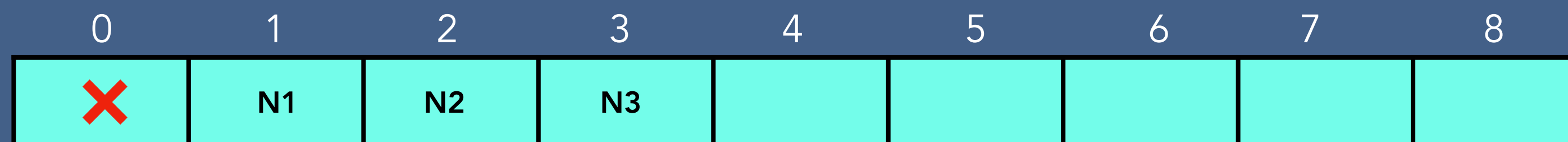
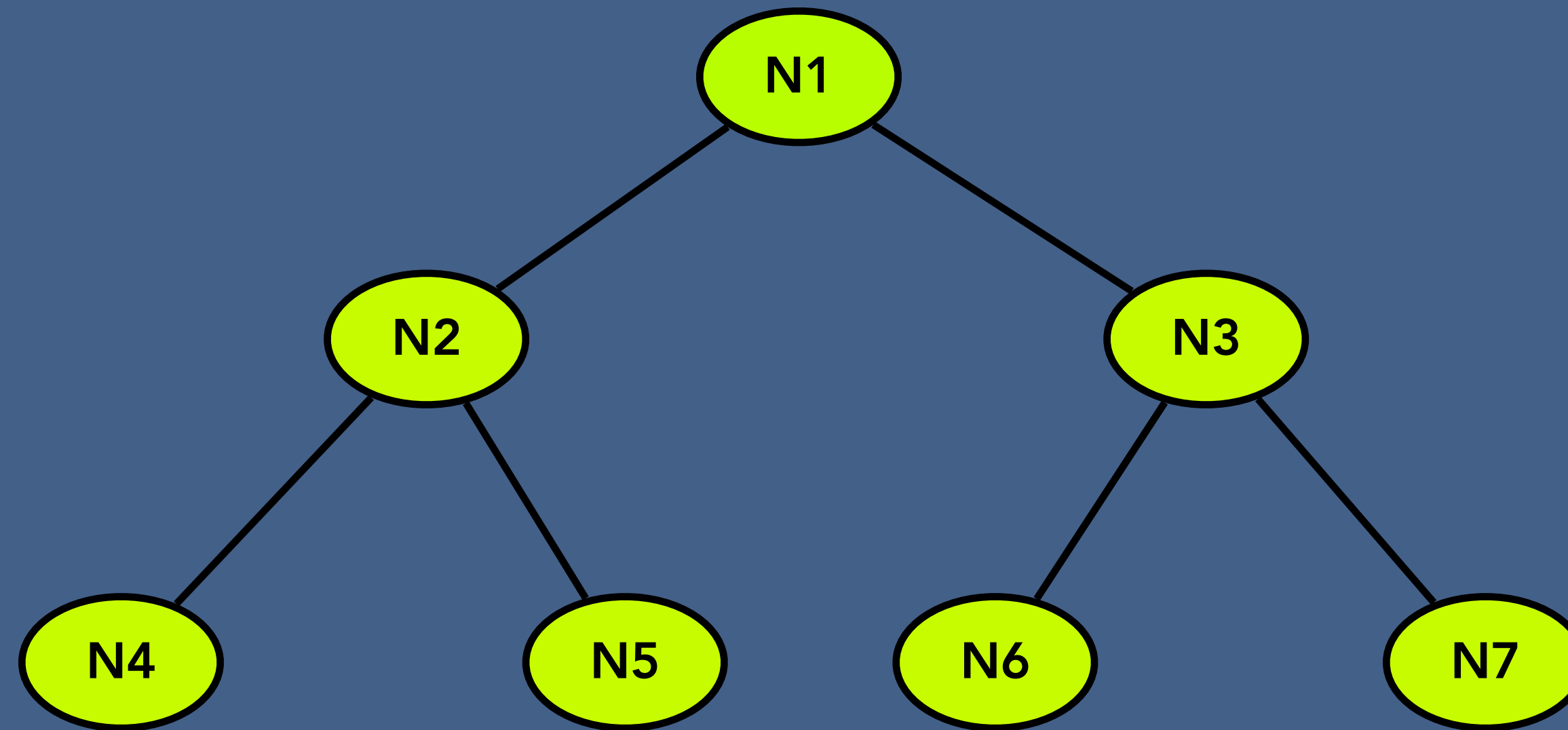


Left child = $\text{cell}[2x]$

Right child = $\text{cell}[2x+1]$

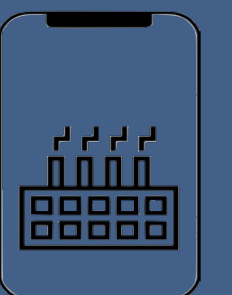


Binary Tree using Array

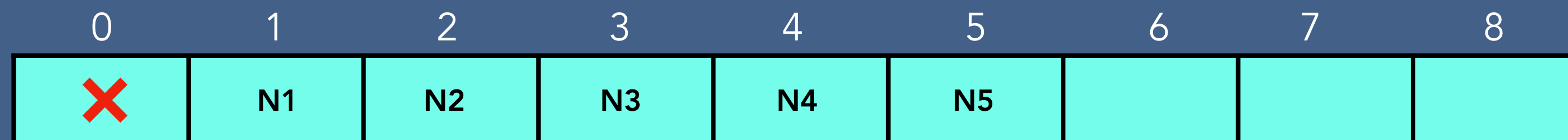
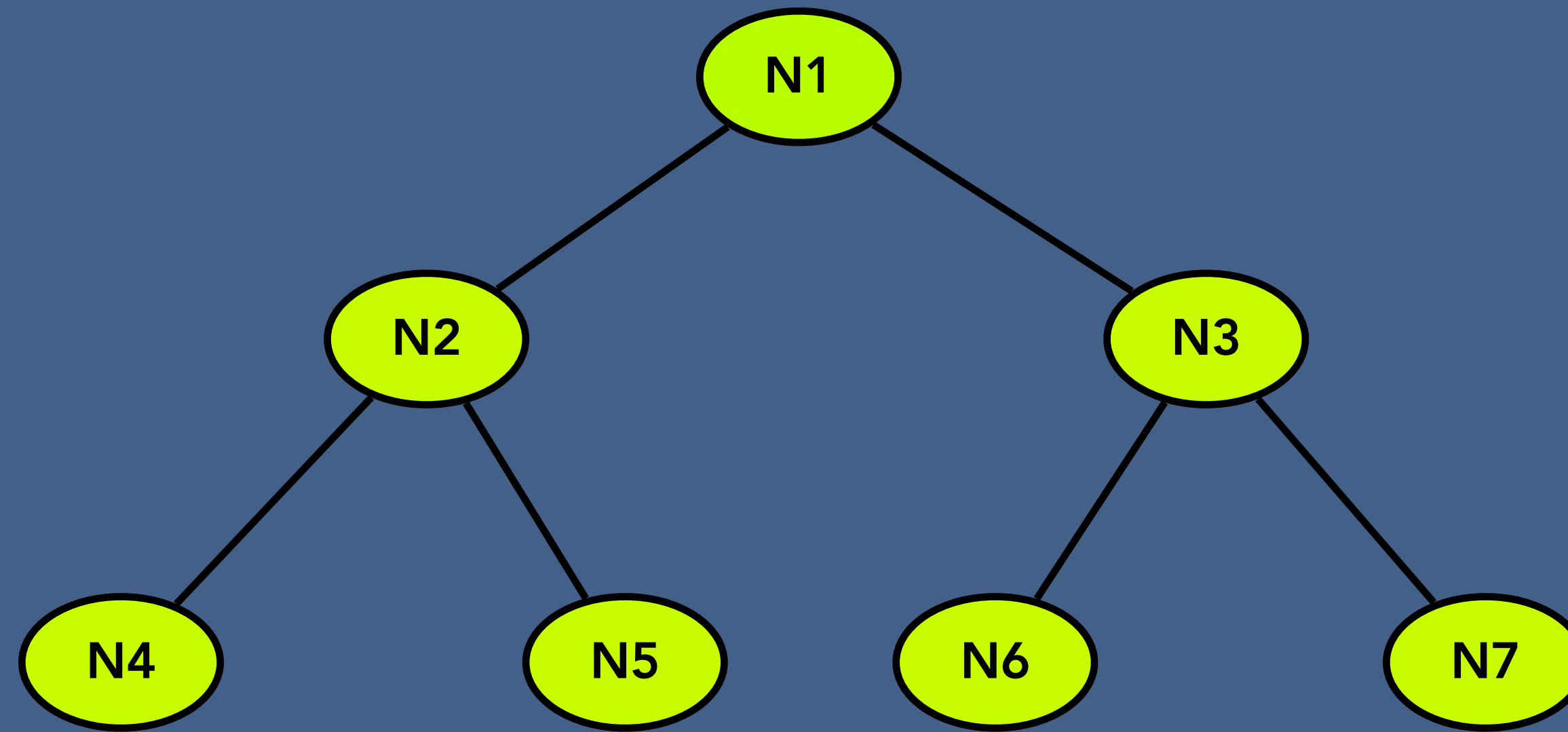


Left child = $\text{cell}[2x]$ $\longrightarrow$ $x = 1$, $\text{cell}[2 \times 1 = 2]$

Right child = $\text{cell}[2x+1]$ $\longrightarrow$ $x = 1$, $\text{cell}[2 \times 1 + 1 = 3]$

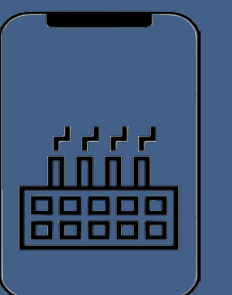


Binary Tree using Array



Left child = $\text{cell}[2x]$ $\longrightarrow$ $x = 2$, $\text{cell}[2 \times 2 = 4]$

Right child = $\text{cell}[2x+1]$ $\longrightarrow$ $x = 2$, $\text{cell}[2 \times 2 + 1 = 5]$

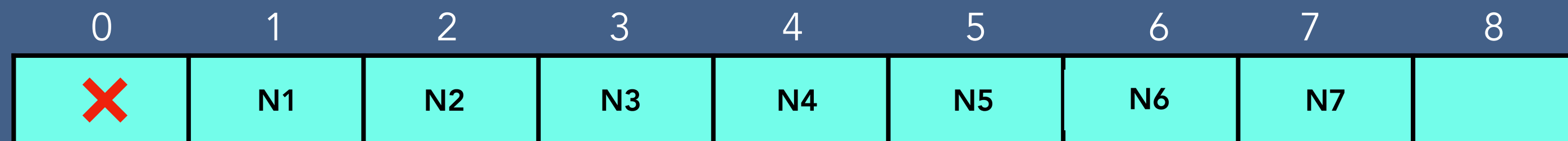
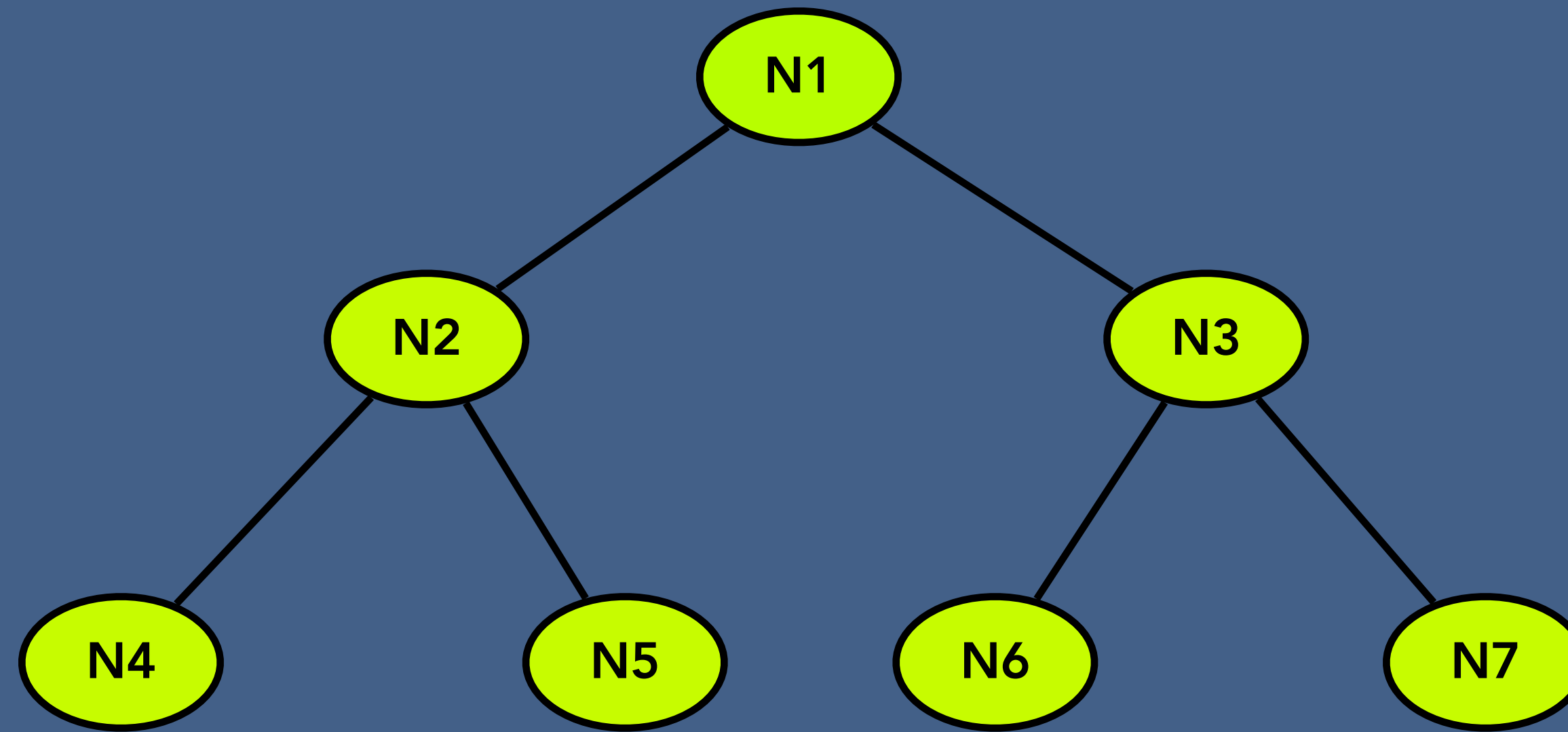


Binary Tree using Array

`newBT = BTclass()`

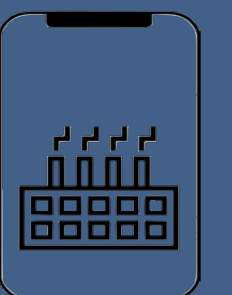
Fixed size Array

lastUsedIndex



Left child = $\text{cell}[2x]$ $\longrightarrow$ $x = 3$, $\text{cell}[2 \times 3 = 6]$

Right child = $\text{cell}[2x+1]$ $\longrightarrow$ $x = 3$, $\text{cell}[2 \times 3 + 1 = 7]$



Binary Tree (Array) - Insert a Node

- The Binary Tree is full
- We have to look for a first vacant place

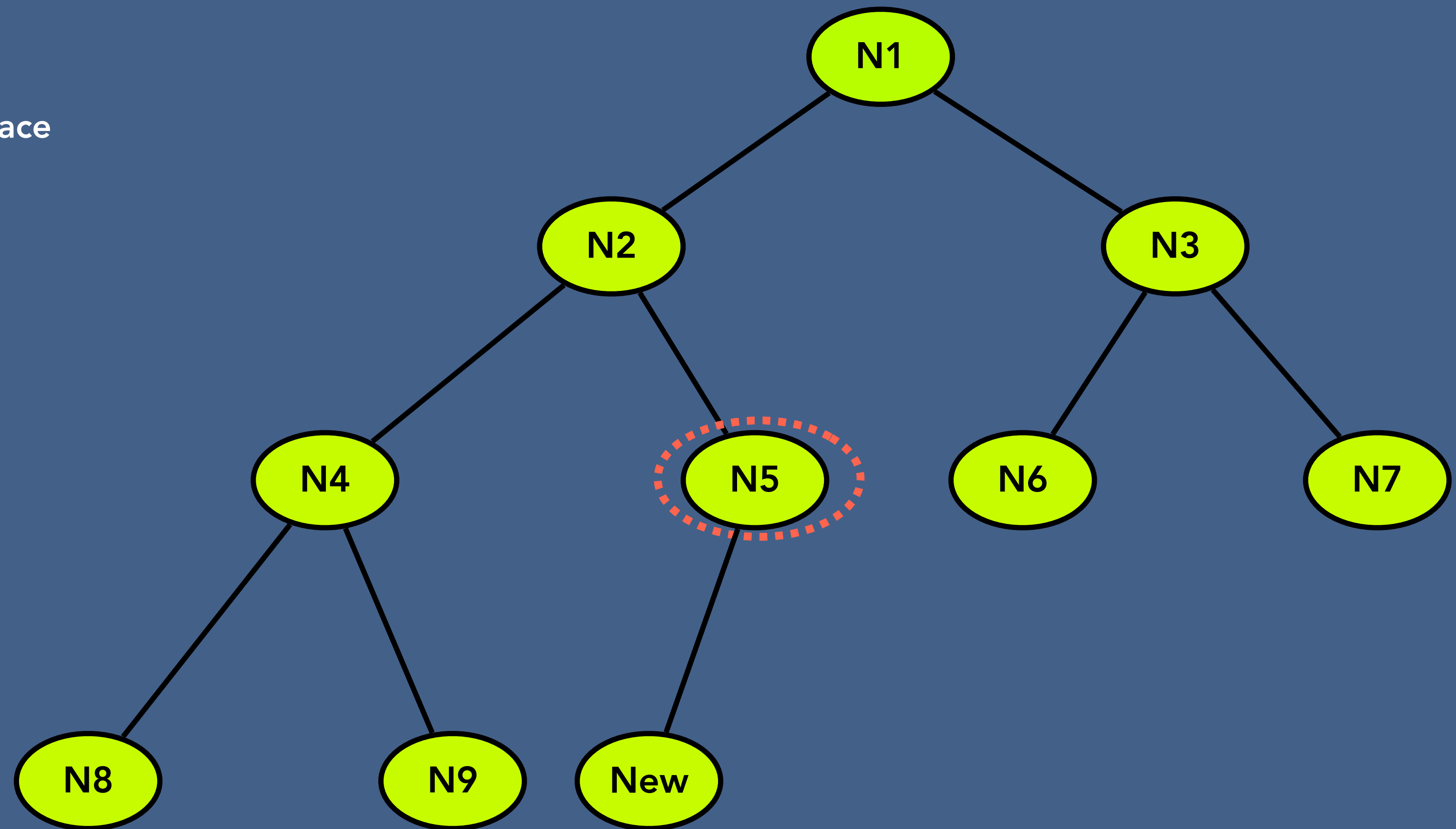
lastUsedIndex = 9

newNode

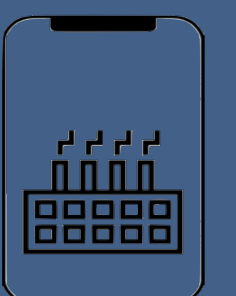
Index = 10

Left child = cell[2x]

indexOfParent = $10/2 = 5$



0	1	2	3	4	5	6	7	8	9	10	11
×	N1	N2	N3	N4	N5	N6	N7	N8	N9	New	



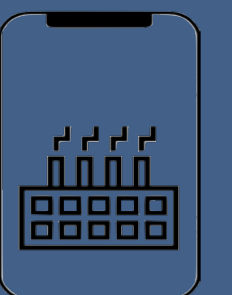
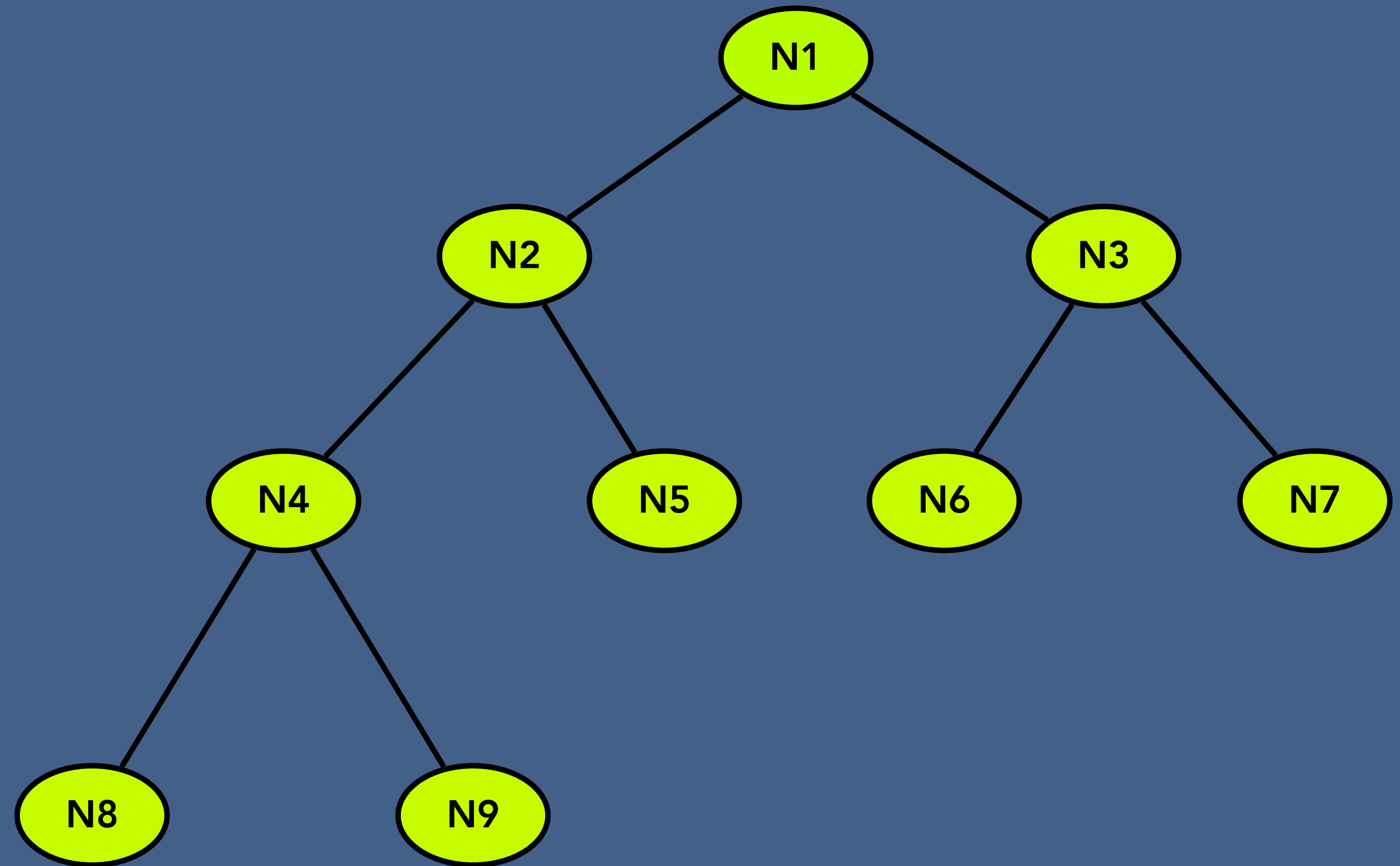
Binary Tree (Array) - Traversal

Depth first search

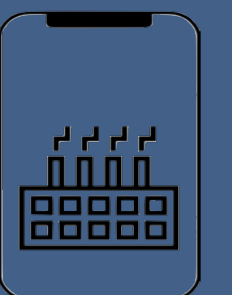
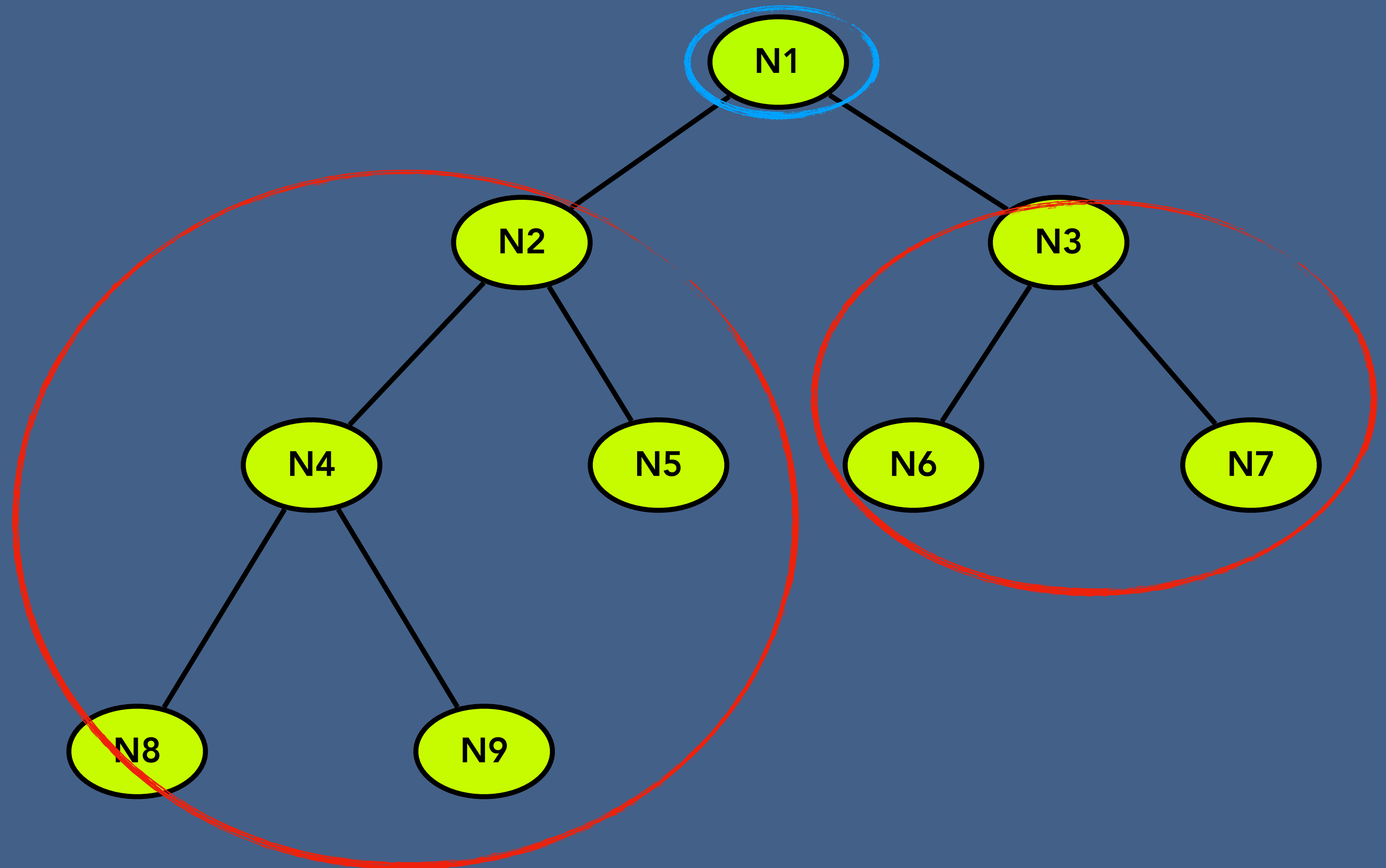
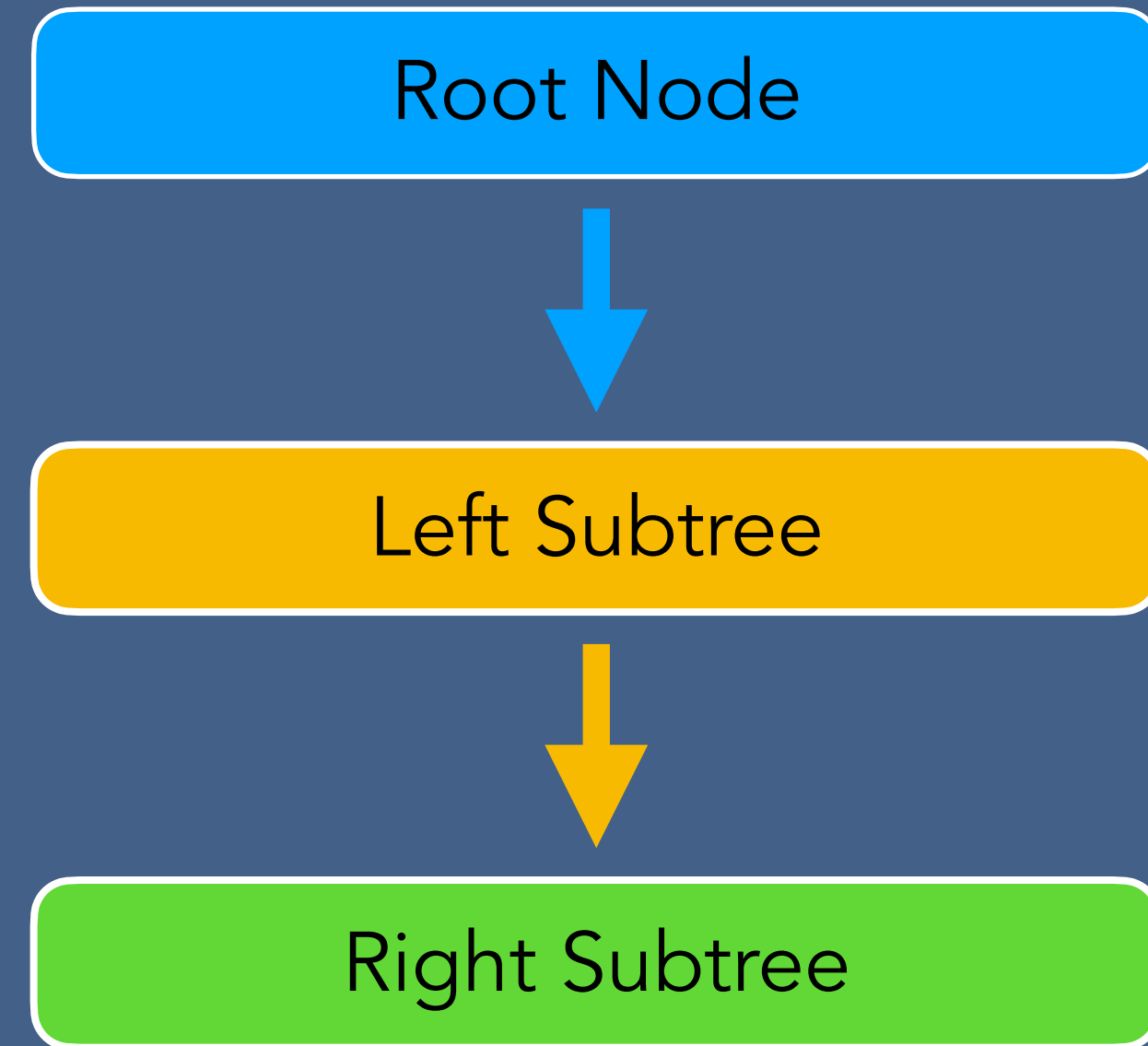
- Preorder traversal
- Inorder traversal
- Post order traversal

Breadth first search

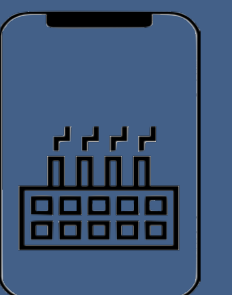
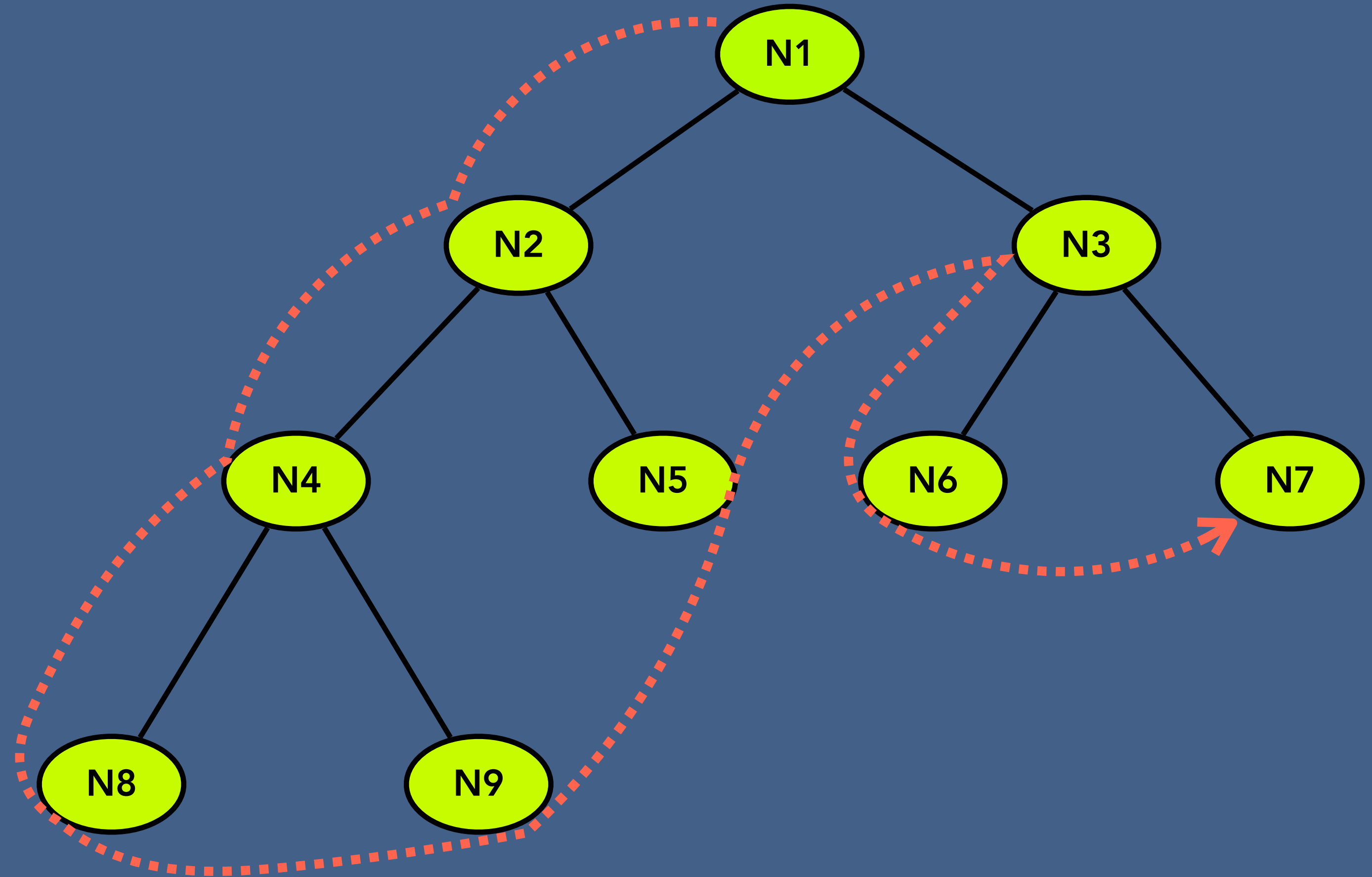
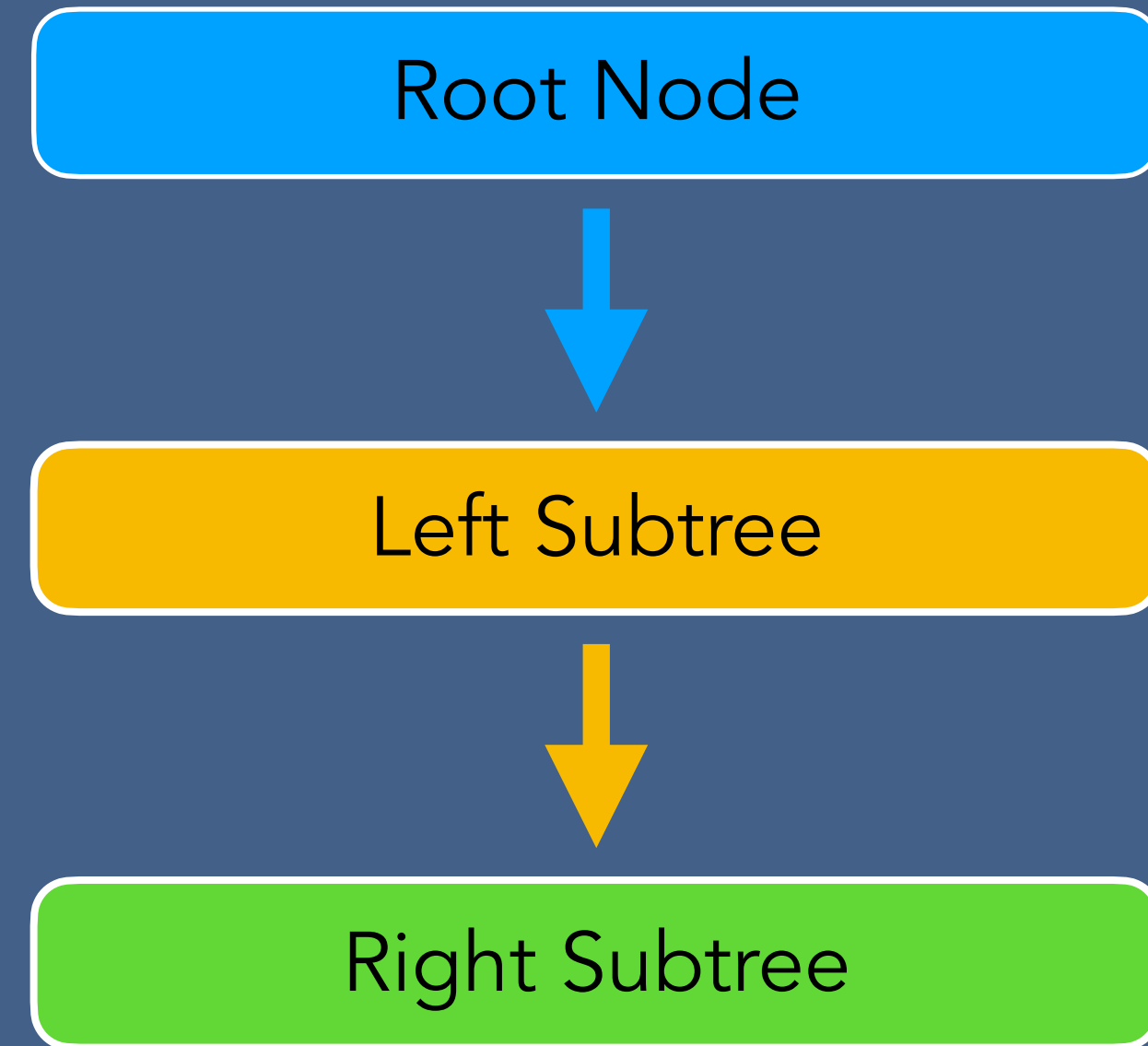
- Level order traversal



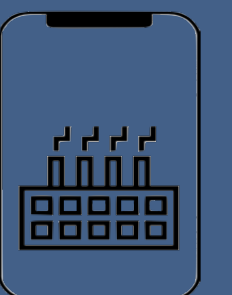
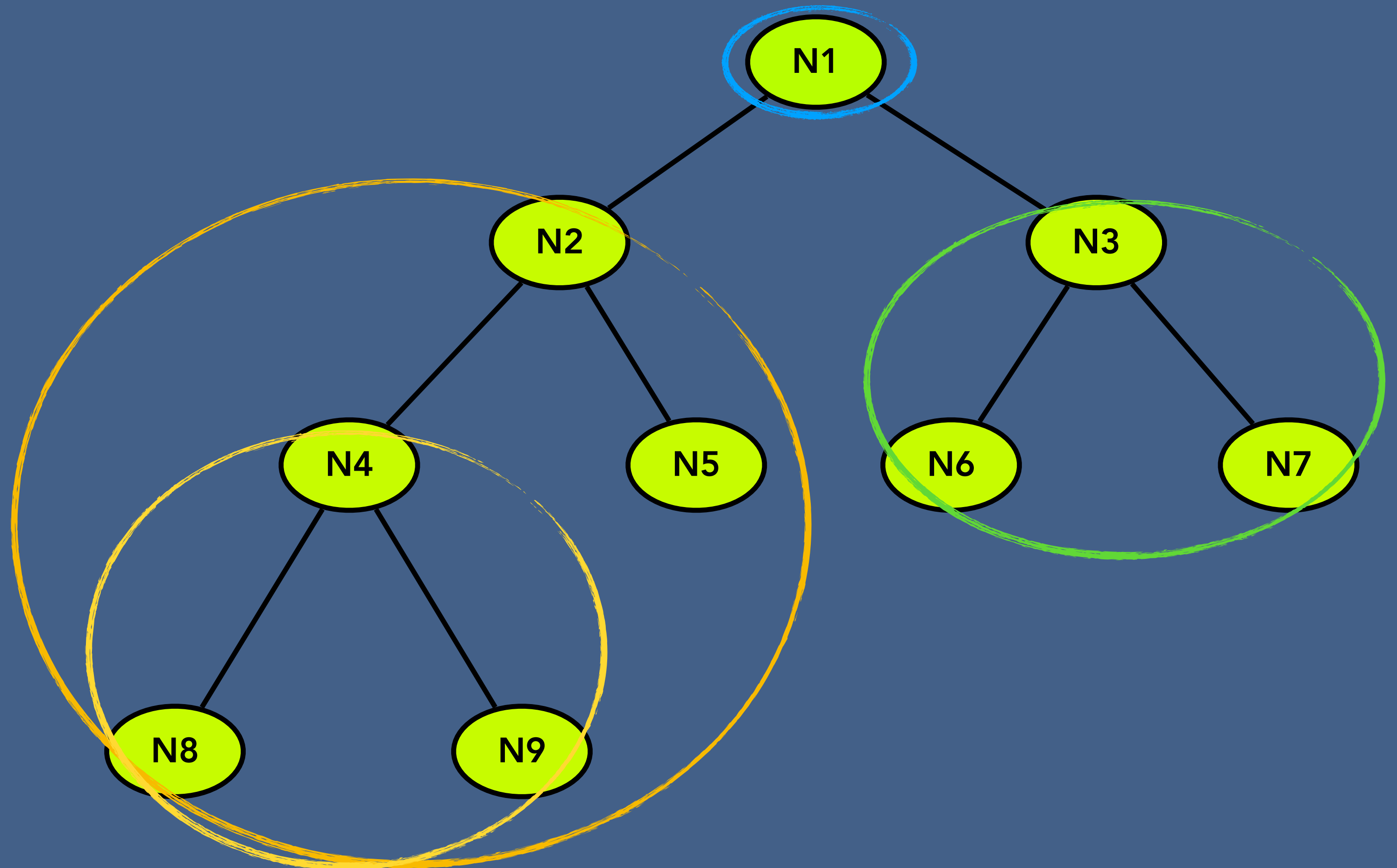
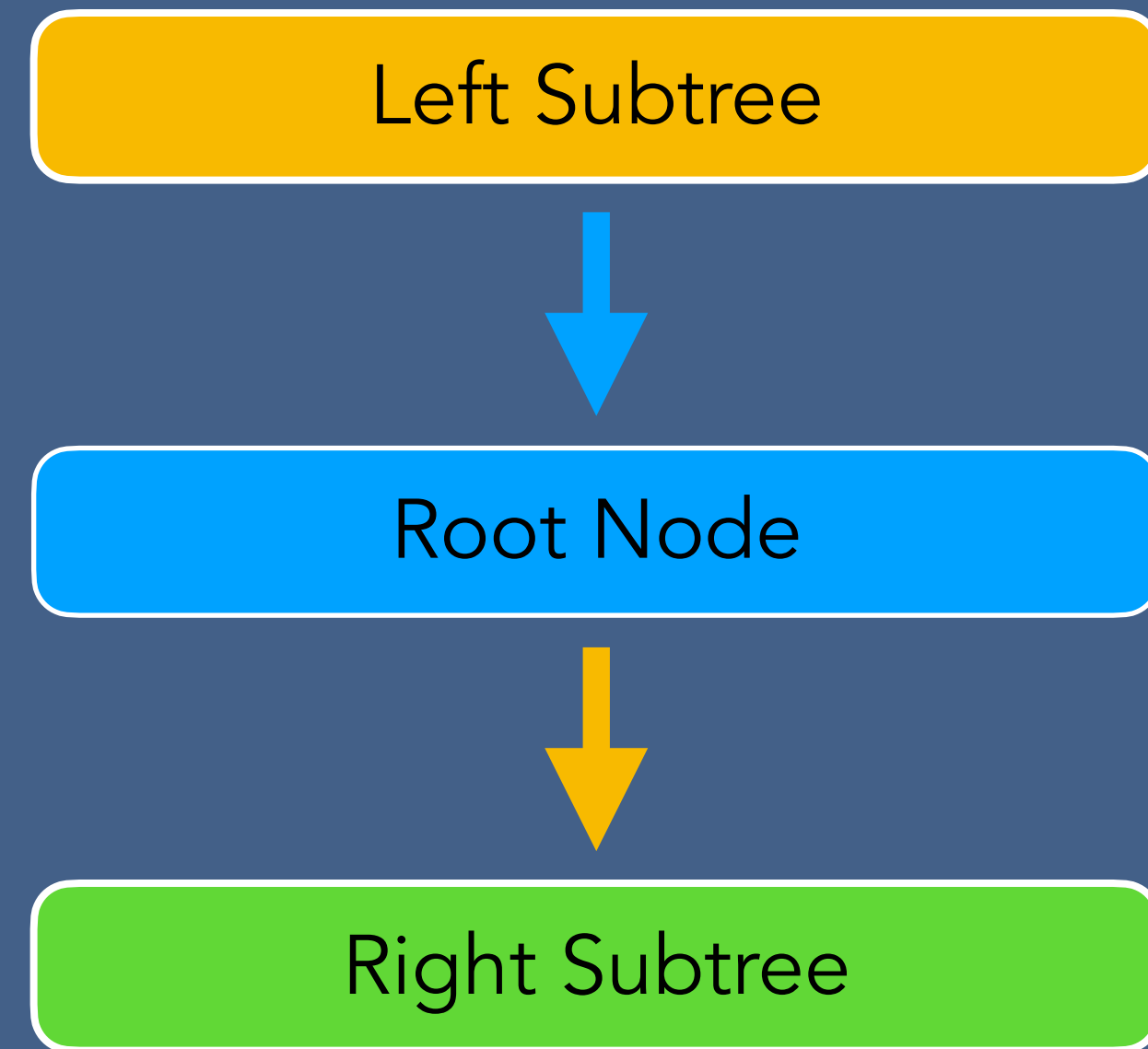
Binary Tree (Array) - PreOrder Traversal



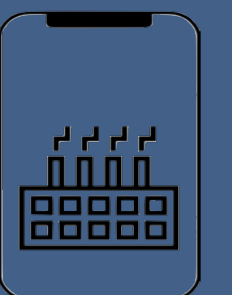
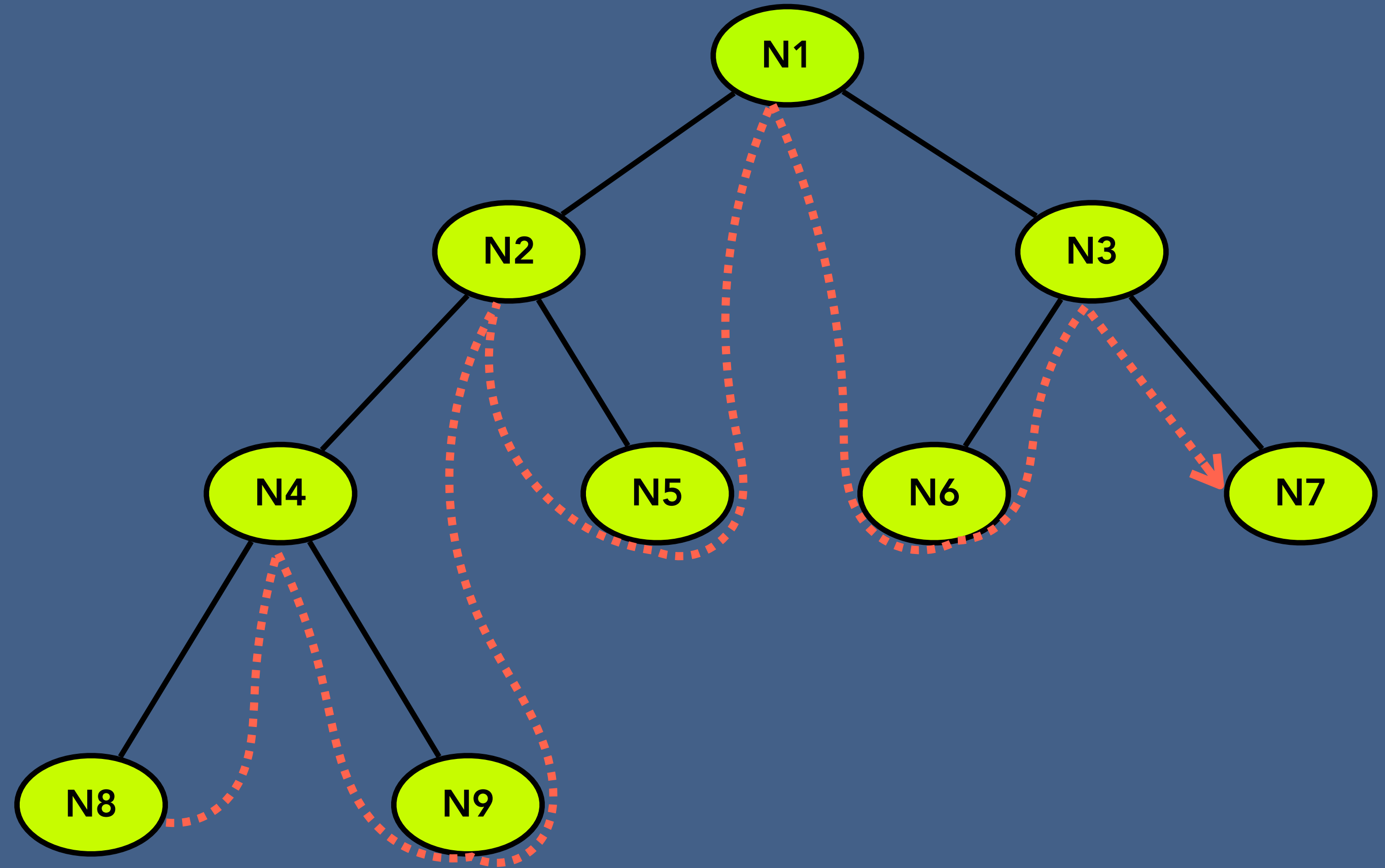
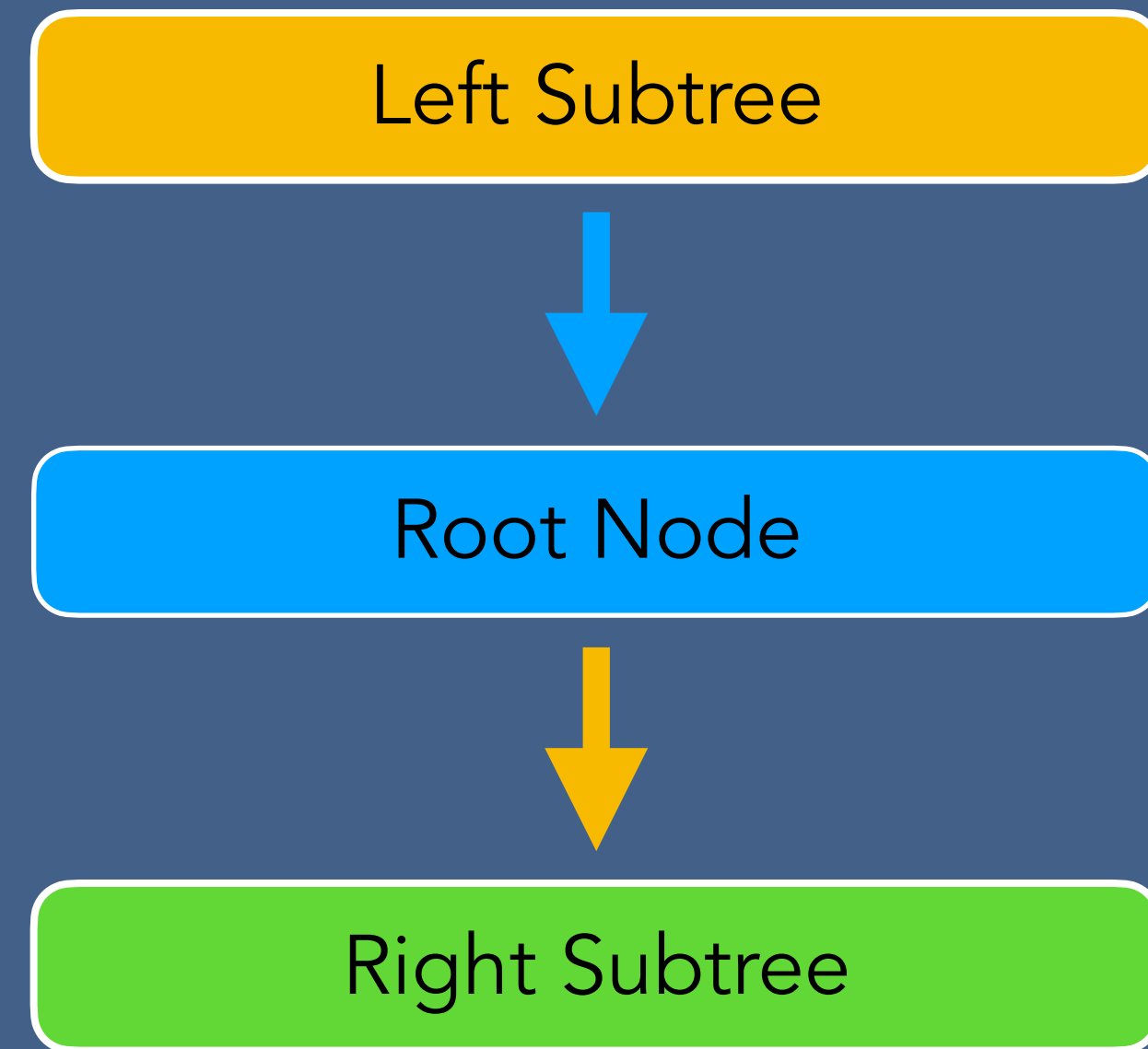
Binary Tree (Array) - PreOrder Traversal



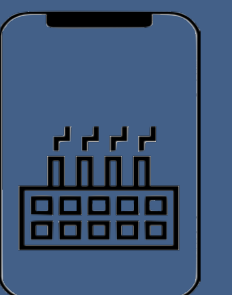
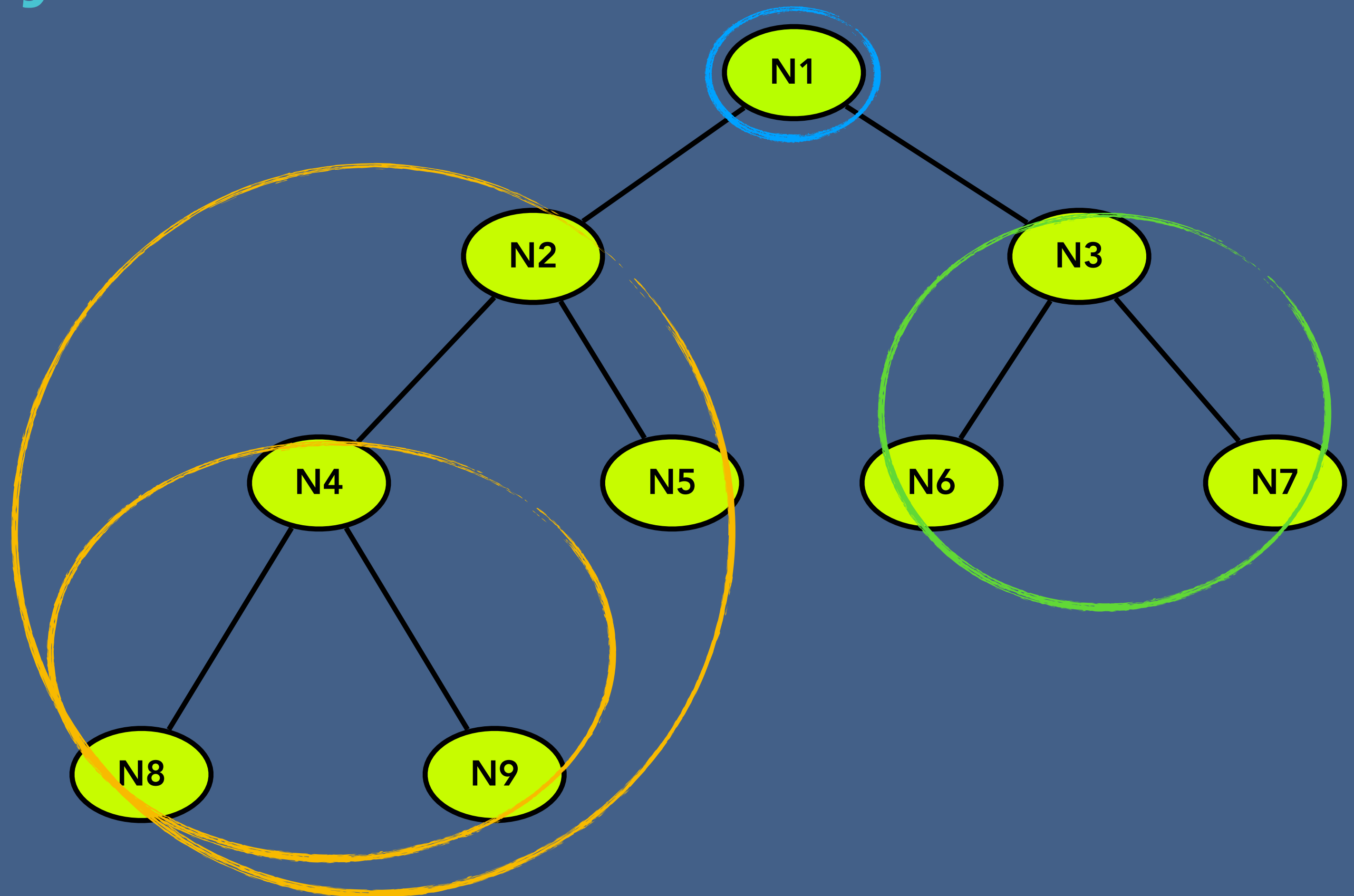
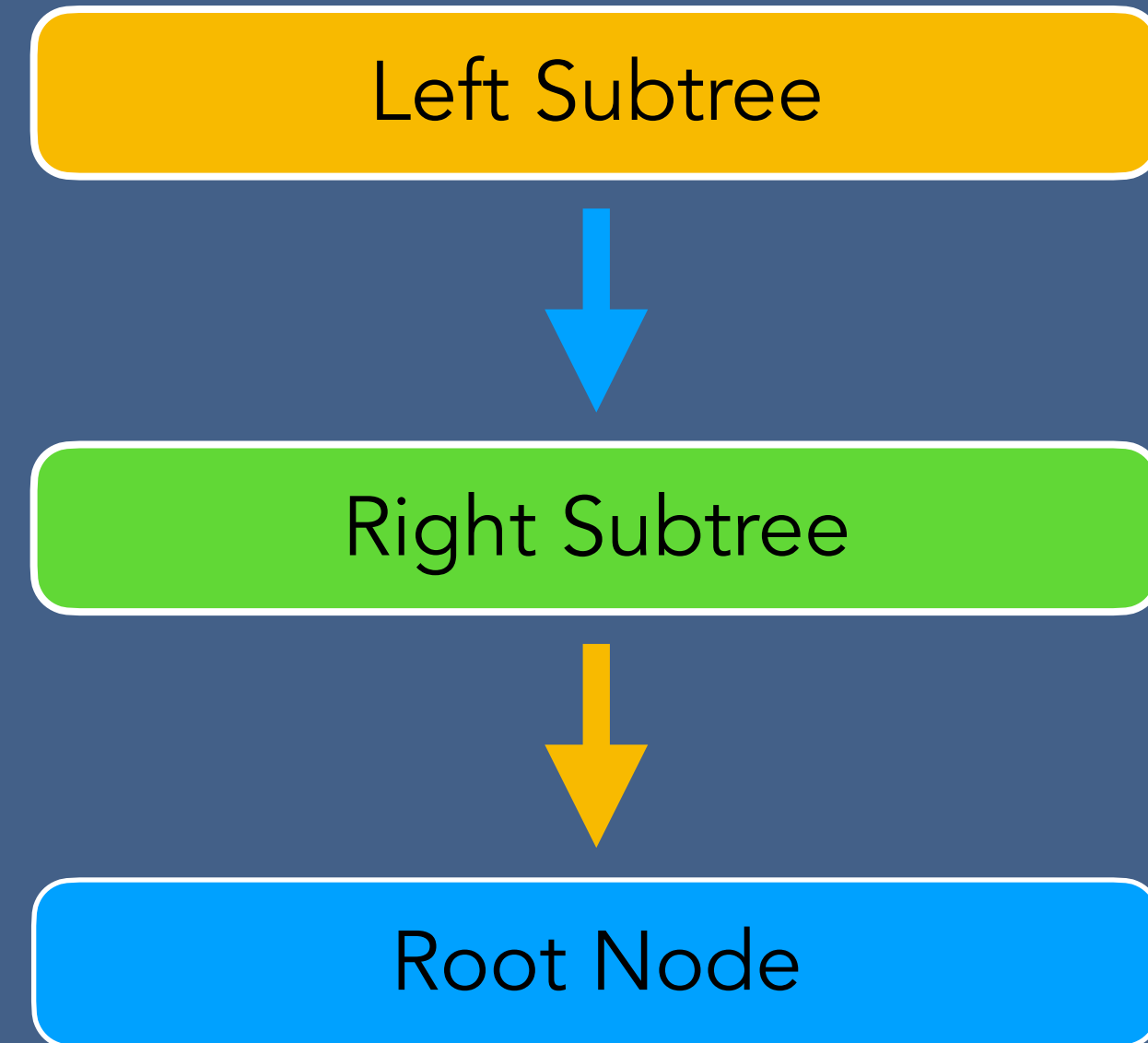
Binary Tree (Array) - InOrder Traversal



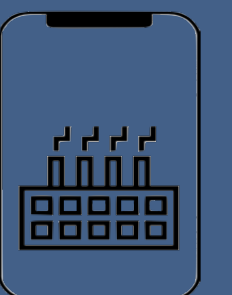
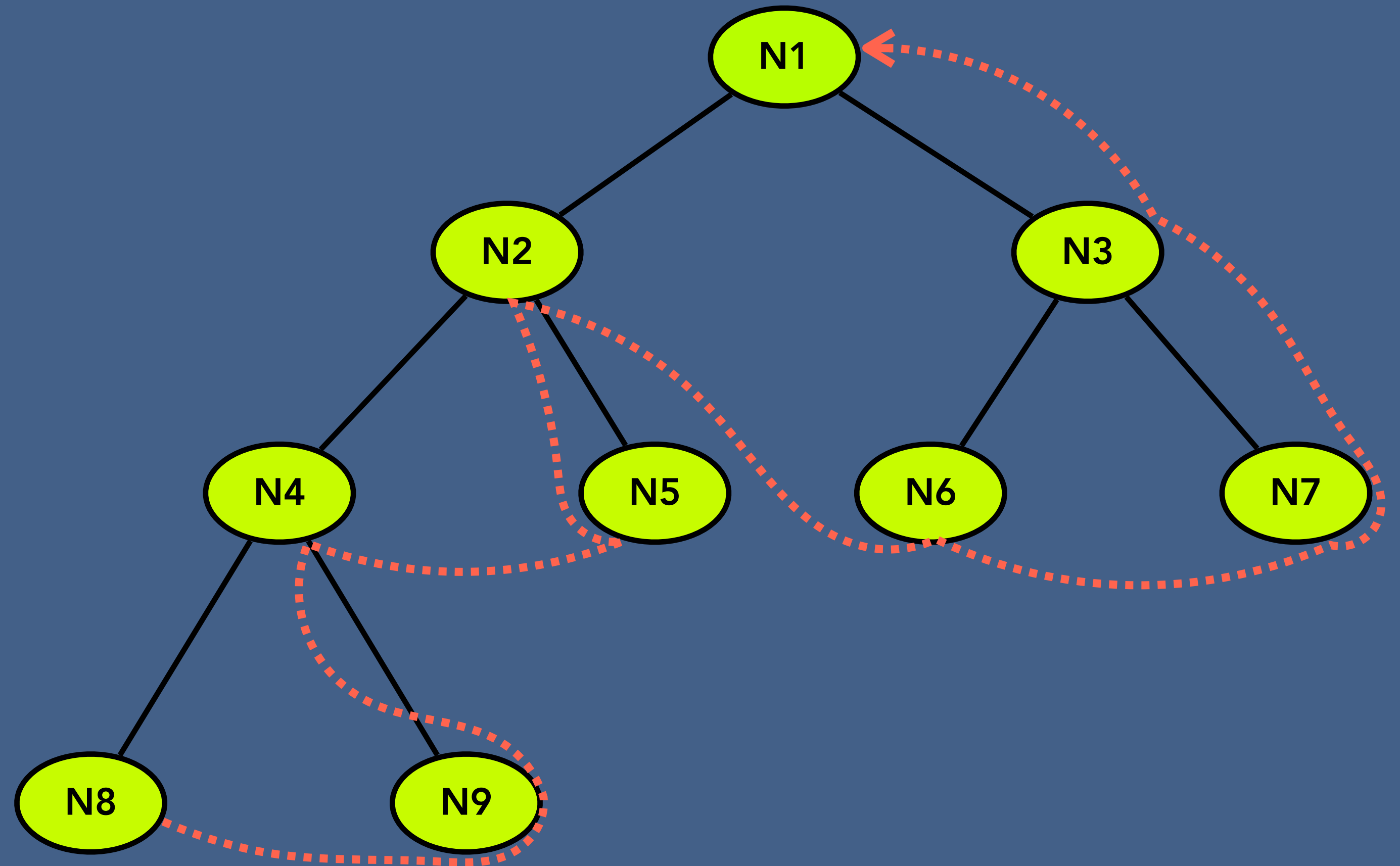
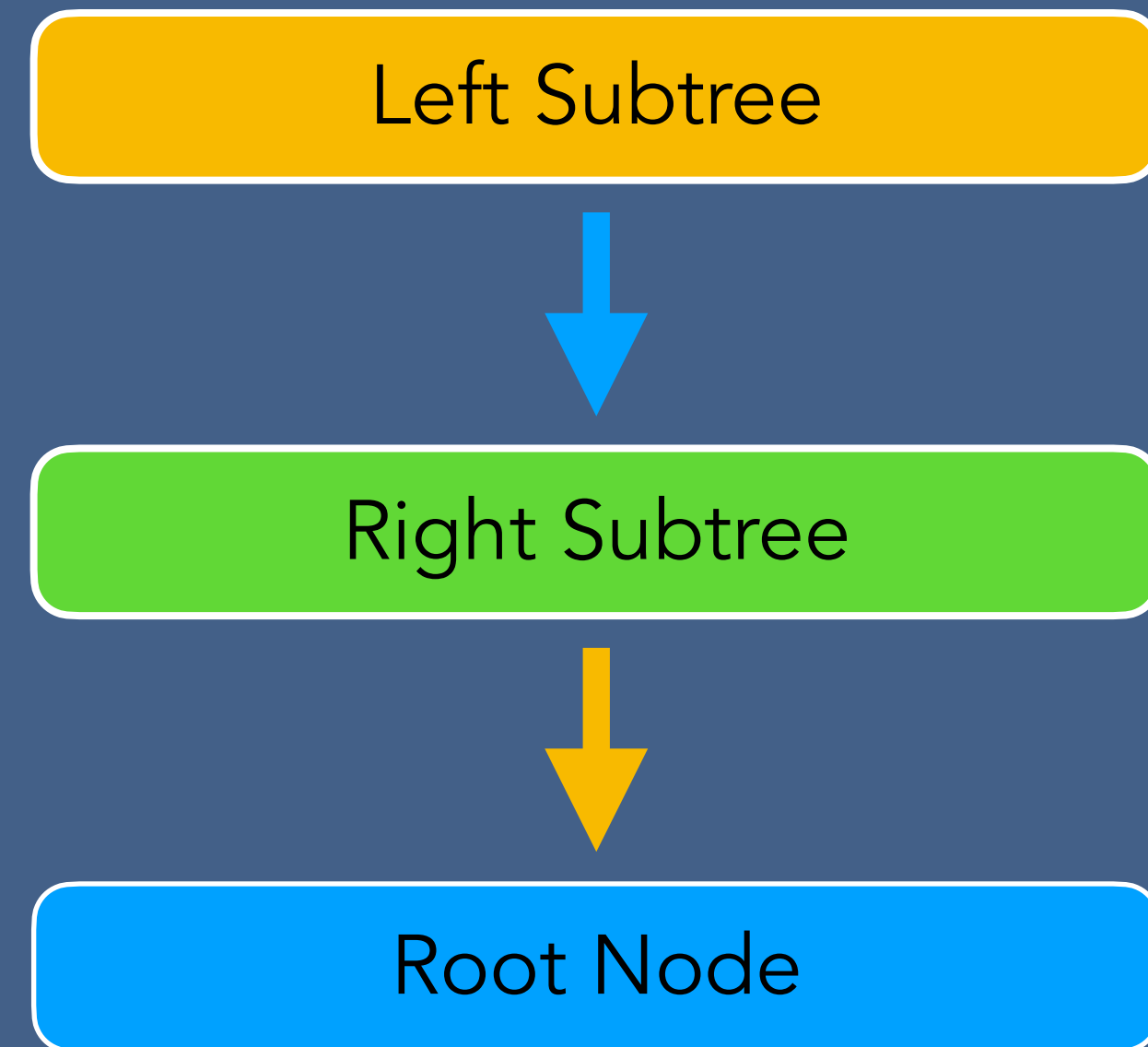
Binary Tree (Array) - InOrder Traversal



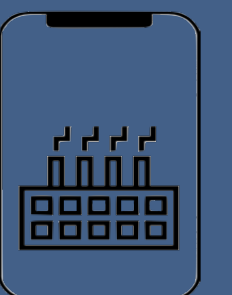
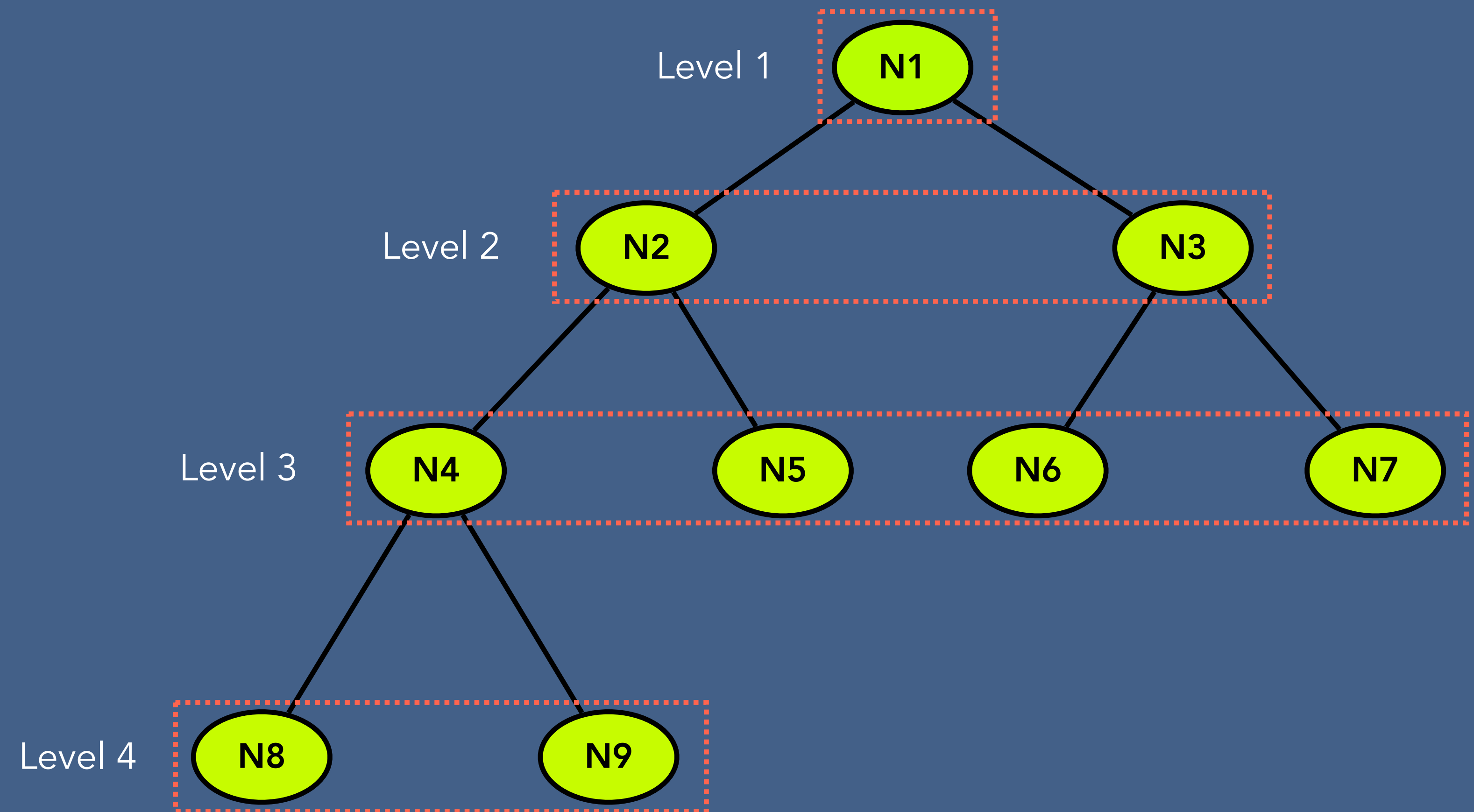
Binary Tree (Array) - Post Traversal



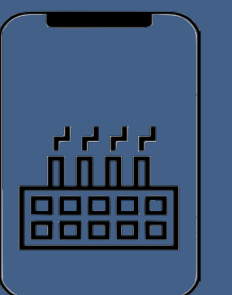
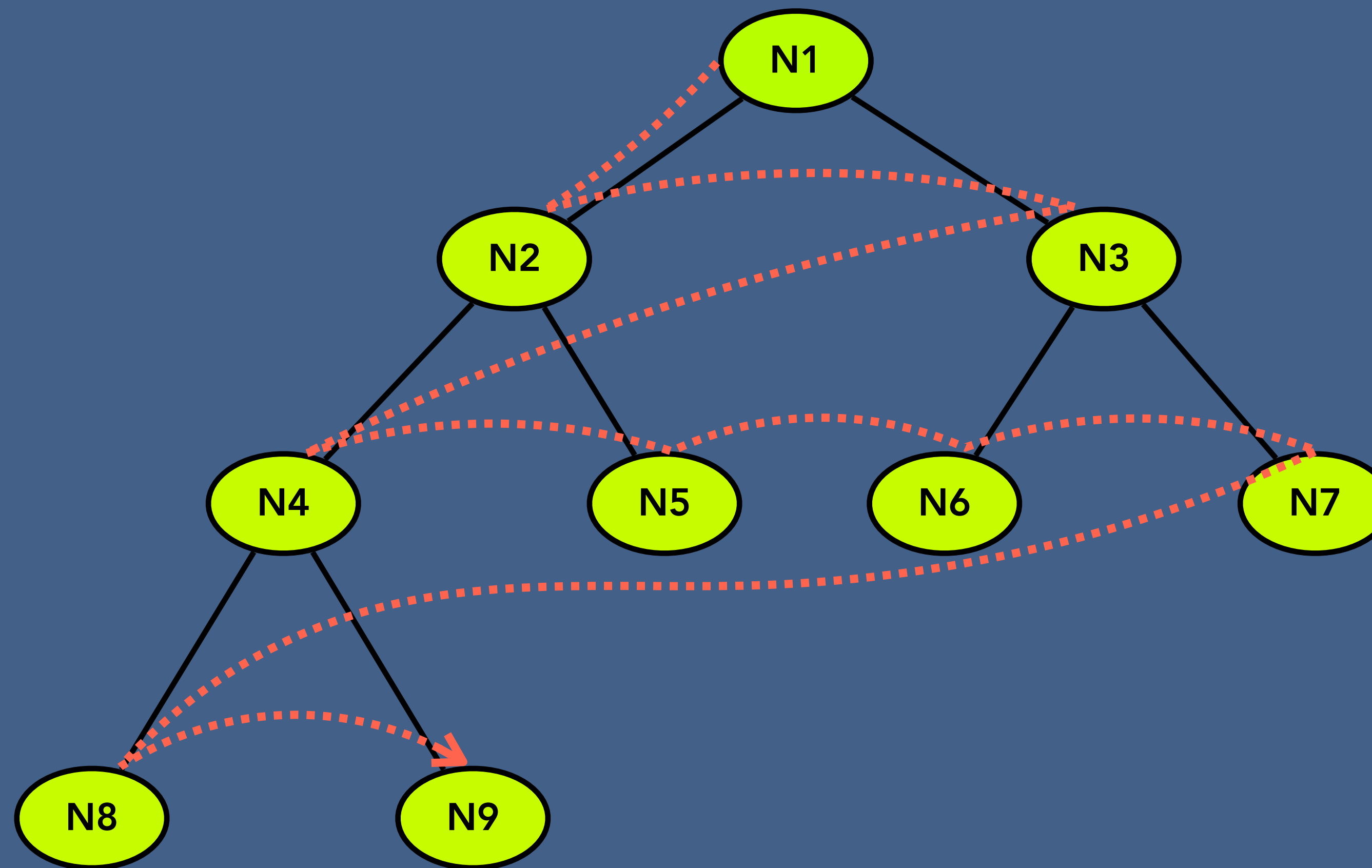
Binary Tree (Array) - Post Traversal



Binary Tree (Array) - LevelOrder Traversal



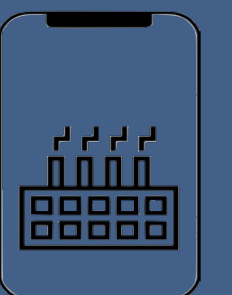
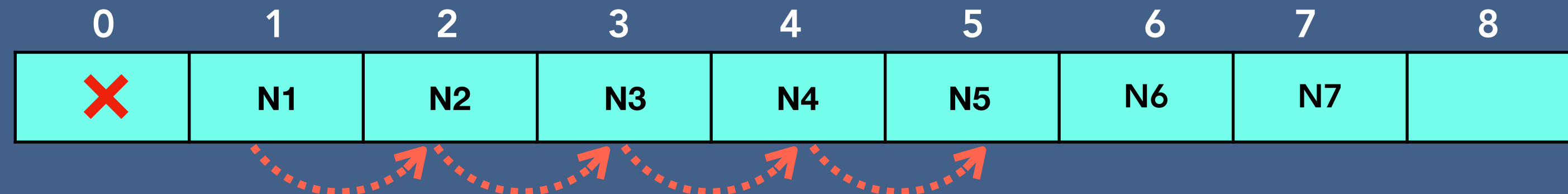
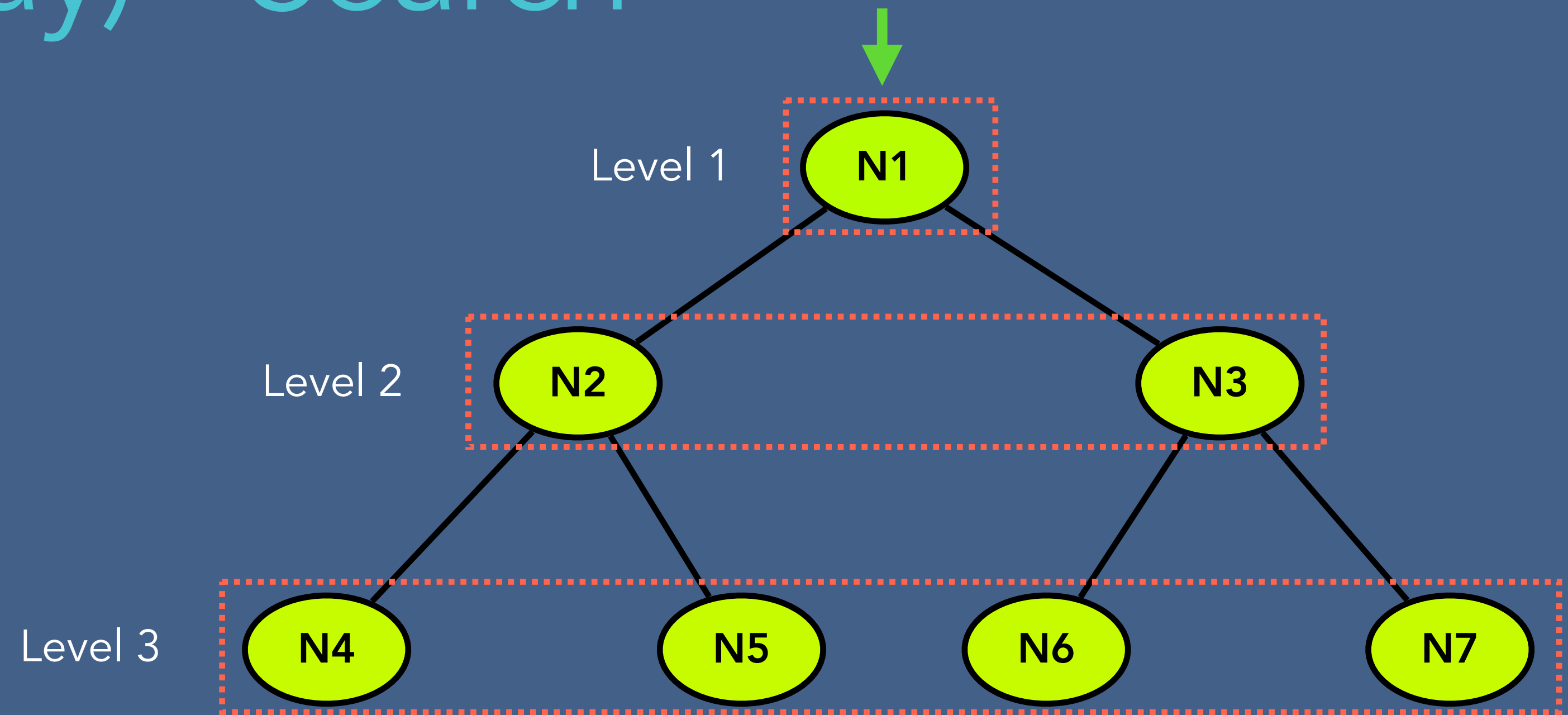
Binary Tree - LevelOrder Traversal



Binary Tree (Array) - Search

Level Order Traversal

N5 → 5

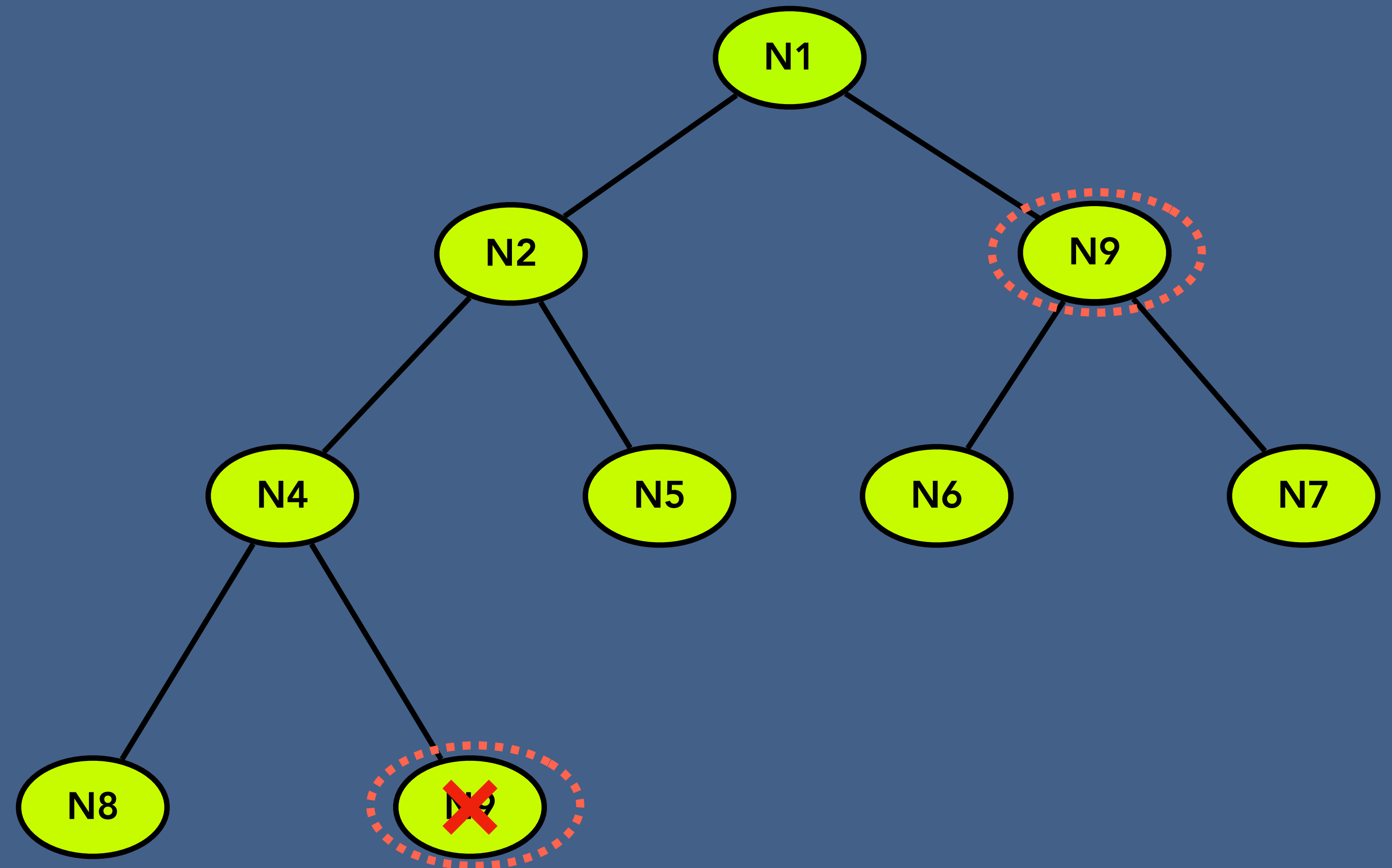


Binary Tree (Array) - Delete a Node

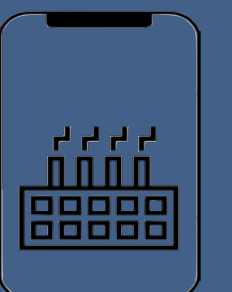
Level Order Traversal

N3

deepestNode = lastUsedIndex

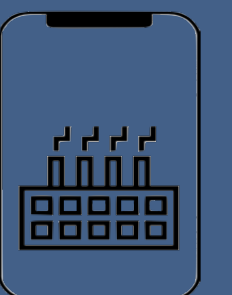
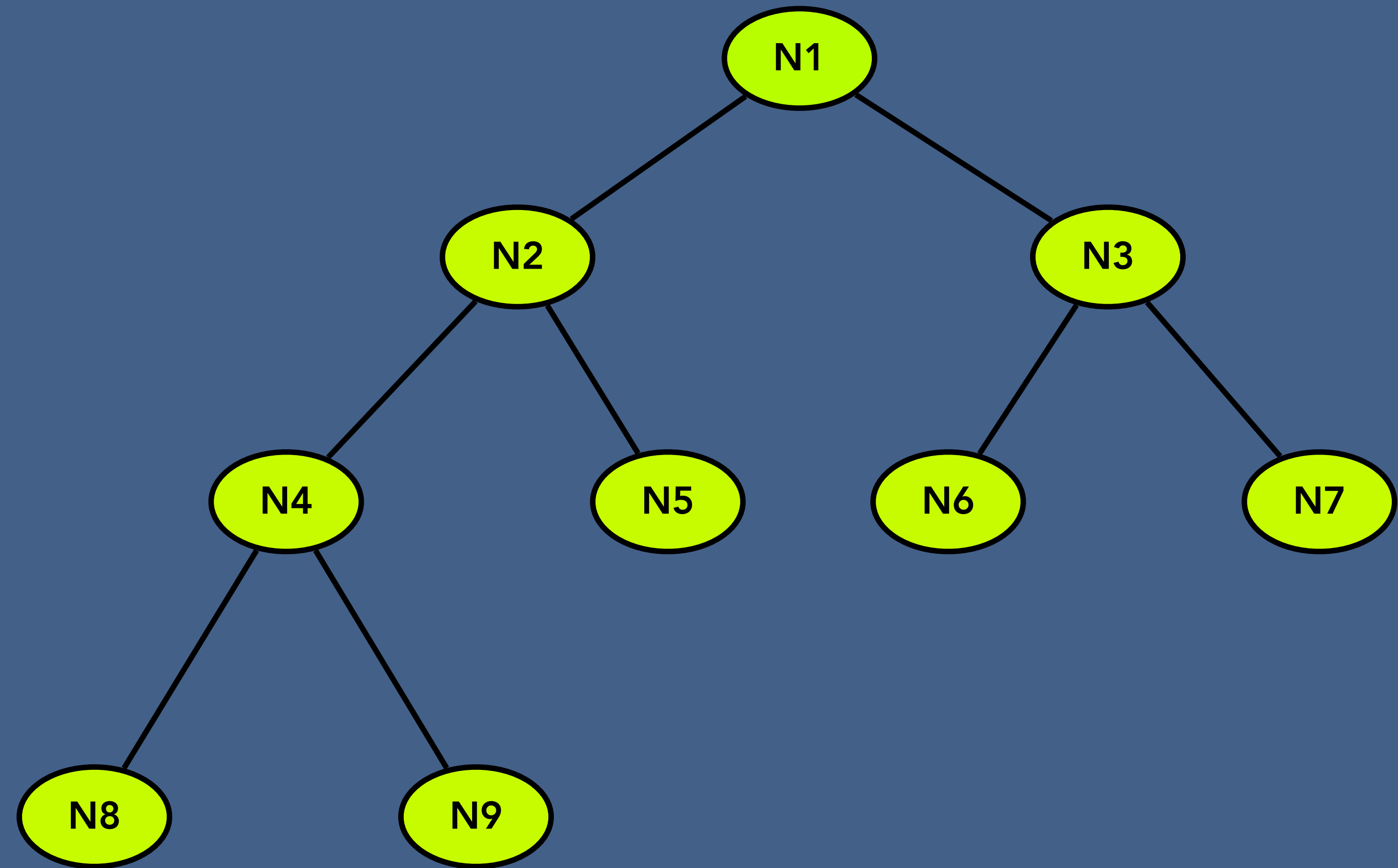


×	N1	N2	N9	N4	N5	N6	N7	N8			
0	1	2	3	4	5	6	7	8	9	10	11



Binary Tree (Array) - Delete Binary Tree

arr = Null



Binary Tree (Array vs Linked List)

	Array		Linked List	
	Time complexity	Space complexity	Time complexity	Space complexity
Create Binary Tree	$O(1)$	$O(n)$	$O(1)$	$O(1)$
Insert a node to Binary Tree	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Delete a node from Binary Tree	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Search for a node in Binary Tree	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Traverse Binary Tree	$O(n)$	$O(1)/O(n)$	$O(n)$	$O(n)$
Delete entire Binary Tree	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Space efficient?	No		Yes	

